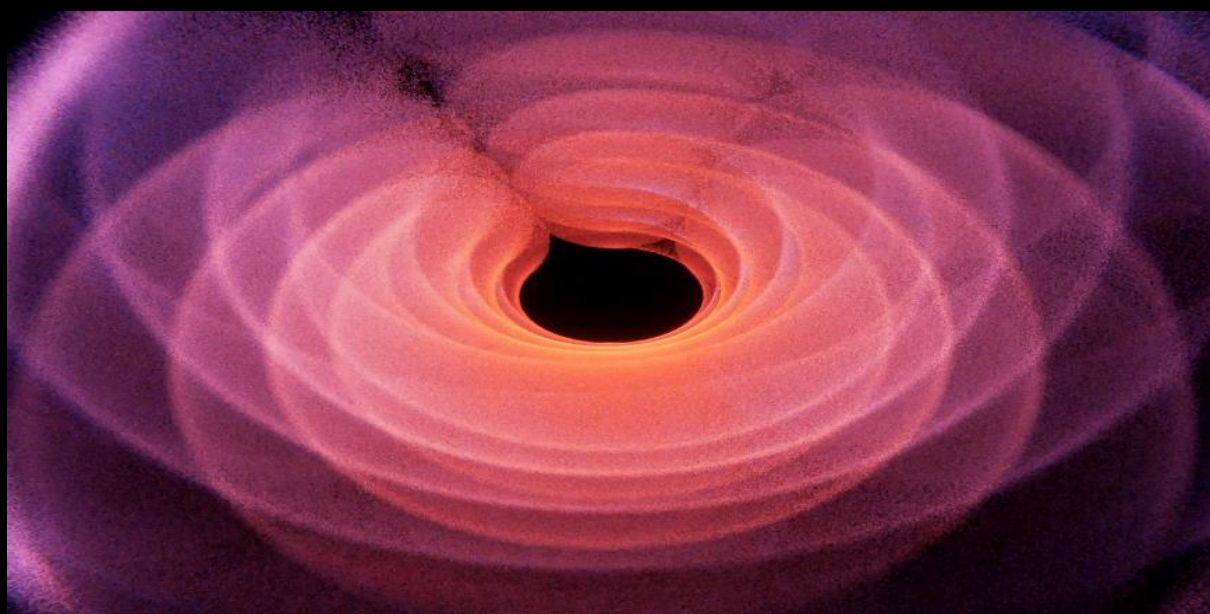


О.В. Гаркуша

UNITY в картинках



Краснодар
2024

Министерство науки и высшего образования
Российской Федерации
КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

О.В. ГАРКУША

UNITY В КАРТИНКАХ

Учебное пособие

Краснодар
2024

УДК 004.9(075.8)
ББК 32.973.39я73
Г 204

Рецензенты:

Доктор физико-математических наук, профессор
Е.Н. Калайдин

Кандидат физико-математических наук, доцент
С.Е. Рубцов

Гаркуша, О.В.

Г 204 UNITY в картинках: учебное пособие / О.В. Гаркуша;
Министерство науки и высшего образования Российской
Федерации, Кубанский государственный университет. –
Краснодар: Кубанский гос. ун-т, 2024. – 167 с. – 500 экз.
ISBN 978-5-8209-2513-9

Содержит материал о возможностях использования
кроссплатформенной системы Unity для проектирования и
создания игр, включая интерфейсы для графики.

Адресуется студентам III курса направления бакалавриата
01.03.02 «Прикладная математика и информатика», 02.03.03
«Математическое обеспечение и администрирование
информационных систем», направления бакалавриата 02.03.02
«Фундаментальные информатика и информационные
технологии», 09.03.03 «Прикладная информатика», а также
магистрантам и аспирантам для изучения основ проектирования
систем в игровой форме.

УДК 004.9(075.8)
ББК 32.973.39я73

ISBN 978-5-8209-2513-9

© Кубанский государственный
университет, 2024
© Гаркуша О.В., 2024

ВВЕДЕНИЕ

Компьютерные игры давно стали частью нашей культуры, через них можно донести мысль до широких масс, они развивают интеллект, реакцию и позволяют пережить целый спектр самых разнообразных эмоций.

Unity – это кроссплатформенный игровой движок, изначально выпущенный Unity Technologies в 2005 г. Основное внимание Unity уделяет разработке как 2D-, так и 3D- игр и интерактивного контента. Unity теперь поддерживает более **20** различных целевых платформ для развертывания, в то время как его наиболее популярными платформами являются системы ПК, Android и iOS.

Unity предоставляет полный набор инструментов для проектирования и создания игр, включая интерфейсы для графики, аудио и инструменты для создания уровней, требующие минимального использования внешних программ для работы над проектами.

1. ЧТО ТАКОЕ ИГРА

Прежде чем углубиться в проектирование и итерации, следует уточнить, что подразумевается под такими терминами, как *игра* и проектирование игры. Многие очень умные люди пытались дать точное определение слову игра:

«Игра – это добровольная попытка преодоления ненужных препятствий». *Бернард Сьюте.*

«Игра – это серия интересных решений». *Сид Мейер.*

«Игра – закрытая формальная система, которая вовлекает игроков в организованный конфликт и разрешает его неопределенность в неравноценности результатов». *Трейси Фуллертон.*

«Игра – это работа по решению проблем с игровым настроением». *Джесси Шелл.*

«Игра – система правил, руководствуясь которыми агенты конкурируют друг с другом, делая неоднозначные, эндогенно значимые решения». *Кит Бургун.*

Блестящими дизайнерами игр не рождаются, ими становятся. Крис Свейн говорил: «Проектирование игр – это 1 % вдохновения

и 99% итераций», – перефразируя известную цитату Томаса Эдисона.

Одна из замечательных особенностей проектирования игр – шанс создать свой дизайн, сыграть пробную игру, внести мелкие изменения и снова сыграть. С каждым прототипом и с каждой итерацией по улучшению прототипа вы улучшаете свои навыки дизайнера.

При создании собственного проекта важно помнить, что разработка кода – это примерно 20% игры; гораздо большее значение в ней имеют другие аспекты:

- геймплей – общее понятие взаимодействия игрока с игровым миром, которое определяет его действия (бежать вперед, преодолевать препятствия, стрелять по мишеням, обгонять других) и цели (прийти первым к финишу, выбить 10 из 10, победить врага в бою, собрать как можно больше монет). Геймплей напрямую связан с жанром игры, так как у каждого из них есть специфичный набор правил и механик;

- игровые механики – конкретные элементы взаимодействия с игрой, которые входят в состав геймплея. Стрельба – это одна механика, сражение на мечах – другая, гонка – третья. Одна игра может сочетать в себе десятки таких механик;

- сюжет – это развитие действия в игре; он должен затянуть игрока, рассказать ему историю, а также показывать персонажей в развитии, чтобы они раскрывались для игрока с новых сторон;

- персонажи – в них важны и дизайн, и характер. Удачно проработанный персонаж обладает узнаваемыми особенностями поведения, интересной историей, а еще для полного погружения он должен иметь что-то общее с игроком, что зацепит его и заставит сопереживать. На эту тему Unity разработали гайд «Пять типов привлекательных игровых персонажей», чтобы у новичков получалось сделать игрового персонажа правдоподобным;

- гайд – инструкция, написанная чаще всего опытным игроком;

- дизайн уровней – внешний вид игры, цветовые решения, общая стилистика объектов, фона, персонажей, предметов, которая создает определенное настроение;

- баланс – соотношение характеристик разных объектов, он тоже отвечает за увлеченность игрока. Например, если меч в игре

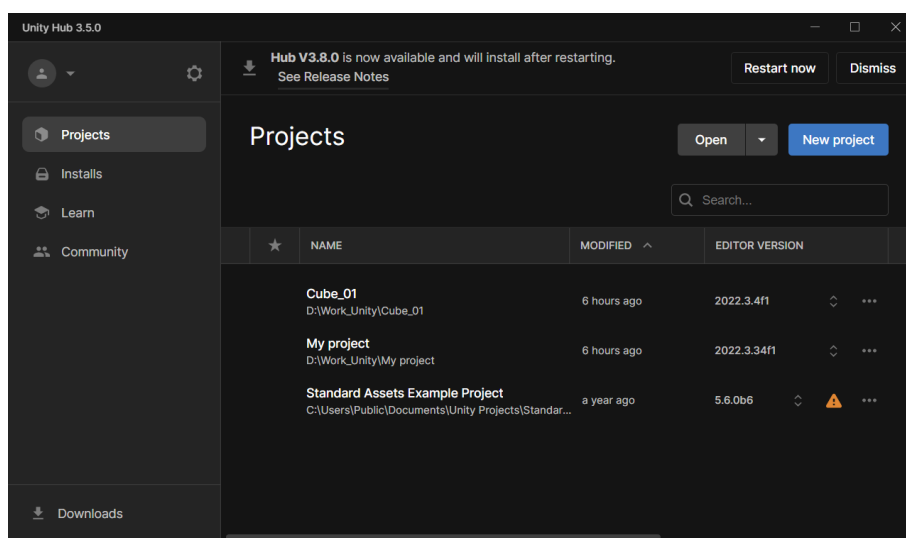
может наносить объекту 3 единицы урона, а объект имеет всего 3 HP (hit points – величина, обозначающая максимальный урон), то его можно уничтожить с первого раза, и играть будет слишком легко. Если объект имеет 30 HP, то игроку придется нанести 10 ударов, чтобы его уничтожить. Такое уже подходит скорее для босса, например, на первом или втором уровне. Разработчику важно грамотно распределить эти величины, чтобы игра была увлекательной и бросала игроку вызовы.

2. СРЕДА РАЗРАБОТКИ UNITY

2.1. ИНСТРУМЕНТЫ РАЗРАБОТКИ

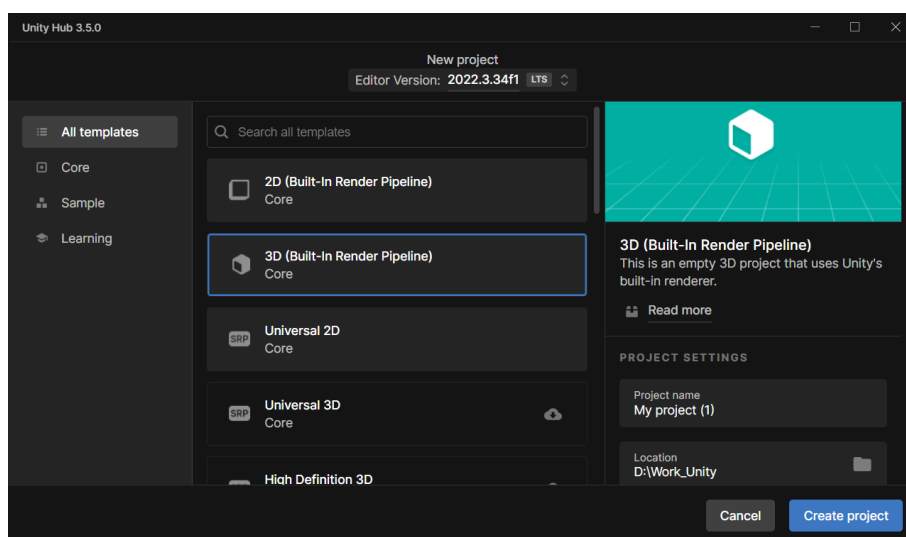
Чтобы создать игру, нужны инструменты разработки. В качестве основного инструмента мы будем использовать Unity, а для написания программного кода понадобится среда разработки, например, Microsoft Visual Studio. Unity – межплатформенная среда разработки компьютерных игр, которую выпустила и активно продвигает американская компания Unity Technologies. Unity позволяет создавать приложения, работающие на более чем 25 различных платформах, включающих персональные компьютеры, игровые консоли, мобильные устройства, интернет-приложения и др.

Unity одинаково подходит как для 2D-, так и для 3D-игр. Все игры, сделанные в Unity, начинаются как Проекты с загрузочного экрана.



После нажатия на иконку «New project» вы увидите экран «Настройка проекта».

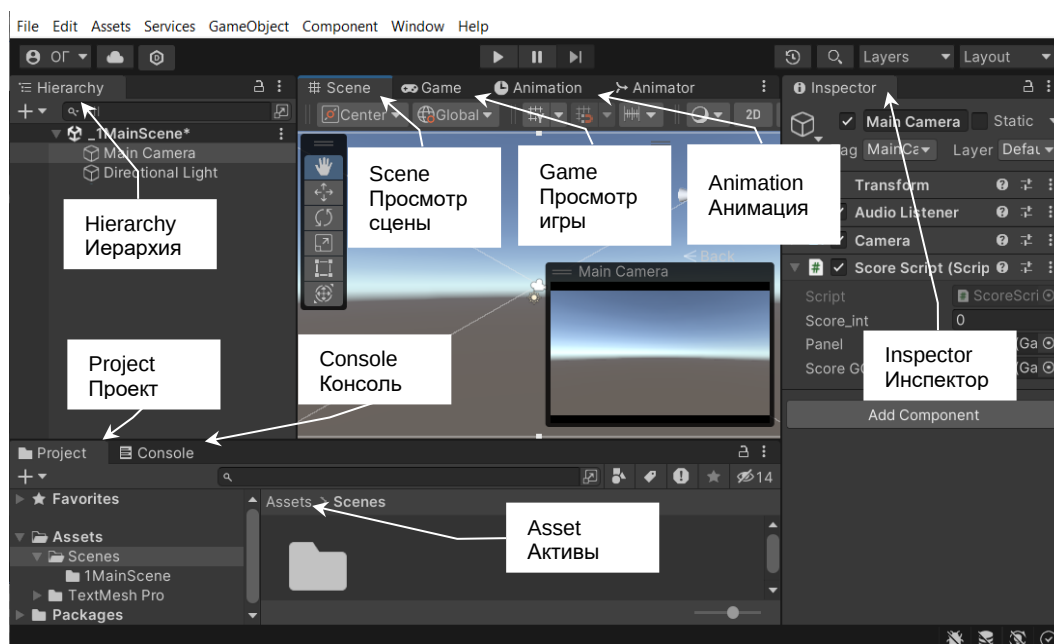
2.2. НАСТРОЙКА ПРОЕКТА



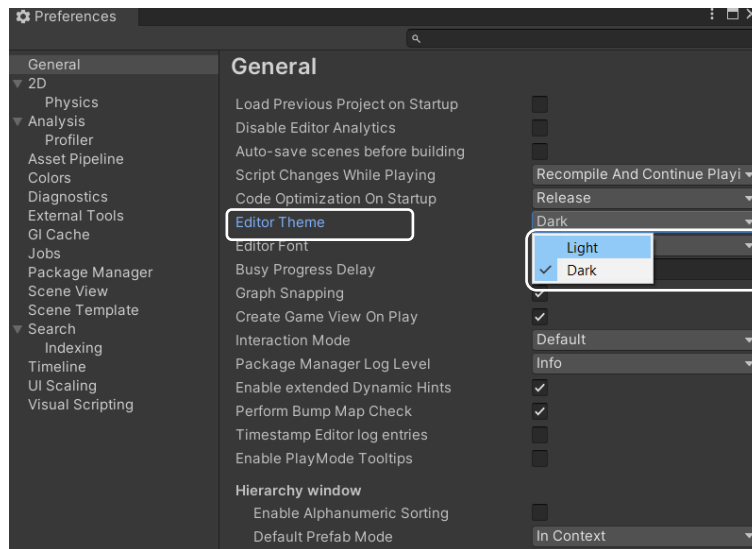
Здесь указывается имя проекта и место, где он будет сохранен, необходимо установить тип проекта и добавить существующие ресурсы.

Нажимаем **Create project** и позволяем Unity настроить основные файлы проекта. Это может занять некоторое время в зависимости от мощности компьютера, предварительно добавленных ресурсов и типа проекта.

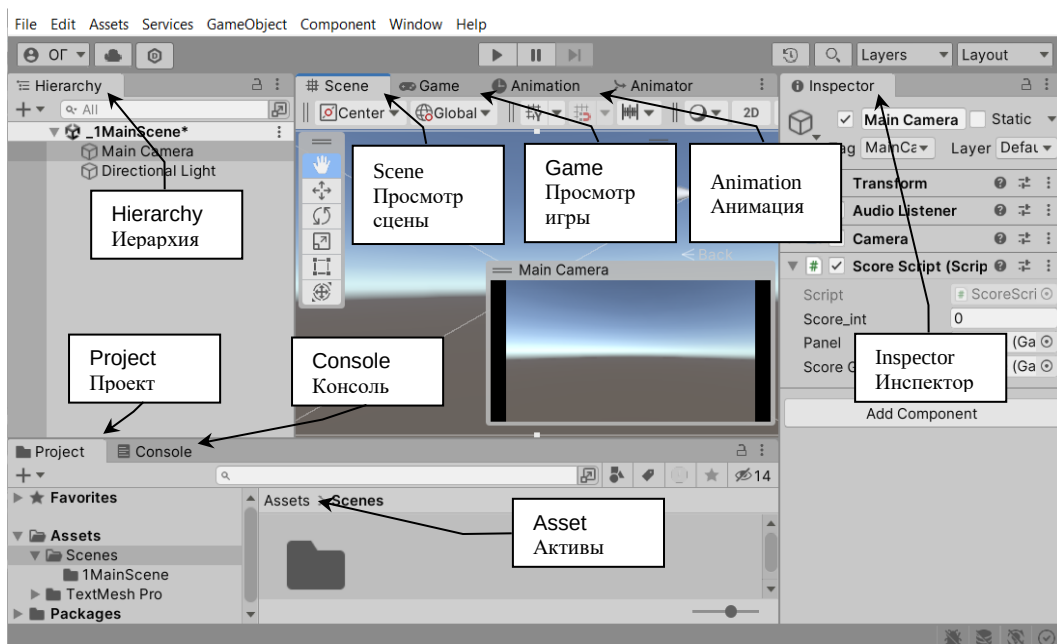
Проект будет создан и открыт в новом окне Unity.



По умолчанию цветовая тема редактора Dark, но можно изменить её на Light: Edit → Preferences → General → Editor Theme



При таких настройках окно редактора Unity выглядит так:



Окно редактора разделено на разделы:

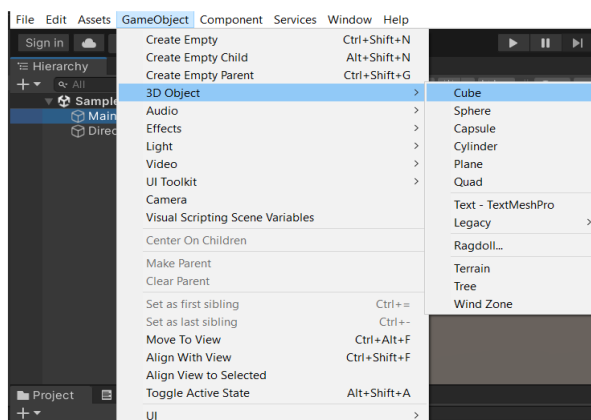
- Hierarchy (Иерархия) – место, где перечислены все объекты в текущей открытой сцене, а также их иерархия родитель-потомок;
- Scene (Просмотр сцены) – уровни, на которых происходит всё в игре. Позволяет размещать и перемещать игровые объекты в сцене;
- Game (Просмотр игры) – предварительный просмотр того, как игрок будет видеть сцену с камеры;
- Inspector (Инспектор) – предоставляет подробную информацию о выбранном GameObject в сцене;

- Project (Проект) – содержит все файлы проекта;
- Assets (Активы) – готовый компонент, который можно использовать для создания своих проектов. Это могут быть скрипты, элементы интерфейса в игре, текстура, фигурка персонажа, шрифт или звук;
- игровой объект (GameObject) – любой Asset, который используется в игровой сцене. Например, изображение монетки, сам ее внешний вид – это Asset, а пять монет, которые должен подобрать персонаж в процессе прохождения уровня – это пять игровых объектов. Сам персонаж при этом тоже станет игровым объектом;
- компоненты (Components) – часть игрового объекта, отвечающая за его поведение в процессе игры: перемещение или реакцию на определенные триггеры;
- скрипт (Script) – код на C#, в котором прописаны конкретные условия работы компонента.

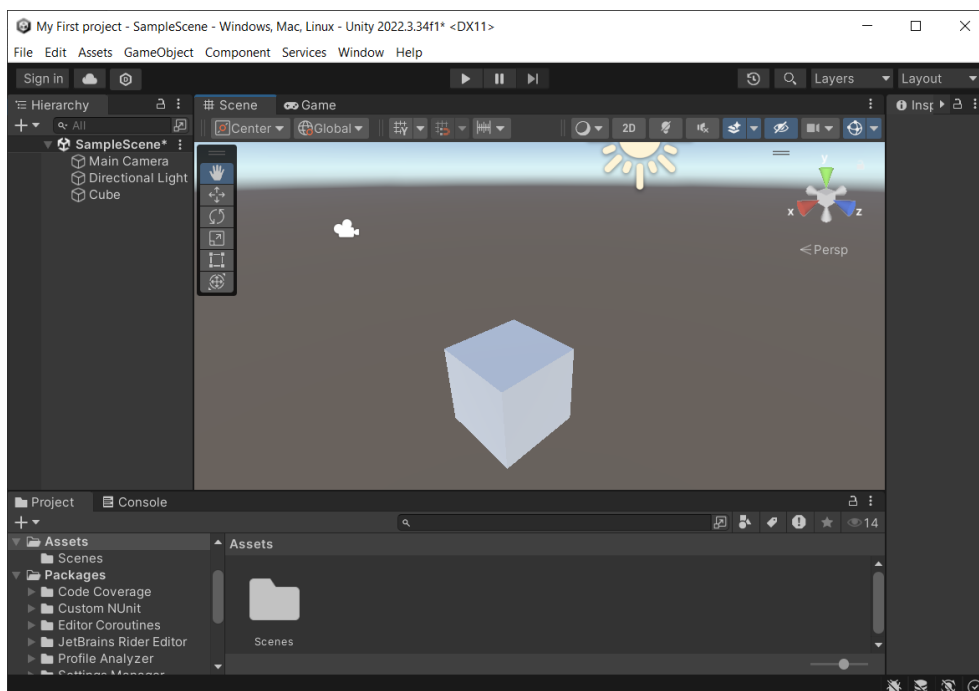
3. СОЗДАНИЕ ПЕРВОГО ПРОЕКТА

3.1. Падающий куб

Чтобы создать игровой объект «Куб», на верхней панели быстрого доступа в среде Unity выберите GameObject → 3D Object → Cube.

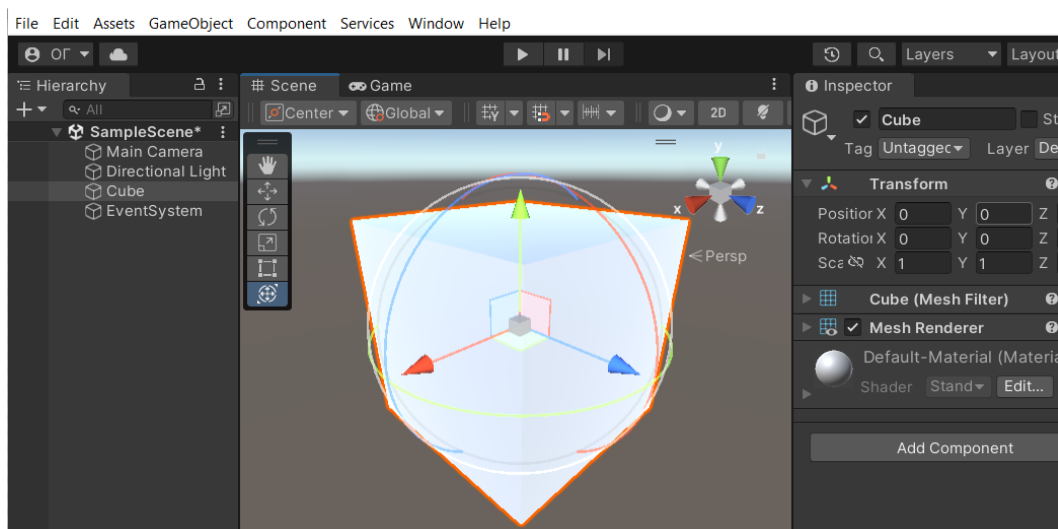


На сцене появится новый игровой объект.

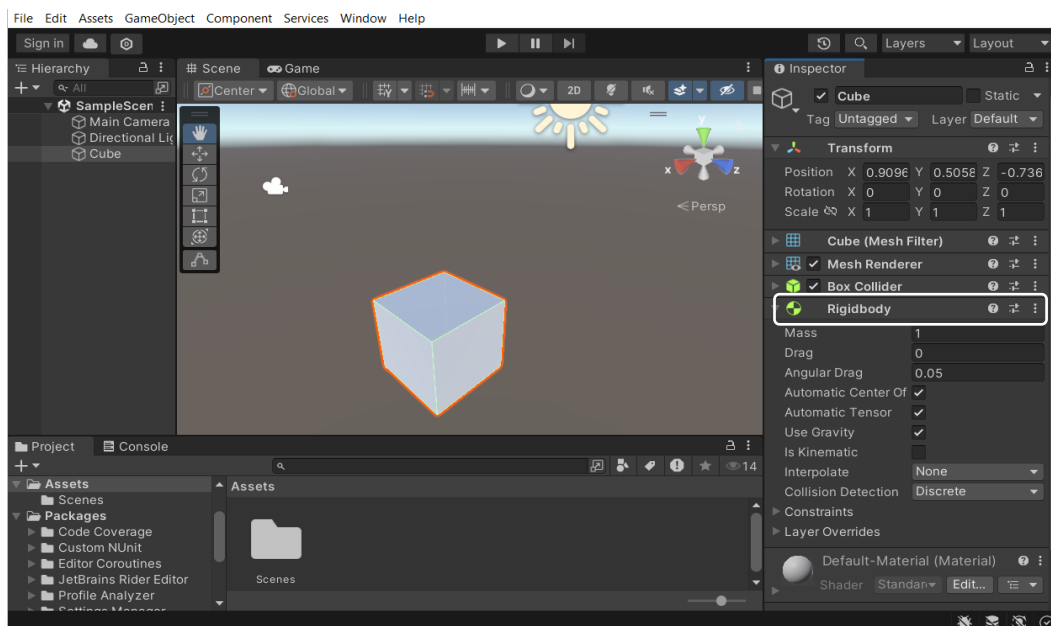
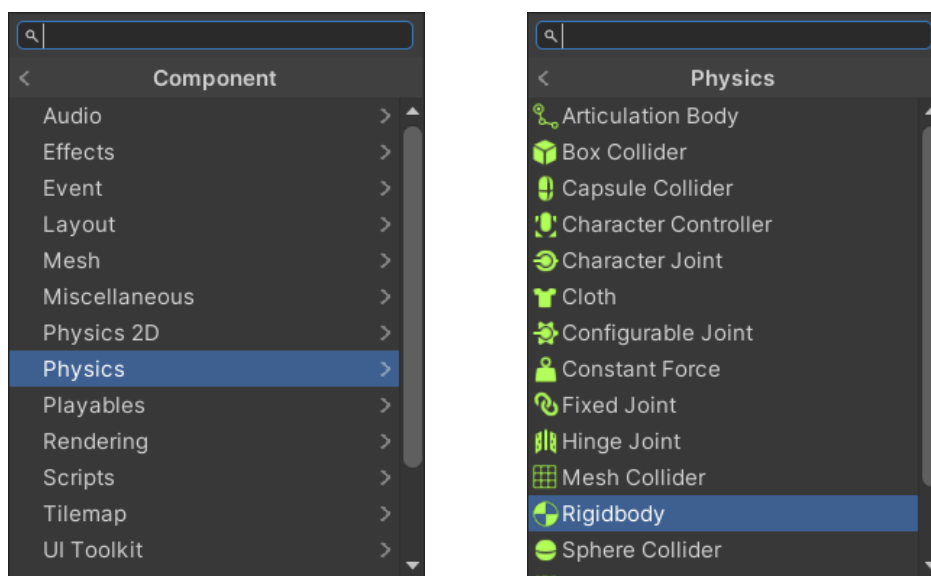


Сделаем так, чтобы созданный 3D-объект Cube при запуске сцены падал вниз. Для этого выделите объект Cube (клик левой кнопкой мыши в окне Hierarchy), после этого в правой части среды разработки станет активно окно Inspector.

Содержание окна Inspector зависит от типа выбранного объекта. В нем содержатся свойства объекта, его параметры, подключаемые Script-файлы и т.д.



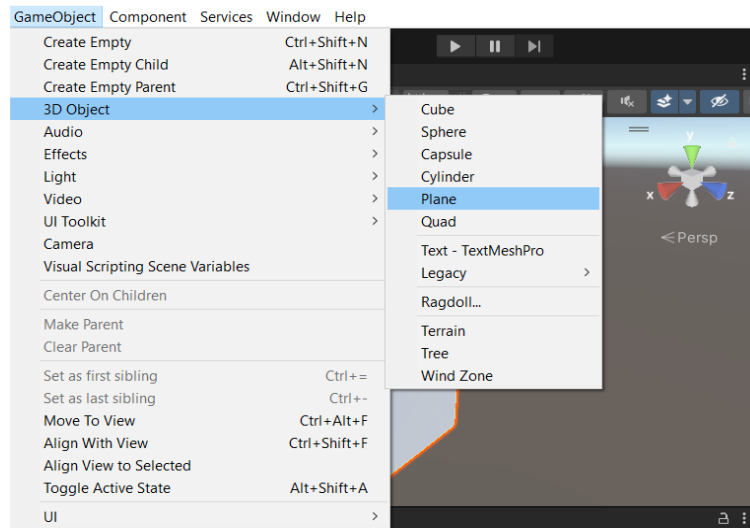
Нажмите в нижней части окна Inspector кнопку Add Component. После этого появится список с перечнем компонентов, которые могут быть подключены к выбранному объекту Cube. Найдите с помощью поиска компонент Rigidbody (раздел Physics) и кликните по нему левой кнопкой мыши так, чтобы он добавился в окно Inspector.



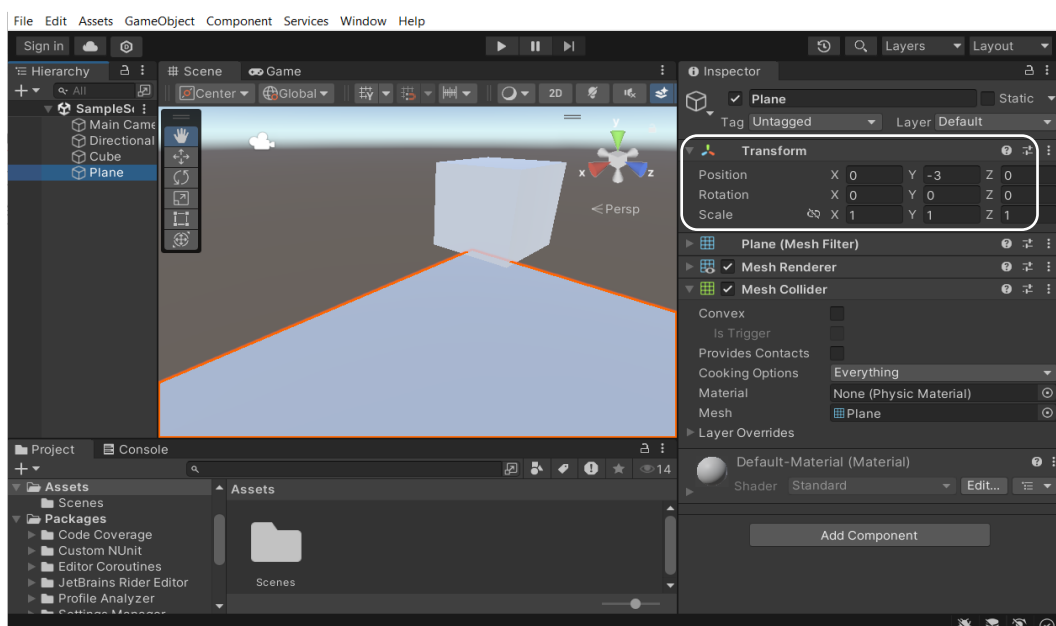
Rigidbody – это механическое свойство объекта, определенное в базовом движке Unity. Другими словами, если назначить этот компонент объекту, то он начнет вести себя в соответствии с механикой абсолютно твердого тела. Запустите сцену (нажмите Play) и убедитесь, что теперь объект Cube начинает падать вниз.

3.2. КУБ, ПАДАЮЩИЙ НА ПЛОСКОСТЬ

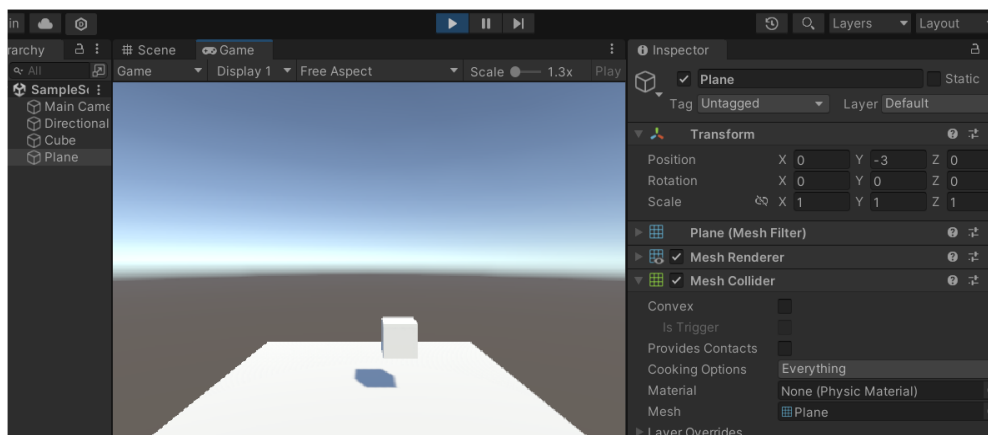
Создадим еще один объект – плоскость (Plane), которая будет ограничивать падение куба за пределы начальной сцены. Для этого выполним действия, которые уже выполнялись при создании объекта Cube, – в верхней части меню выберите GameObject → 3D Object → Plane:



После создания плоскости переместим ее немного ниже уровня объекта Cube. Для этого выделите объект Plane в окне Hierarchy (клик левой кнопкой мыши) и в окне Inspector в верхней части установите значения напротив свойства Transform – Position [X: 0, Y: -3, Z: 0]:



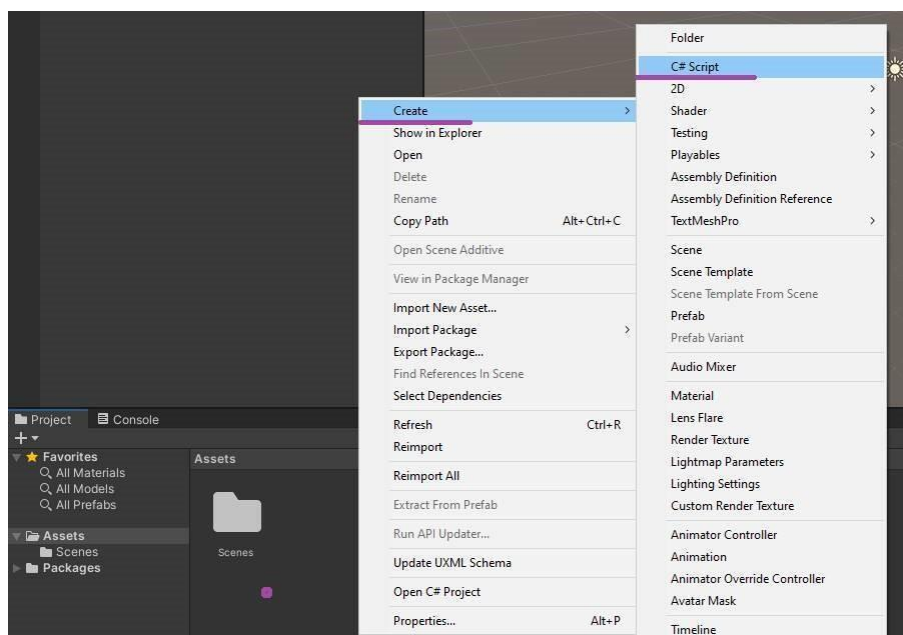
Запускаем сцену еще раз. Теперь объект куб (Cube) падает на плоскость (Plane) при старте сцены.



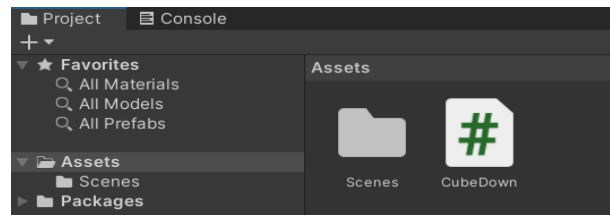
Создадим новый C# Script-файл с командой, которая выводит сообщение «Cube Down». Также впишем небольшой функционал, который будет уничтожать объект Cube при нажатии клавиши пробел.

Для этого на панели Project перейдите в папку Assets, в данный момент в ней находится только одна папка с названием Scenes.

Кликните правой кнопкой мыши внутри папки Assets и выберите из контекстного меню Create → C# Script.



Назовите созданный скрипт-файл **CubeDown.cs**. Содержимое папки **Assets** после этого должно выглядеть так:



Откройте файл **CubeDown.cs**, кликнув по нему дважды. Файл автоматически откроется в Visual Studio Community.

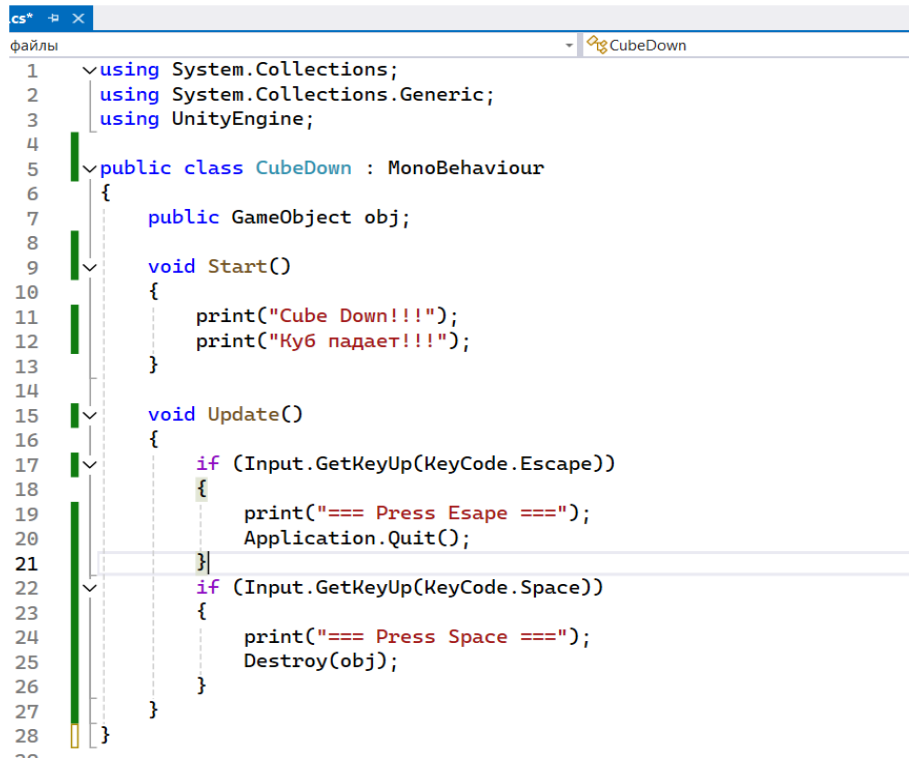
Содержимое скрипт-файла **CubeDown.cs**:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class CubeDown : MonoBehaviour
{
    public GameObject obj;

    void Start()
    {
        print("Cube Down!!!");
        print("Куб падает!!!");
    }
    void Update()
    {
        if (Input.GetKeyUp(KeyCode.Space))
        {
            Destroy(obj);
        }
        if (Input.GetKeyUp(KeyCode.Escape))
        {
            print("=== Press Esape ===");
            Application.Quit();
        }
    }
}
```

Листинг дублируется в виде скриншота из MS Visual Studio.



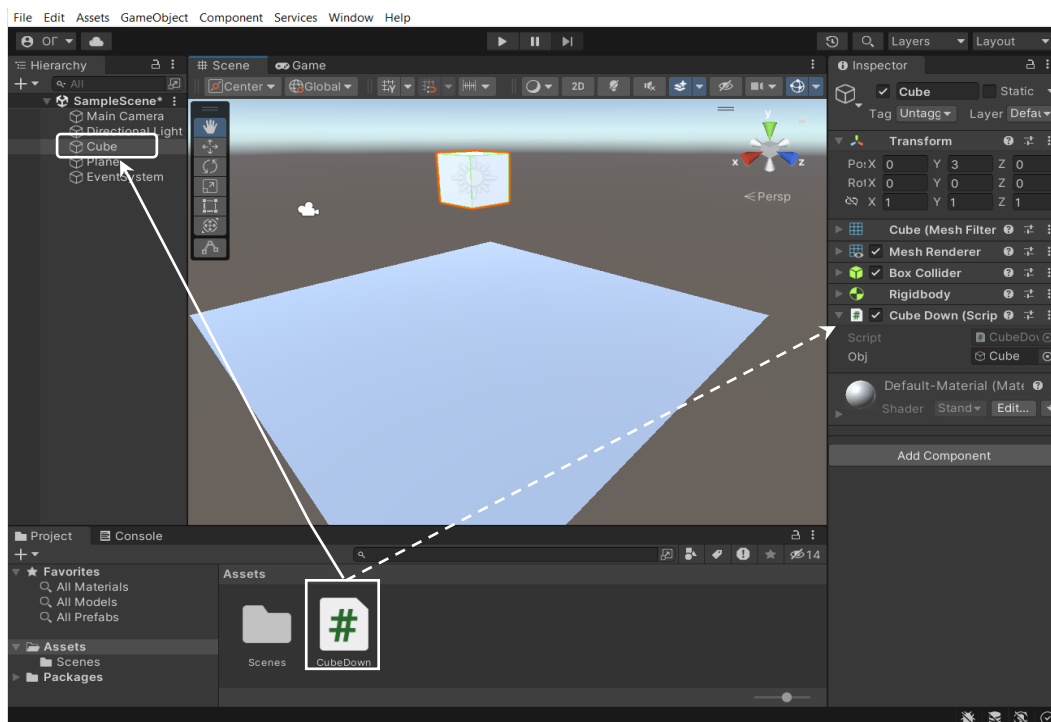
```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class CubeDown : MonoBehaviour
6 {
7     public GameObject obj;
8
9     void Start()
10    {
11        print("Cube Down!!!");
12        print("Куб падает!!!");
13    }
14
15    void Update()
16    {
17        if (Input.GetKeyUp(KeyCode.Escape))
18        {
19            print("=== Press Esape ===");
20            Application.Quit();
21        }
22        if (Input.GetKeyUp(KeyCode.Space))
23        {
24            print("=== Press Space ===");
25            Destroy(obj);
26        }
27    }
28 }
```

Обращаем внимание на то, что внутри кода содержатся два метода: **void Start()** и **void Update()**:

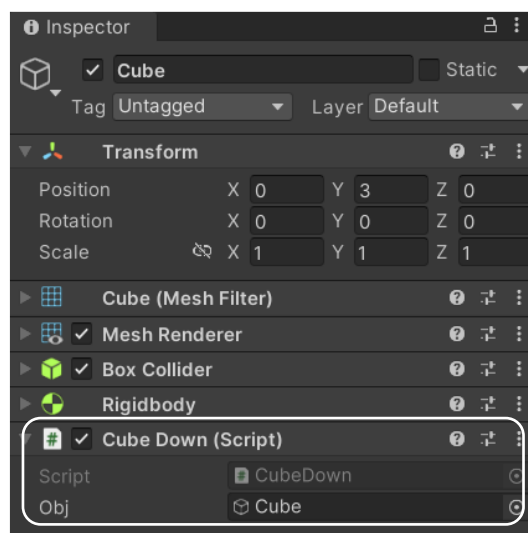
- **Void Start()** – метод, который запускается при старте игры в Unity. Это значит, что команды, написанные внутри фигурных скобок этого метода, отработают один раз при запуске сцены в Unity;

- **Void Update()** – метод, который запускается при выводе каждого кадра на сцене. Другими словами, в метод **Update()** следует писать тот функционал, который требует регулярного обновления в процессе игры.

Чтобы подключить скрипт **CubeDown.cs** к объекту **Cube**, можно просто перетащить (зажав левую кнопку мыши) скрипт-файл на куб или в инспектор.

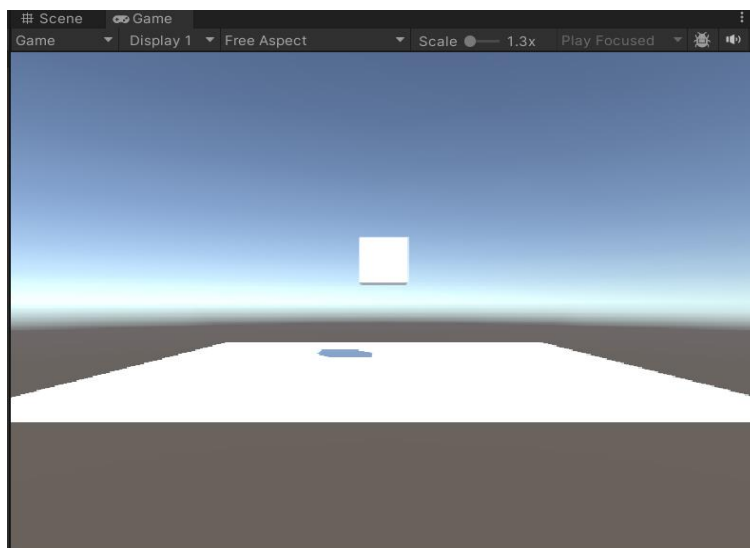


Теперь, если выделить объект **Cube**, кликнув по нему левой кнопкой мыши, то можно увидеть, что в правой части среды разработки (окно **Inspector**) к кубу подключился скрипт-файл **CubeDown.cs**, и внутри него появилась новая переменная **Obj**.

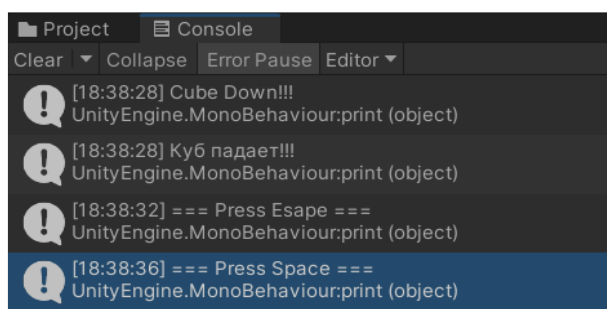


Запускаем сцену (Play). Она работает следующим образом.

- куб (**Cube**) начинает падать;
- куб падает на плоскость **Plane** и останавливается;
- при нажатии на клавишу пробел объект **Cube** удаляется;



- в окно Console выводятся две строки «Cube Down!!!» и «Куб падает!!!»



3.2.1. Принятие пользовательского ввода

Конечно, требуется, чтобы игра реагировала на ввод пользователя. Наиболее распространенные способы сделать это – использовать следующие методы в функции `Update ()` компонента (или в любом другом месте, которое вам нравится):

- `Input.GetKey (KeyCode.W)` возвращает `True`, если удерживается клавиша `W`;
- `Input.GetKeyDown (KeyCode.Escape)` возвращает `True` при первом нажатии клавиши `Esc`;
- `Input.GetAxis` («Вертикальный»), `Input.GetAxis` («Горизонтальный») возвращает значение между `[-1, 1]` перемещения мыши при вводе.

3.2.2. Комментарии

В код были внесены следующие изменения:

- создается переменная класса `GameObject` с именем `obj`, в которую можно будет поместить объект (как это сделать будет показано далее). Класс `GameObject` – это встроенный класс Unity, в дальнейшем мы познакомимся с множеством встроенных классов и их методов;

- создается условие `if`, которое уничтожает объект (`Destroy(obj)`) при нажатии клавиши `Space` (Пробел). При этом используется метод `Input.GetKeyUp`, который срабатывает, после того как игрок отпустил клавишу. Также есть похожий метод `Input.GetKeyDown`, который срабатывает сразу при нажатии клавиши. В качестве эксперимента можете изменить метод на `Input.GetKeyDown`.

Попробуйте самостоятельно внести модификации в некоторые пункты на свой выбор. Так вы сможете более детально разобраться в устройстве взаимосвязей между объектами, скрипт-файлами и некоторыми элементами интерфейса Unity.

Ниже приведен некоторый список возможных изменений в проекте Unity, которые можно внести:

- замените объект `Cube` на плоскость `Plane` внутри скрипта в окне `Inspector`;
- сделайте так, чтобы в `Console` выводилось сообщение «Куб быстро падает!!!»;
- добавьте на сцену больше объектов произвольной формы, измените их размер, положение и ориентацию;
- модифицируйте скрипт-файл таким образом, чтобы разные объекты могли быть удалены со сцены при нажатии разных клавиш на клавиатуре;
- перенесите строку кода `print («Куб падает!!!»)` из фигурных скобок метода `Start()` в фигурные скобки метода `Update()`, и проверьте работу сцены. Что изменилось в выводе в командной строке `Console`?

3.3. КОМПОНЕНТЫ GAMEOBJECTS

Сама сцена состоит из объектов, называемых GameObjects. GameObjects может быть чем угодно: от модели игрока до графического интерфейса на экране, от кнопок и врагов до невидимых «менеджеров», таких как источники звука.

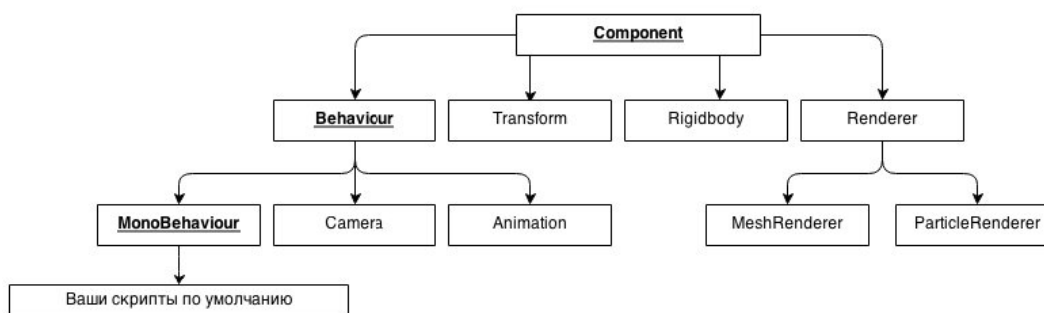
К GameObjects прикреплен набор компонентов, которые описывают, как они ведут себя на сцене, а также как они реагируют на других на сцене, т.е. GameObject – предмет, а компоненты – его свойства.

Зависимость и связи между GameObject и Component



Важно понимать, что компонент – любой класс, наследуемый от Component.

Иерархия наследования стандартных компонентов.

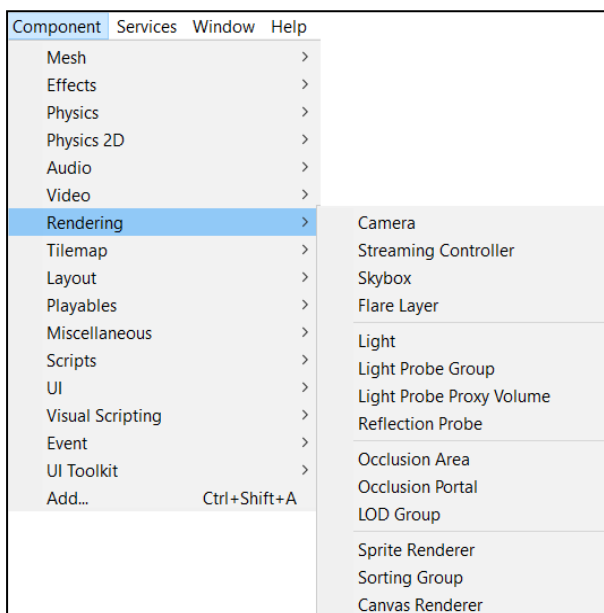


На самом деле, мы можем исследовать это прямо сейчас. Нажмите на основную камеру в иерархии сцен и посмотрите на инспектора. Теперь он не будет пустым; вместо этого в нем будет ряд «модулей».

Наиболее важным компонентом для любого GameObject является его компонент Transform. Любой объект, который существует в сцене, будет иметь трансформацию, которая определяет его положение, вращение и масштаб относительно игрового мира или его родителя, если таковой имеется.

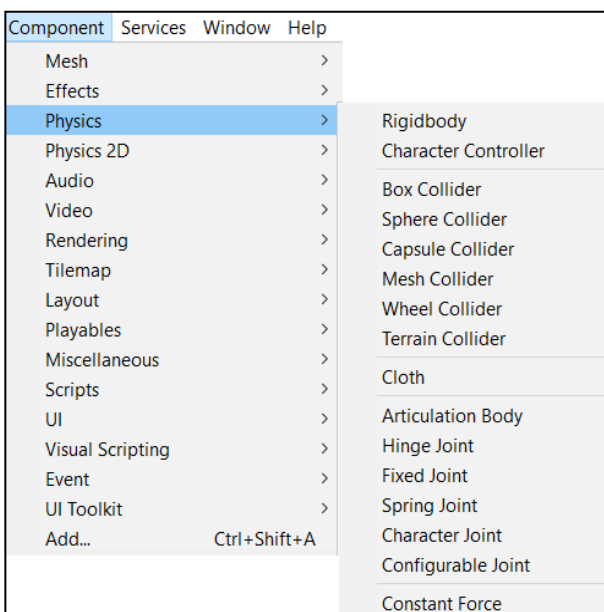
Примеры компонентов:

– **Renderer** – перевод трёхмерной сцены в двухмерное растровое изображение с учётом заданных параметров: освещения, точки наблюдения, материалов. Художник рисует линии, получается силуэт, который можно раскрасить. Потом добавляет текстуры, тени и блики, учитывает условный источник света и в итоге получает объёмное и реалистичное изображение;



– **Collider** – компонент, который представляет собой невидимые «границы» объекта. Часто они совпадают с формой самого объекта (как в реальном мире), хотя это и не обязательно. Unity поддерживает разнообразные формы коллайдеров: **BoxCollider** – форма прямоугольного параллелепипеда. **SphereCollider** – сфера;

– **Physics** – определяет физические свойства в реальном времени, такие как вес и гравитация. Это основной компонент, подключающий физическое поведение объекта. С прикреплённым **Rigidbody** объект немедленно начнёт реагировать на гравитацию. Если добавлен один или несколько компонентов **Collider**, то при коллизиях (столкно-



вениях) объект будет передвигаться;

- Rigidbody – описывает поведение игровых объектов в реальном времени, такие как вес, гравитация, скорость и сила трения;

- Character control (Управление персонажем) – управление персонажами на основе физики для персонажей от первого и третьего лица;

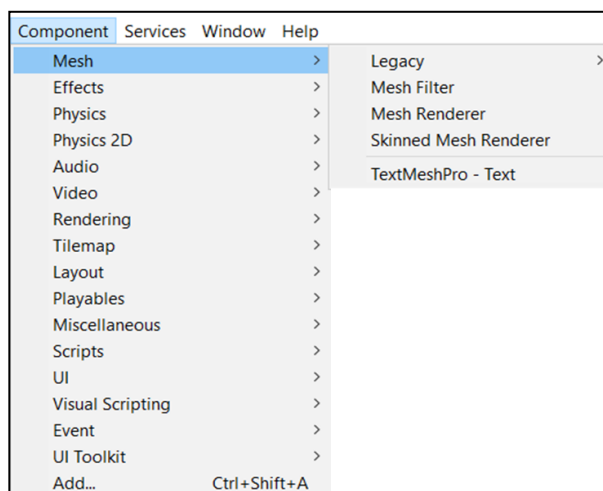
- Collider ... – коллайдеры используются для настройки столкновения между игровыми объектами;

- Cloth (Ткань) – имитация движения ткани для одежды персонажа и другого текстиля, используемого в приложении;

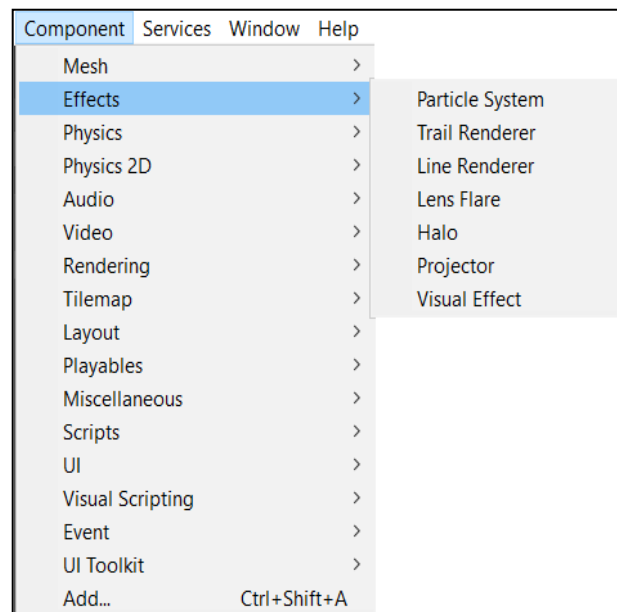
- Articulations (Артикуляции) – настройка сложных систем твердых тел и соединений;

- Joints (Соединения) ... – соединения игровых объектов и имитация физической силы для поворота, перемещения и ограничения.

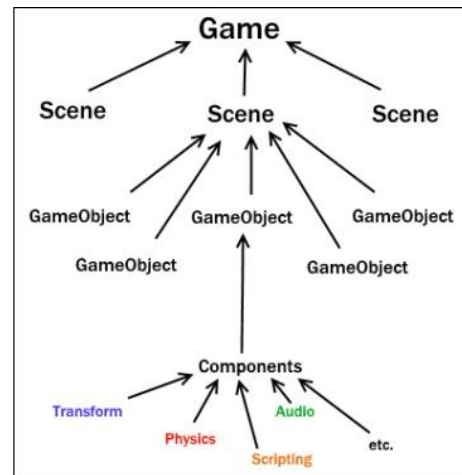
- Mesh – базовый скриптовый интерфейс для геометрии сетки объекта. Он использует массивы для представления треугольников, положений вершин, нормалей и текстурных координат, а также предоставляет ряд других полезных свойств и функций, облегчающих генерацию сетки.



- Effects – позволяет создавать различные эффекты:
 - Particle System (Система частиц)
 - Trail Renderer (Средство визуализации следов)
 - Line Renderer (Средство визуализации строк)
 - Lens Flare (Блик на линзе).



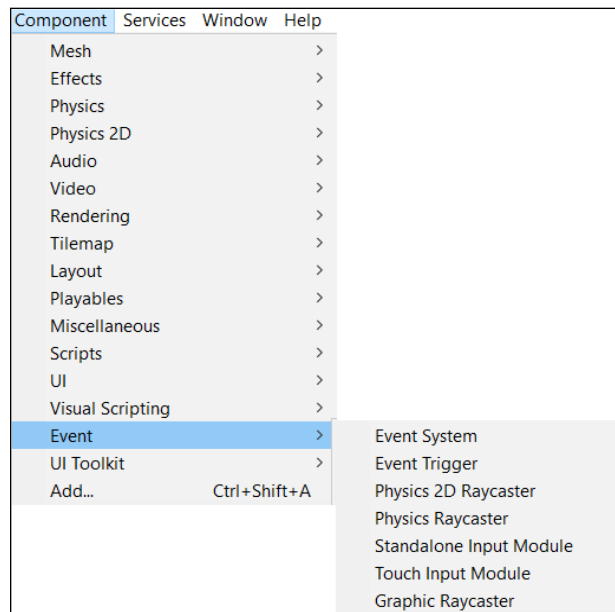
- Audio – предоставляет свойства объекта для воспроизведения и сохранения звука;
- Animator – дает объекту доступ к системе анимации;
- Свет – заставляет объект вести себя как источник света с различными эффектами;
- Камера – определяет область просмотра игрока, которая будет прикреплена к GameObject.



3.4. СИСТЕМА СОБЫТИЙ (EVENTSYSTEM)

Система событий – способ отправки событий к объектам в приложении, основанный на вводе с клавиатуры или мыши; с помощью касаний или персональных устройств. Система состоит из нескольких компонентов, работающих вместе.

Добавив компонент EventSystem к игровому объекту, вы заметите, что он не содержит особой функциональности, потому что предназначен для управления и организации связи между модулями компонента.



Первичные роли системы событий:

- определяет, какой объект считается выделенным;
- определяет, какой InputModule используется;
- управляет райкастингом (если нужно), обновляет все модули ввода (InputModules), если требуется.

3.4.1. Модули ввода

В модулях ввода находится основная логика желаемого поведения EventSystem, они используются для:

- обработки ввода;
- управления состояниями событий;
- отправки событий объектам на сцене.

Только один модуль ввода может быть активным в системе событий одновременно. Он должен находиться на том же игровом объекте, что и компонент EventSystem.

Если вы желаете написать собственный модуль ввода, рекомендуется отправлять события, поддерживаемые уже существующими UI-компонентами Unity. Однако вы можете расширять и создавать собственные события, как описано в документации по системе сообщений (Messaging System).

3.4.2. Трассировщики лучей (Raycasters)

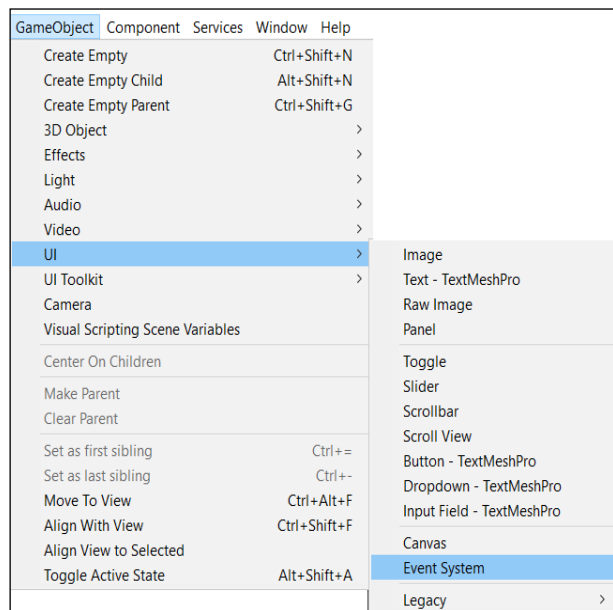
Трассировщики лучей используются для определения объекта, над которым находится указатель.

По умолчанию предоставляется 3 трассировщика лучей:

- GraphicRaycaster – используется для UI-элементов;
- 2DPhysicsRaycaster – для двумерных физических элементов;
- 3DPhysicsRaycaster – для трехмерных физических элементов.

Если в сцене настроен 2D / 3D–трассировщик, очень просто сделать, чтобы не-UI элементы получали сообщения от модуля ввода. Следует присоединить скрипт, который реализует один из интерфейсов обработки событий.

Если компонента EventSystem нет в Иерархии (Hierarchy), то добавить его можно так. В верхней части меню выбираем GameObject → UI → EventSystem:



4. ГРАФИЧЕСКИЙ ИНТЕРФЕЙС ПОЛЬЗОВАТЕЛЯ

Graphic User Interface (GUI), или графический пользовательский интерфейс, позволяет перемещаться по игре и выполнять действия с помощью визуальных индикаторов и графических значков.

В рамках Unity элементы GUI могут быть созданы самостоятельно с помощью любого графического редактора либо скачаны с Asset Store, где содержится довольно большое количество самых разнообразных скинов интерфейса самой разнообразной тематики (для удобства поиска используйте вкладку Assets – GUI).

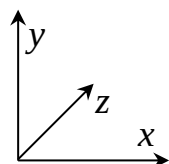
Графический интерфейс имеет достаточно большой функционал. Без него невозможно удобно запустить, сохранить

игру и выйти из нее, без графического интерфейса неинтересно играть, ведь элементы GUI – это показатель счетчика очков и достижений, инвентарь, умения и др. Поэтому работа над графическим интерфейсом не менее важна чем и сама игра, ведь в итоге пользователь будет взаимодействовать с игрой через интерфейс.

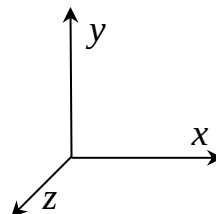
4.1. КООРДИНАТЫ В GUI UNITY3D

4.1.1. Мировые координаты

В Unity параметр Transform использует мировую систему координат, которая делится на левую и правую системы координат. Среди них **левосторонняя** система координат – это **мировая** система координат в Unity, правосторонняя система координат используется в математике.



Мировая система координат в Unity



Правосторонняя система координат

Информация о координатах объекта в Unity хранится в Transform.position и переменной Vector3, которая является трехмерным вектором и хранит информацию XYZ.

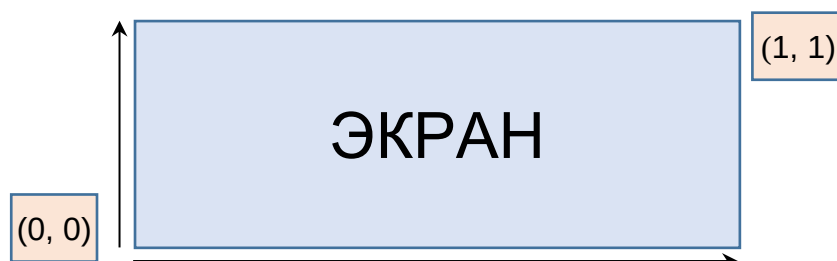
Мировые координаты представляют абсолютное расположение объектов в сцене с помощью «точек». Можно выбрать единицы измерения для представления длины, например, 1 ед. = 10 м. То, что на самом деле отображается на экране, определяется тем, где расположена камера и как она ориентирована.

4.1.2. Система координат экрана

Параметры экрана стандартизированы, что больше подходит для пропорциональных расчетов.

Координаты просмотра – координаты в окне просмотра данной камеры.

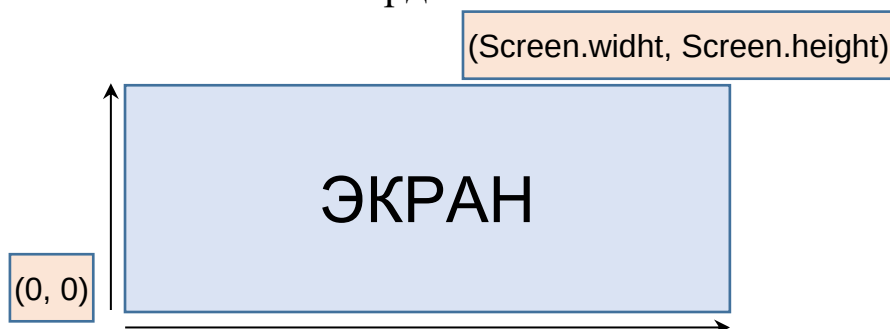
Окно просмотра – это воображаемый прямоугольник, через который просматривается мир. Эти координаты являются пропорциональными и варьируются от (0,0) – нижний левый угол до (1,1) – верхний правый угол.



4.1.3. Система координат графического интерфейса

Координаты экрана – фактические координаты пикселей, обозначающие положение на экране устройства.

Система координат экрана рассматривает экран как систему координат, начиная с левого нижнего угла, который равен (0,0), и правого верхнего угла (`Screen.width`, `Screen.height`), поэтому он также называется системой координат в пикселях.



Координаты позиции мыши относятся к системе координат экрана. Благодаря взаимному преобразованию между координатами экрана и мировыми координатами можно получить фактическую интерактивную позицию мыши в Unity3D.

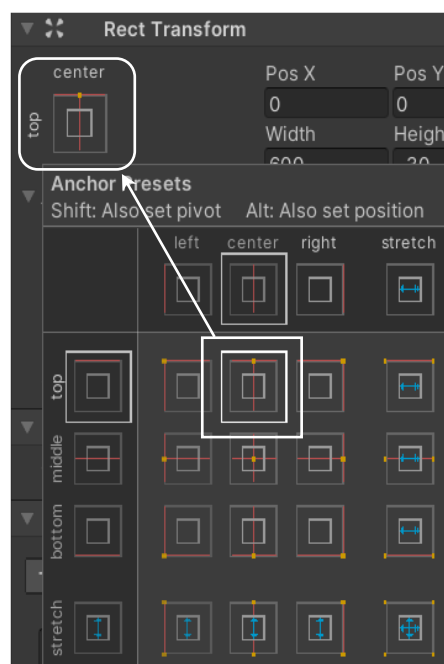
Следует отметить, что мировые координаты любого объекта всегда будут одинаковыми, независимо от того, какая камера используется для просмотра, тогда как координаты экрана зависят от используемой камеры. Кроме того, координаты экрана зависят от разрешения устройства и расположения камеры на экране.

Объект «Камера» предоставляет несколько методов для преобразования между этими различными системами координат, таких как `ScreenToViewportPoint`, `ScreenToWorldPoint` и т.д.

У обычного Transform есть потомок RectTransform, который и отвечает за положение и многие вещи, связанные с размером. И благодаря ему мы можем получить следующие варианты координат:

- position;
- .localPosition;
- anchoredPosition (есть еще anchoredPosition3D, это подвид test.anchoredPosition).

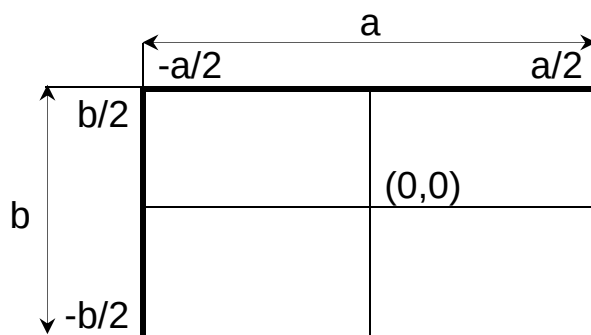
Обычная позиция не относится к системе координат UI. Это некие общие координаты объекта во всём мире Unity, и некое представление позиции объекта которое не связано с интерфейсом. Если вы захотите сдвинуть объект на его ширину, это не сработает, так как у Canvas используются координаты в пикселях относительно экрана. А в глобальных координатах это не пиксели. Поэтому объект сместится в мире, допустим, на 200 единиц измерения мира, а не на 200 пикселей, и просто улетит в неведомые дали.



Якорная позиция (anchoredPosition) – координаты объекта относительно якоря, которую можно увидеть в инспекторе.

Если требуется сместить объект на его ширину, то надо менять anchoredPosition или.localPosition.

Локальная позиция (localPosition) – это система координат, когда в центре родительского объекта находится (0,0).



То есть, если ширина прямоугольника 800 пикселей, то в середине $x = 0$, в правом верхнем 400, в левом верхнем – 400.

Поэтому если нужно без учёта якорей посчитать координаты, лучше использовать локальные позиции, при условии, что у них общий предок.

Якоря можно использовать не только как конкретную точку на родительском объекте, но и как пропорции относительно родительского объекта. Во втором случае `anchoredPosition` будет возвращать `localPosition`.

4.1.4. Комментарии

В инспекторе у UI элементов показываются `anchoredPosition` или пропорции относительно родительского объекта.

`AnchoredPosition` считается в зависимости от положения якоря.

`localPosition` позволяет получить общие координаты относительно родителя, без привязки к якорям.

`position` – это глобальная позиция элемента в пространстве сцены, не привязана к UI.

5. ИГРОВОЙ ПРОТОТИП «ДИРИЖАБЛЬ»

Создание геймплея (игрового процесса) называют игровым дизайном (Game Design). Обычно перед началом разработки следует определиться с основными требованиями, предъявляемыми к игре. Если говорить более обобщенно, то в списке требований, с которыми следует определиться при начале разработки, можно выделить следующие:

- платформа, под которую осуществляется разработка (Windows, WebGL, Android / iOS);
- экран: разрешение и ориентация;
- длительность игровой сессии;
- система управления;
- однопользовательская или многопользовательская игра, система монетизации приложения.

5.1. СОЗДАНИЕ ИГРОВОГО ОБЪЕКТА – «ДИРИЖАБЛЯ»

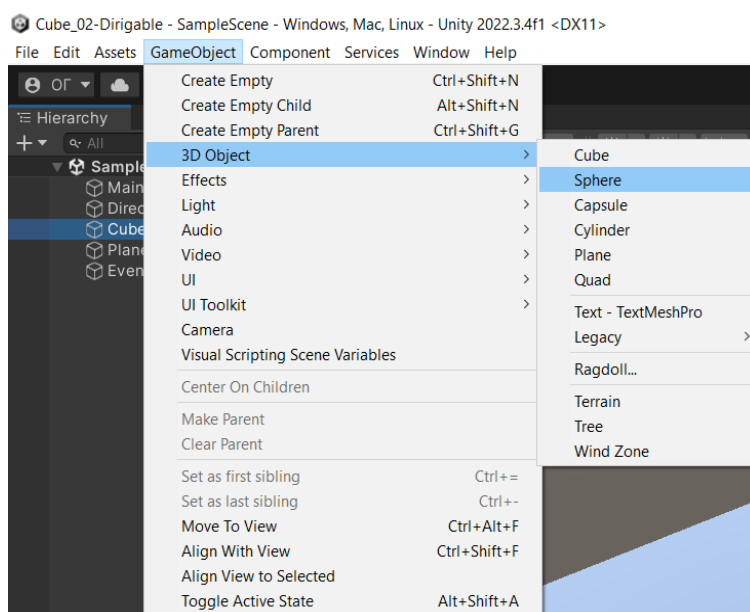
В качестве основного источника визуальных моделей и визуального оформления для своих игр на первом этапе вы можете использовать Unity Asset Store – это магазин, в котором можно найти:

- 3D-модели;
- звуки / музыку;
- элементы пользовательского интерфейса;
- шейдеры / частицы;
- наборы спрайтов;
- готовые скрип-файлы и др.

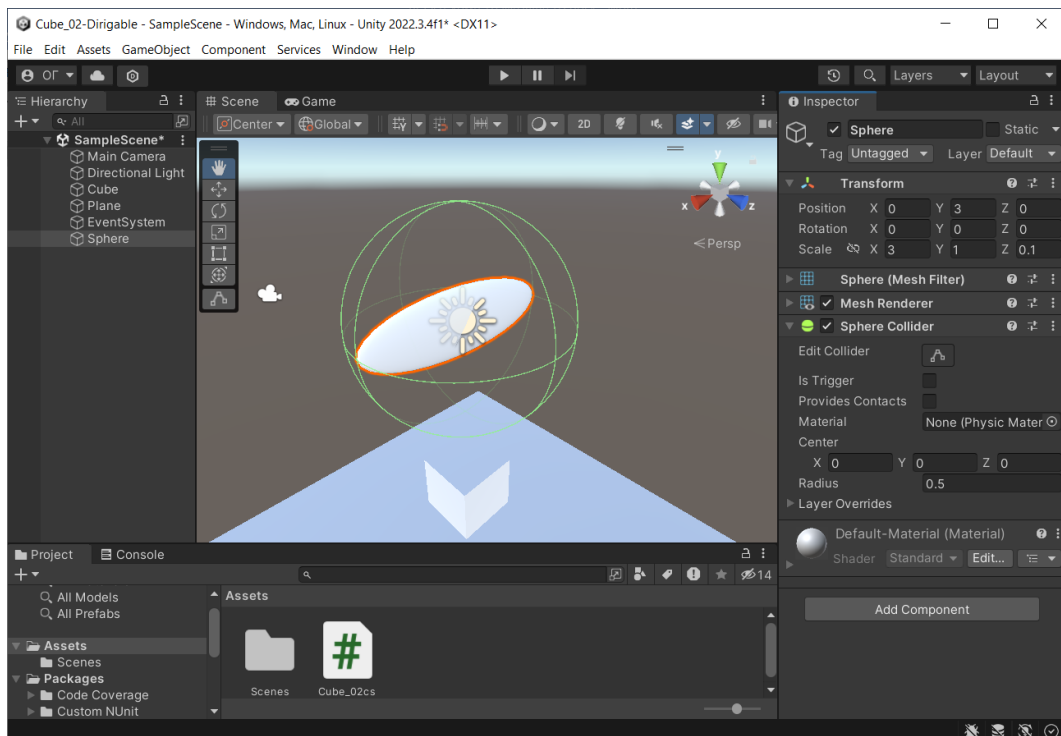
Мы воспользуемся встроенными возможностями среды разработки Unity.

Пусть с некоторого летающего объекта, например, дирижабля, сбрасывается гуманитарная помощь (подарки в коробках).

Добавим объект сфера (Sphere).

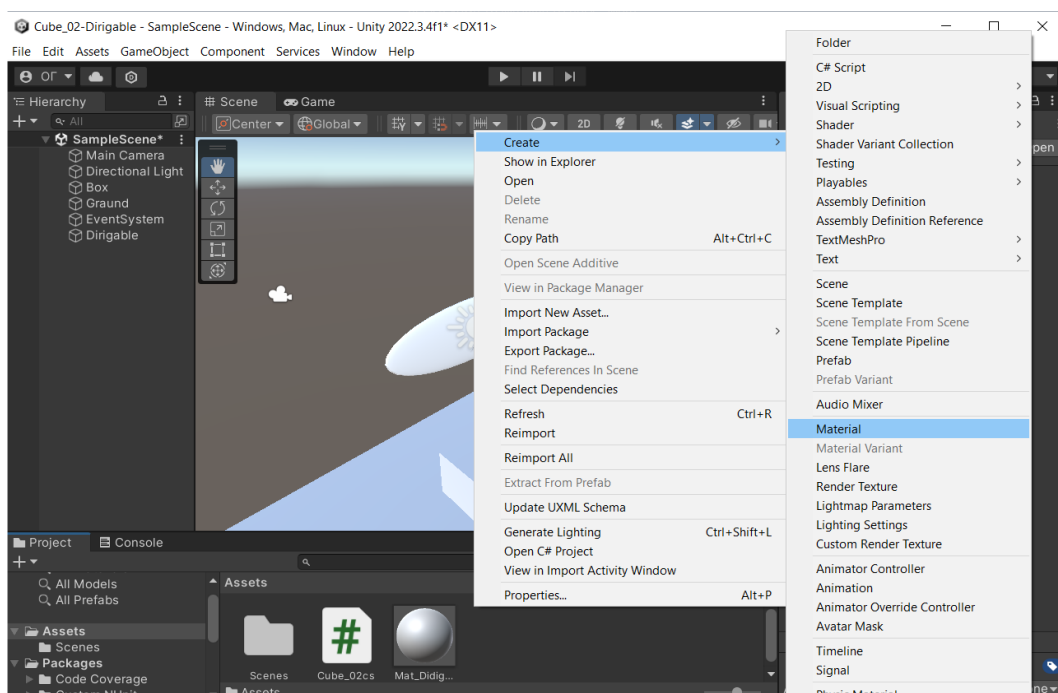


Изменяя пропорции, сделаем сферу, похожей на дирижабль.



Переименуем Sphera в Dirigable, Cube в Box, Plane в Ground.

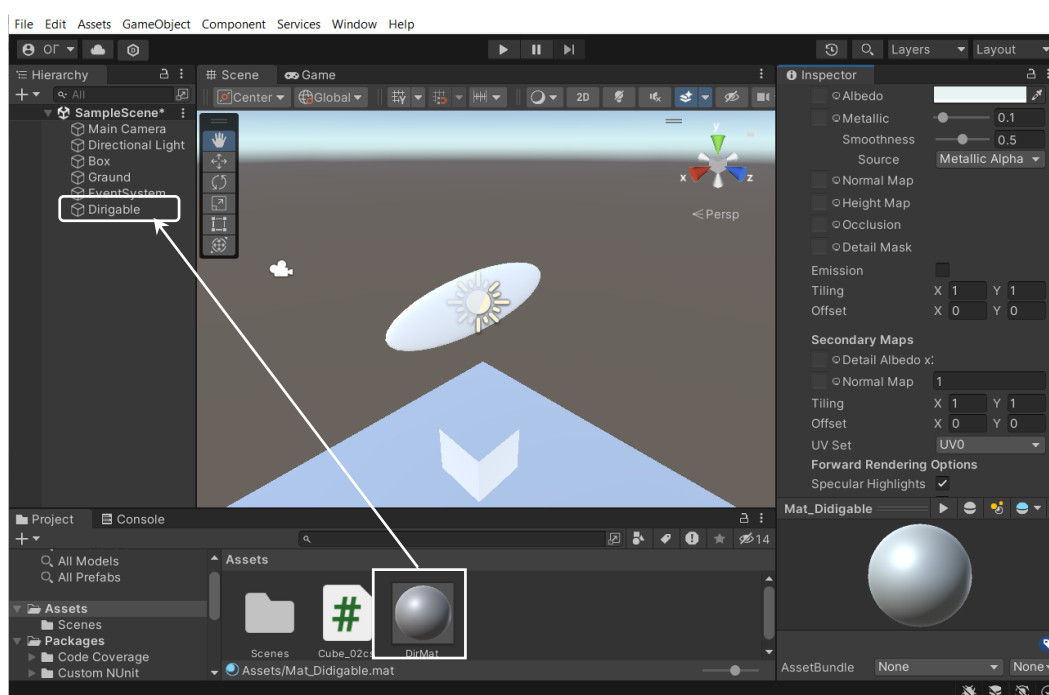
Создадим материал для дирижабля, кликнув внутри Scenes правой кнопкой мыши и выбрав Create → Material, назовём его DirMat:



Кликните по материалу DirMat, в окне Inspector (в правой части среды разработки) появятся свойства материала, доступные для редактирования. Например, кликнув по полю Albedo, можно изменить основной цвет, также можно использовать бегунки Metallic и Smoothness, поэкспериментировать и установить цвет, который вам покажется наиболее подходящим. В любом случае, всегда можно вернуться к этому пункту и отредактировать свойства материала. Настройки цвета материала в нашем примере показаны на рисунке справа.



Чтобы назначить созданный материал объекту, просто перетяните материал DirMat (из папки Assets – Scenes) на объект Dirigable (в окне Hierarchy). Так материал будет назначен созданному ранее 3D-объекту.



5.2. НАСТРОЙКА 3D АНИМАЦИИ

Анимация – это метод, с помощью которого неподвижные фигуры обрабатываются для отображения в виде движущихся фигур.

С помощью анимации можно визуальнo оживить любой объект, персонажей, задний фон в игре и т.д. Анимация

используется повсеместно: в рекламе, кино, играх, на телевидении. В зависимости от целей и вида анимации для ее создания применяют множество разнообразных программ.

Анимацию объектов (перемещение, вращение, изменение размера, смена цвета) можно легко сделать самостоятельно в самом Unity, используя окно Animation.

А вот анимацию персонажей уже так просто не сделать. Персонаж должен быть подготовлен, у него должен быть «скелет», т.е. набор виртуальных «костей», которые образуют дерево и отражают структуру модели. Процесс создания скелета называется риггингом. Этим обычно занимаются либо сами проектировщики игры на этапе создания модели, либо аниматоры.

В Unity создать свою анимацию персонажа можно, но это очень трудоёмкий и долгий процесс. На самом деле, анимацию персонажей редко создают вручную, обычно для этих целей используются специальные костюмы, а процесс её создания называется Motion Capture. Вручную происходит только так называемая чистка, в ходе которой исправляют недочеты в записанной анимации.

Лучше использовать готовую анимацию, а если нет ничего подходящего, то заказывают её изготовление у профессионалов по анимированию. Несложную анимацию можно создать прямо в Unity.

В Unity для анимации будем использовать Animation и Animator:

- Animation – окно представляет собой таймлайн (временную шкалу), на котором вы можете анимировать любые объекты, используя ключевые кадры. Ключевыми кадрами, или ключами, называются кадры, в которых происходят изменения;

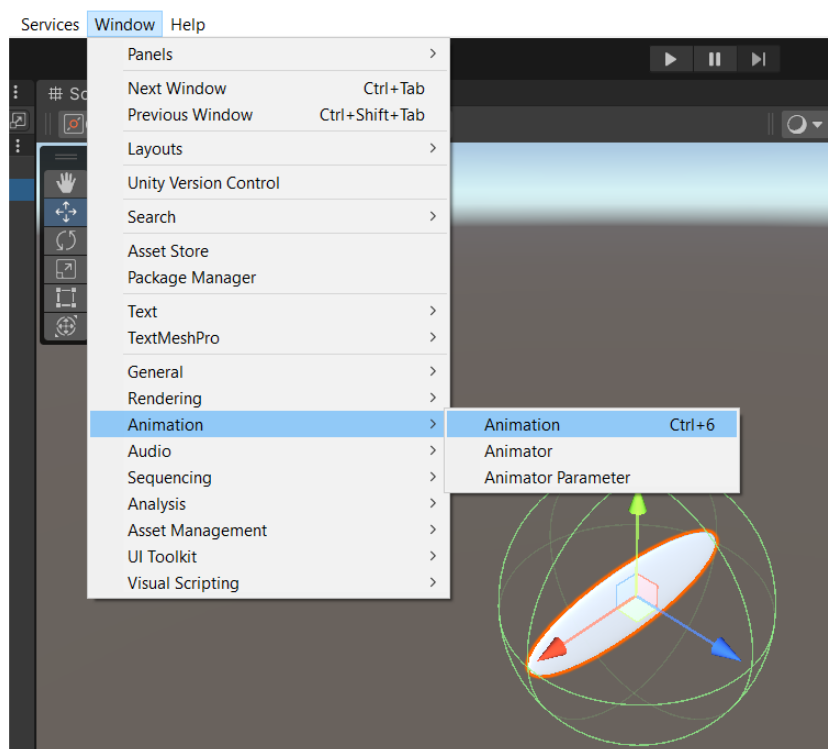
- Animator – система, которая управляет анимациями. Animator воспроизводит файлы анимации так, как плеер – музыку. Он позволяет гибко выстроить дерево из анимаций со всевозможными ветвлениями. Эта система анимации в Unity называется Mecanim.

Приступим к созданию простой анимации 3D-модели в Unity. Например, мы хотим, чтобы дирижабль вращался. Сделать эту анимацию можно двумя вариантами – через Animation или через код. Посмотрим, как ее реализовать через Animation.

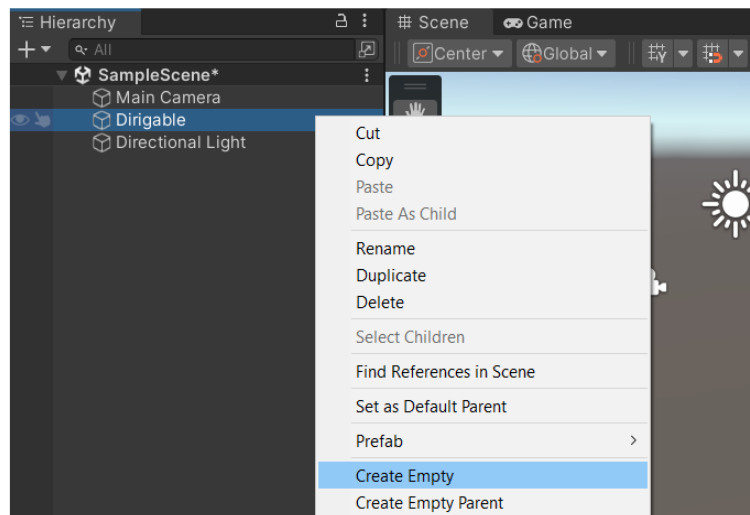
Создадим пустой объект, сбросим его Transform (позиция и поворот на ноль).

Пустой объект нам нужен для того, чтобы при анимировании позиции анимация привязывалась к координатам. Соответственно, при переносе предмета из одного места в другое при запуске анимации его будет переносить обратно в те точки, которые указаны в анимации. В случае, когда анимируемый объект вложен в другой объект, его позиция определяется и анимируется относительно родителя, а не мира.

Откроем окно Animation (Window → Animation → Animation (Ctrl + 6)).



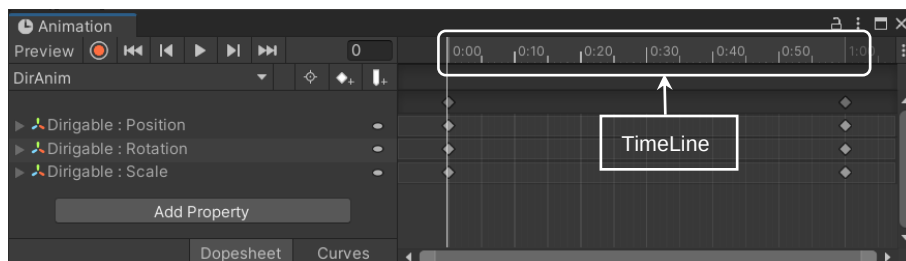
Выберем объект для анимации и нажмем Create в окне Animation. Создадим папку для хранения анимаций, дадим название нашей анимации DirAnimation и сохраним ее в этой папке.



Красный кружок — это запись. Можно нажать на него, переходить по кадрам и вручную менять параметры объекта прямо на сцене — все ключи будут автоматически записываться в выбранный кадр.

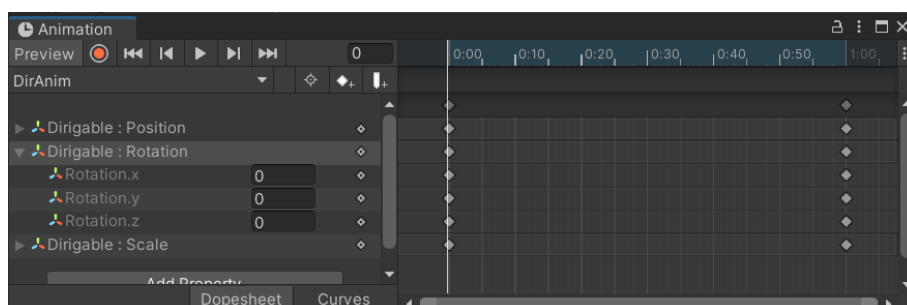
Следующие кнопки — стандартные: перейти в начало анимации, перейти на предыдущий ключ, проиграть анимацию. Маленькое окошко справа показывает, в каком кадре вы сейчас находитесь.

Нажмём Add Property и выберем Rotation.

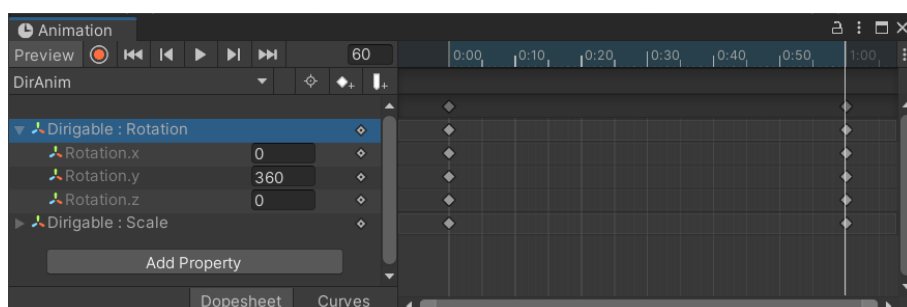


На TimeLine появятся ключевые кадры (ромбики) в начале и в конце анимации. Нажимая на линейку над TimeLine, можете перемещаться по нему.

Перейдем к первым кадрам. Развернем Rotation в левом меню, там должны быть все оси установлены на 0.



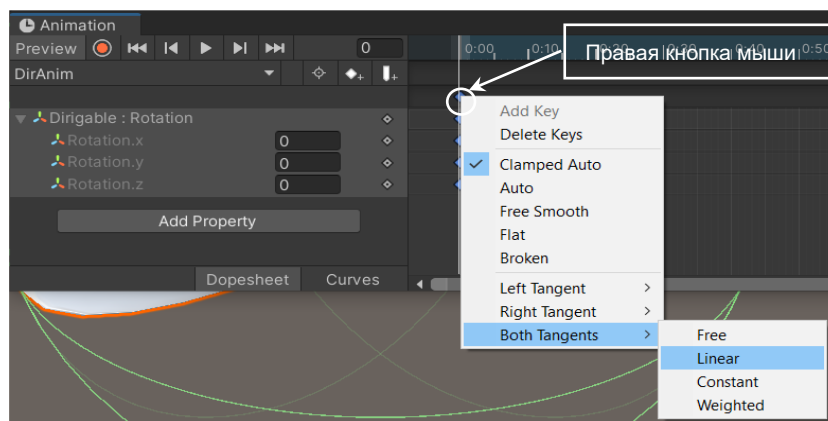
Перейдём в последние ключи и выставим там ось Y на 360. Таким образом, наш объект сделает один полный оборот за всю анимацию.



Используя колесико мыши на TimeLine, можно отдалить его видимость и перенести последние ключи чуть дальше, чтобы анимация длилась подольше.

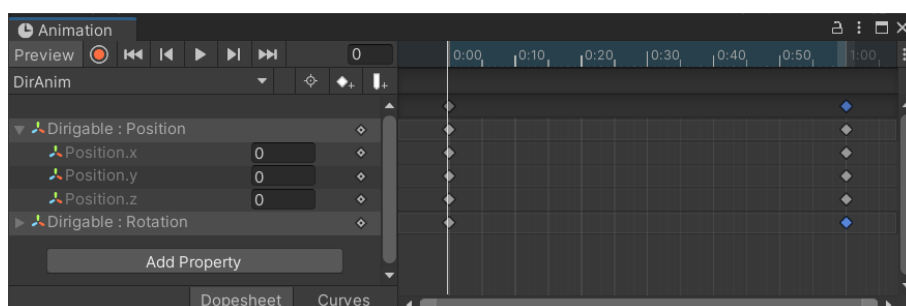
Проигрываем анимацию, нажав на воспроизведение в левом меню. В данный момент наша анимация плавно стартует и плавно останавливается. Наша задача – сделать анимацию равномерной.

На самом первом, верхнем ключевом кадре выберем (правая кнопка мыши) пункт меню Both Tangents → Linear, таким образом задав линейную скорость.

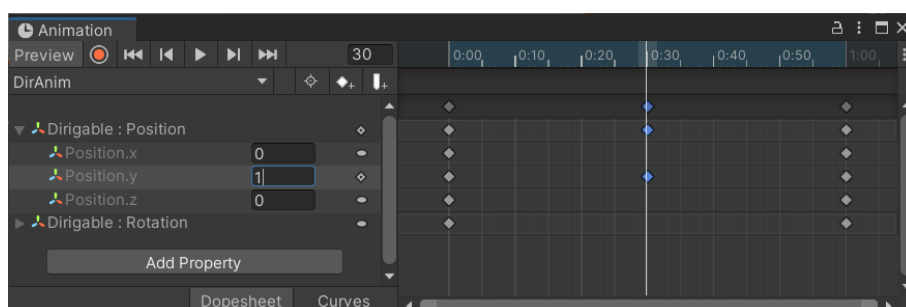


То же самое сделаем и для последних кадров. Теперь наша анимация проигрывается с одинаковой скоростью.

Дополним анимацию движением дирижабля вверх-вниз. Для этого нажмем Add Property → Position. В начале и в конце все оси Position со значением 0.

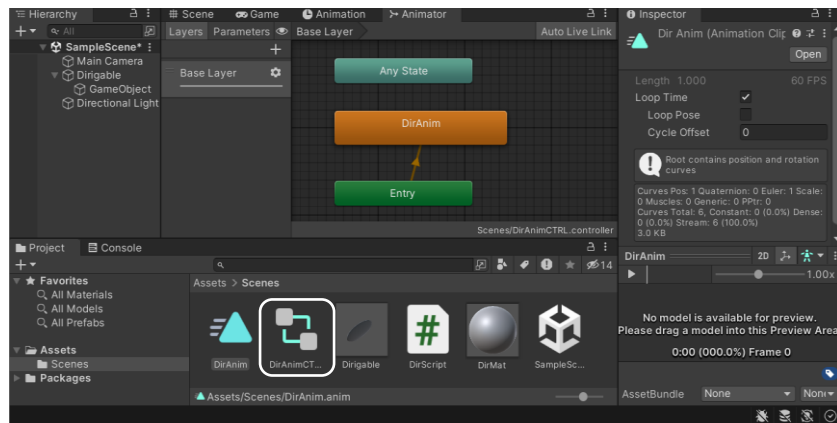


Переходим в середину анимации и ставим новый ключ, выставляем позицию по Y на 1. Поставить дополнительный ключ можно, нажав под пультом на ромбик с плюсом.

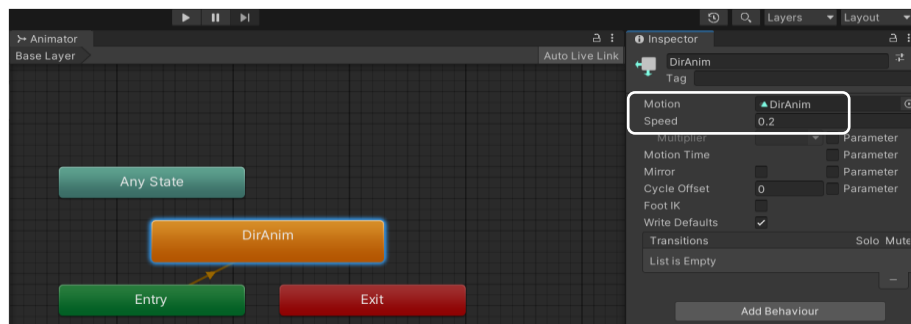


Теперь наш дирижабль анимирован по двум свойствам.

Если два раза кликнуть на DirAnimCTRL, то попадаем в окно Animator.



Чтобы изменить скорость вращения дирижабля, в окне **Animator** выделяем **DirAnim**, в поле **Speed** устанавливаем, например, 0.2:



Примерно то же самое через код:

```
using UnityEngine;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DirScript : MonoBehaviour
{
    private float _lengthFlay = 0.1f;
    private float _speedRotation = 0.5f;
    void Start() { }
    void Update()
    {
        transform.Rotate(eulers:
            Vector3.up*(Time.deltaTime * _speedRotation),
            relativeTo: Space.World);
        transform.localPosition =
            new Vector3(transform.localPosition.x,
                y: Mathf.PingPong(t: Time.time, _lengthFlay),
                transform.localPosition.z);
    }
}
```

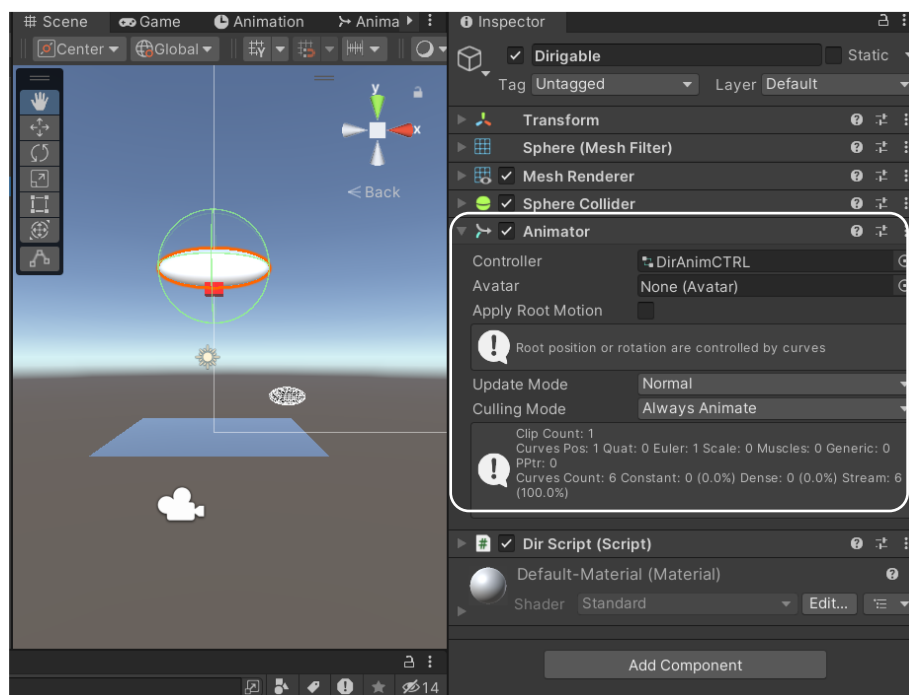
И скриншот из VS Studio.

```
DirScript.cs
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DirScript : MonoBehaviour
{
    private float _lengthFlay = 0.1f;
    private float _speedRotation = 0.5f;

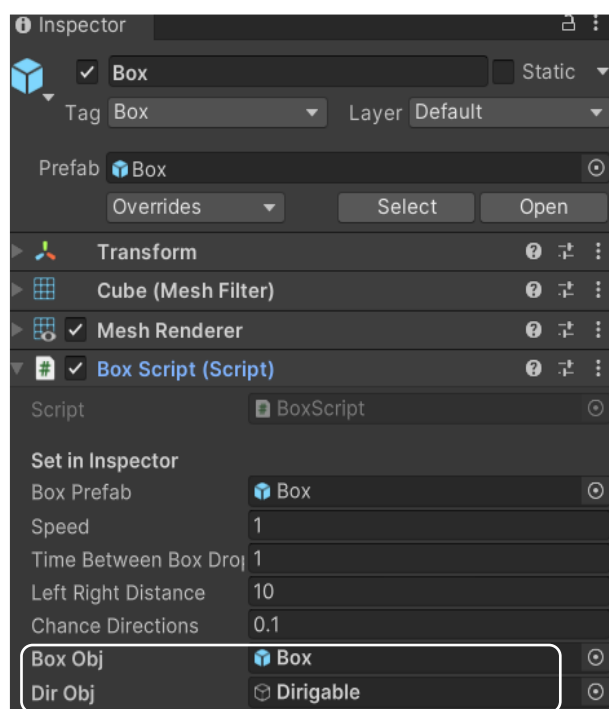
    void Start()
    {
    }
    void Update()
    {
        transform.Rotate(eulers: Vector3.up * (Time.deltaTime * _speedRotation),
            relativeTo: Space.World);
        transform.localPosition =
            new Vector3(transform.localPosition.x,
                y: Mathf.PingPong(t: Time.time, _lengthFlay),
                transform.localPosition.z);
    }
}
```

Если выбрать объект дирижабль Dirigable в иерархии, то в окне Inspector можно увидеть, что добавился компонент Animator. Он и воспроизводит нашу анимацию.



Так как дирижабль Dirigable вращается и движется вдоль оси X, то координата X ящика Box должна совпадать с координатой X дирижабля Dirigable.

Для этого в инспекторе для скрипта объекта Box следует добавить (просто перетащить) DirObj.



Код BoxScript.cs изменится так:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class BoxScript : MonoBehaviour
{
    [Header("Set in Inspector")]
    public GameObject BoxPrefab;
    public float speed = 1f;
    public float timeBetweenBoxDrops = 10f;
    public float leftRightDistance = 10f;
    public float chanceDirections = 0.1f;

    public static float bottomY = 0.0f;

    public GameObject BoxObj;
    public GameObject DirObj;
    private DirScript DirCoord;
```

```

void Start()
{
    DirCoord = DirObj.GetComponent<DirScript>();
    Invoke("DropBox", 2f);
}
private void OnTriggerEnter(Collider other) [ . . . ]

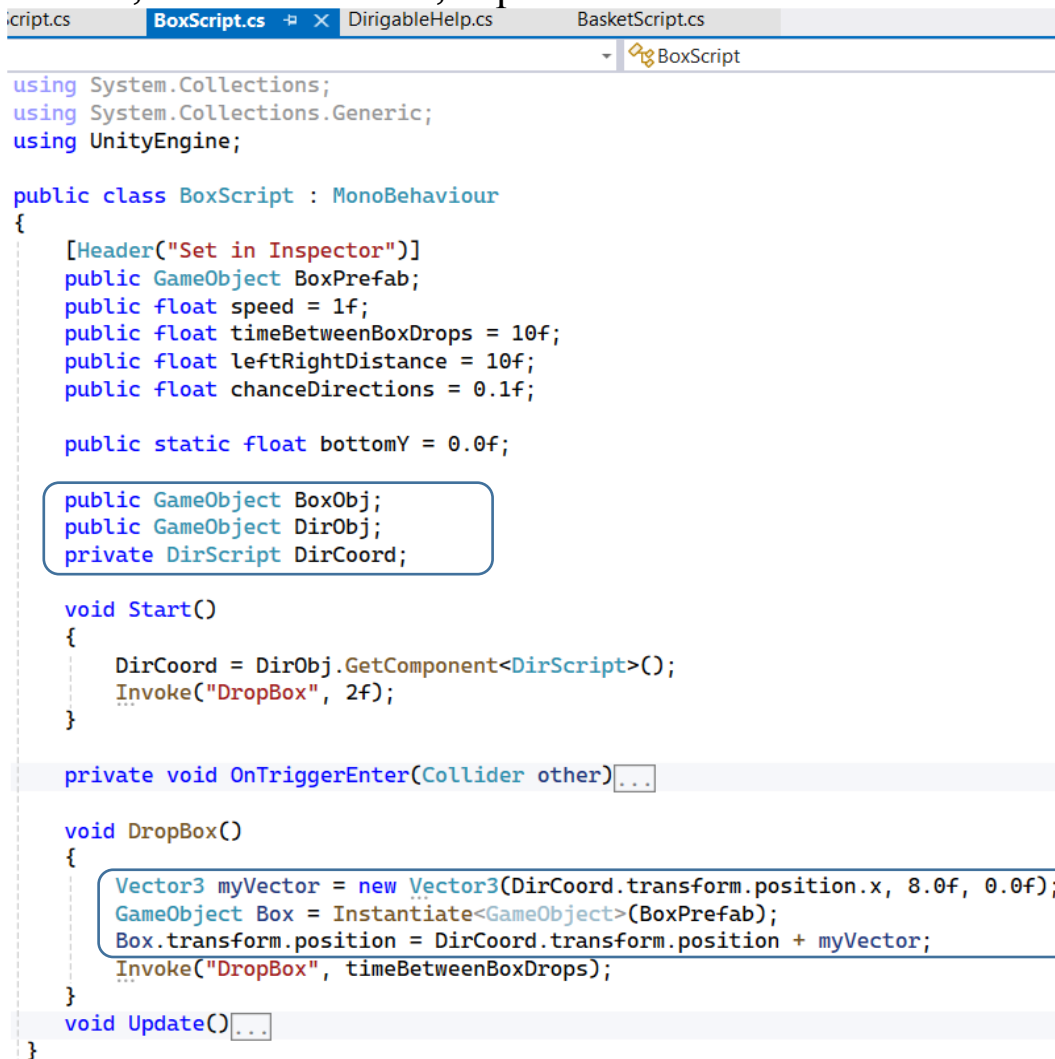
void DropBox()
{
    Vector3 myVector = new Vector3(DirCoord.transform.position.x,
                                   8.0f,
                                   0.0f);
    GameObject Box = Instantiate<GameObject>(BoxPrefab);
    Box.transform.position = DirCoord.transform.position +
                           myVector;
    Invoke("DropBox", timeBetweenBoxDrops);
}
void Update()
{
    Vector3 posBox = transform.position;
    posBox.x += speed * Time.deltaTime;
    transform.position = posBox;

    if (posBox.x < -leftRightDistance)
    {
        speed = Mathf.Abs(speed);
    }
    else if (posBox.x > leftRightDistance)
    {
        speed = -Mathf.Abs(speed);
    }
    if (transform.position.y < bottomY)
    {
        Destroy(BoxObj);
        DirigableHelp apScript=Camera.main.GetComponent<DirigableHelp>();
        apScript.BoxDestroyed();
    }

    if (UnityEngine.Random.value < chanceDirections)
    {
        speed *= -1;
    }
}
}

```

И, соответственно, скриншот листинга:



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class BoxScript : MonoBehaviour
{
    [Header("Set in Inspector")]
    public GameObject BoxPrefab;
    public float speed = 1f;
    public float timeBetweenBoxDrops = 10f;
    public float leftRightDistance = 10f;
    public float chanceDirections = 0.1f;

    public static float bottomY = 0.0f;

    public GameObject BoxObj;
    public GameObject DirObj;
    private DirScript DirCoord;

    void Start()
    {
        DirCoord = DirObj.GetComponent<DirScript>();
        Invoke("DropBox", 2f);
    }

    private void OnTriggerEnter(Collider other) { ... }

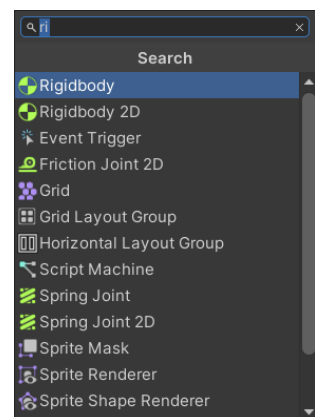
    void DropBox()
    {
        Vector3 myVector = new Vector3(DirCoord.transform.position.x, 8.0f, 0.0f);
        GameObject Box = Instantiate<GameObject>(BoxPrefab);
        Box.transform.position = DirCoord.transform.position + myVector;
        Invoke("DropBox", timeBetweenBoxDrops);
    }

    void Update() { ... }
}
```

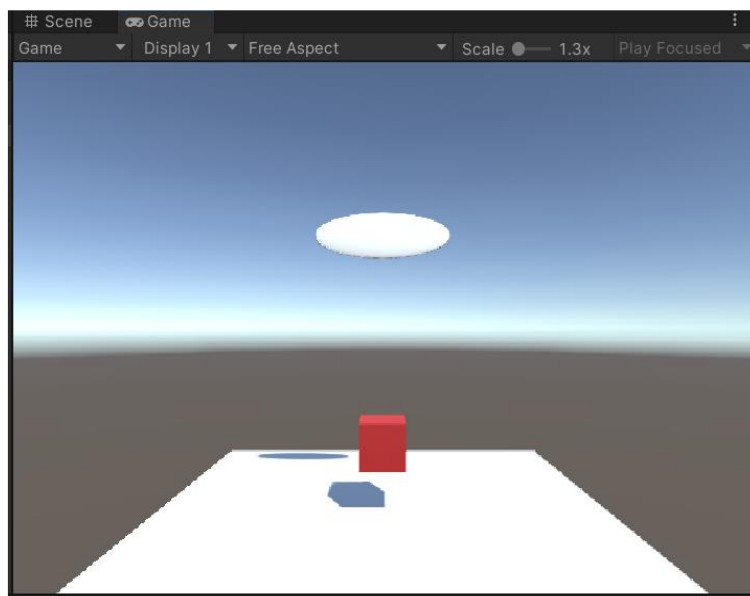
5.3. СОЗДАНИЕ ЯЩИКА С ПОДАРКОМ

По аналогии создадим и назначим материал для Вох (VoxMat).

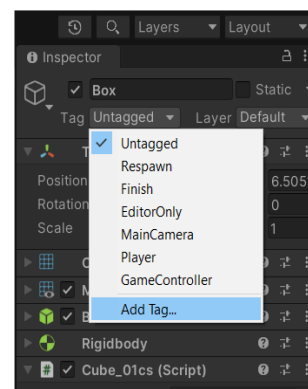
Добавим ящику свойство твердого тела. Выберите объект Вох в панели Hierarchy и в окне Inspector нажмите кнопку Add Component. Откроется окно добавления компонентов объекту. В появившейся строке поиска напишите Rigidbody и кликните по найденному компоненту Rigidbody в окне поиска Search.



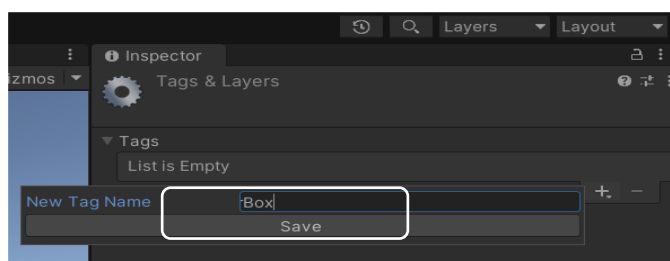
Теперь при воспроизведении сцены (нажатии кнопки Play) ящик Вох будет падать, тогда как дирижабль останется парить в воздухе.



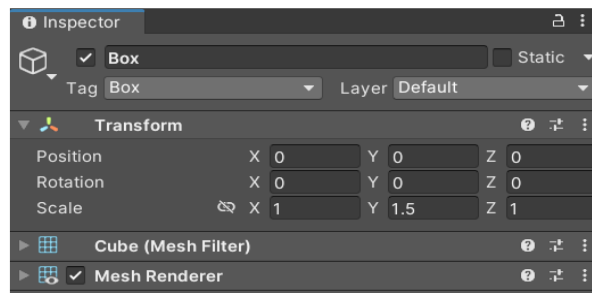
Так как на экране будет появляться множество объектов Box, позднее нам придется работать с массивами этих объектов. Для простоты дальнейшей работы привяжем к Box специальный тег. Для этого выберите Box в окне иерархии Hierarchy, далее нажмите кнопку Tag в окне Inspector и выберите Add Tag...:



После этого откроется диспетчер тегов и слоев. Внутри окна Tags & Layers кликните по значку «+» и добавьте новый тег с именем Box. После того как напишите новое имя, нажмите Save:



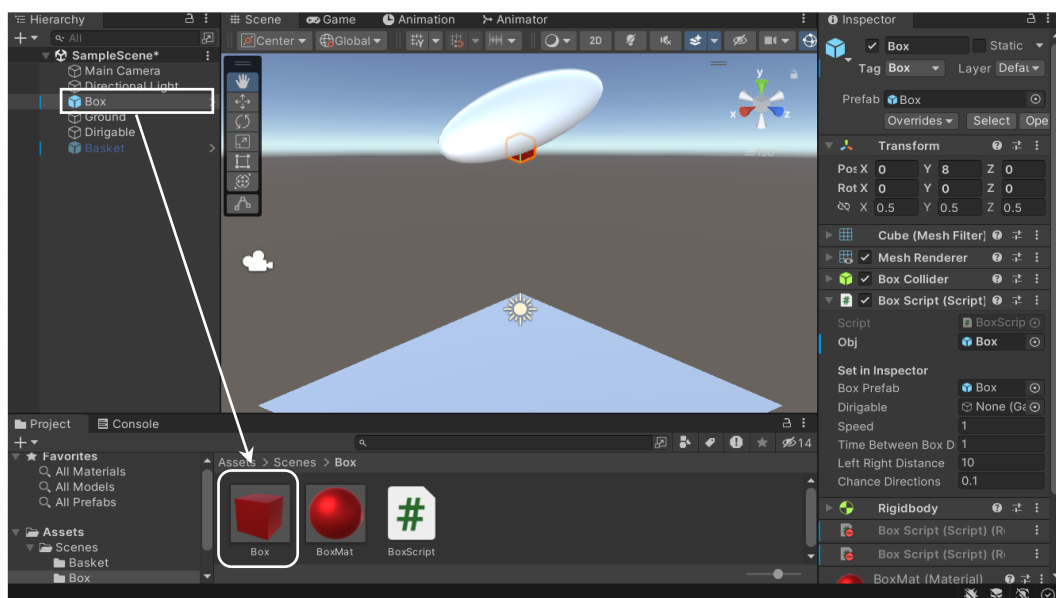
Теперь в поле Tag (окно Inspector) для объекта Box можно задать новый тег с именем Box:



Так как ящик будет генерироваться автоматически, то его нахождение на сцене не требуется. В этом случае в Unity принято работать с так называемыми префабами. Unity prefab – это особый тип шаблона для объекта, позволяющий хранить весь GameObject со всеми компонентами и значениями свойств.

С помощью префабов можно создать «образец» предмета с определенными свойствами, а потом использовать такие предметы на всей игровой сцене. Если изменить префаб, то изменятся все объекты, созданные на его основе.

Теперь создадим префаб для Box. Чтобы это сделать, просто перетащите 3D-объект с именем Box из окна Hierarchy в окно Project:



Если все сделано правильно, то Box в окне Hierarchy поменяет свой цвет.

В листинг добавили следующие строки кода:

– `Invoke("DropBox", 2f);` позволяет выполнять какие-либо действия отложенным образом, в том числе и по времени.

`Invoke` полагается на `Time.timeScale`. То есть, если вызывается `Invoke("DropBox", 2f)` при `Time.timeScale = 0.5`, то `DropBox()` начнет выполняться через 4 с., а при 0 – `DropBox` не начнет выполняться вовсе;

- с помощью команды `[Header("Set in Inspector")]` создали заголовок в окне Inspector;

- `public GameObject BoxPrefab` создает префаб для ящика `Box`;

- созданная переменная `speed` определяет скорость движения объекта `Box`;

- переменная `timeBetweenBox` определяет скорость генерации объектов `Box`;

- переменная `leftRightDistance` определяет расстояние от края экрана, на котором меняется направление движения;

- `chanceDirections` определяет вероятность изменения направления движения.

- `Vector3 pos` – это локальная переменная для хранения текущей позиции;

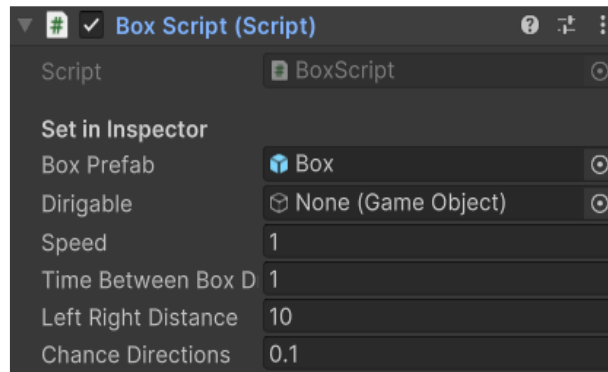
- `x` увеличивается на произведение скорости и временного интервала `Time.deltaTime`. Это встроенная переменная, которая будет изменять свое значение при разной частоте кадров. Если говорить простыми словами, то она отвечает за плавность игры при разном FPS;

- если значение `pos.x` оказалось меньше значения `leftRightDistance` – переменная `speed` имеет положительное значение (движение вправо), иначе, если значение `pos.x` оказалось больше значения `leftRightDistance` – переменной `speed` присваивается отрицательное значение (движение влево).

Подобрать значение переменной `chanceDirections` можно так, чтобы объект `Box` случайно менял направление в среднем 1 раз в секунду. Подобрать подходящее значение поможет формула

$50 \text{ вызовов FixedUpdate/сек.} \cdot \text{вероятность} = \text{среднее число срабатываний в секунду.}$

Подбирать подходящие значения можно в окне Inspector (сразу после сохранения скрипта). Например, можно использовать следующие значения:



5.3.1. Комментарии

Таким образом, мы создали заготовку для нашего будущего проекта и даже добавили парящий дирижабль с анимацией. Для того, чтобы более глубоко разобраться в проекте и получить игру, которая будет отличаться от этой, рекомендуем попробовать самостоятельно внести следующие изменения в проект:

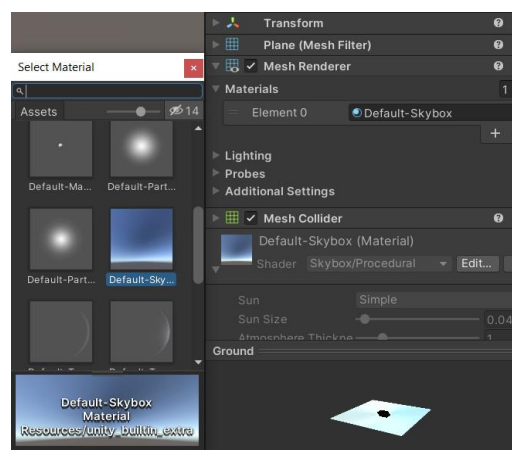
- заменить префаб с дирижаблем на любой другой;
- заменить анимацию полета;
- поэкспериментировать со значениями scale в префабе, это позволит изменить размер игрового объекта.

5.4. НАСТРОЙКА ПОВЕРХНОСТИ

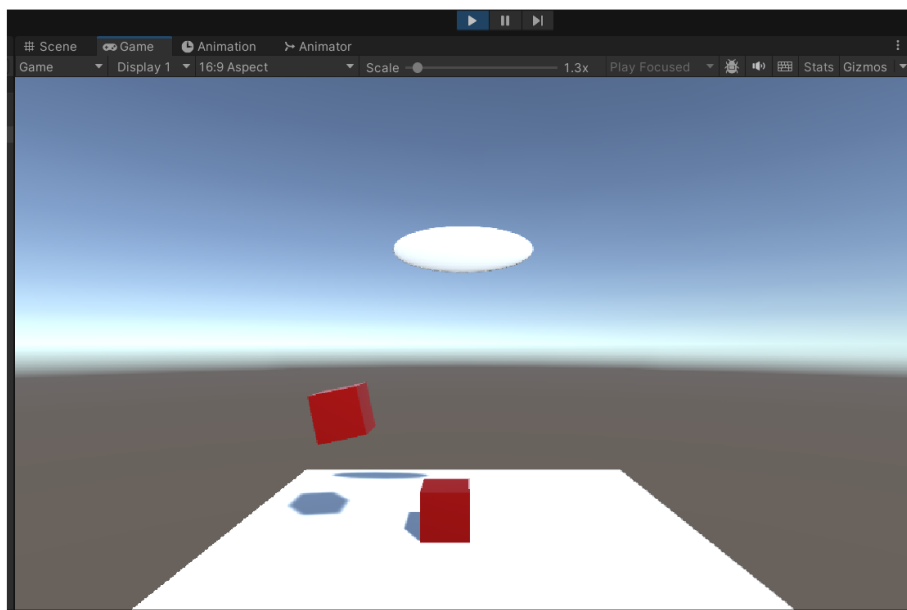
Установим размеры объекта Ground:

- Position: 0, -8, 0;
- Rotation: 0, 0, 0;
- Scale: 5, 5, 5;

Изменим визуальное оформление элемента Ground. В окне Inspector (в правой части) разверните компонент Mesh Renderer(), нажмите на «мишень» и из появившегося меню Select Material выберите материал, например, Default – SkyBox.



Запуская сцену, можем проверить ее работу. Выглядит она примерно так:



Можно легко найти различные текстуры и использовать для объектов.

Генерируемые объекты Box все время существуют на сцене после создания. Со временем их количество будет увеличиваться, поэтому будет правильнее удалять объекты Box после того, как они будут уходить за пределы сцены.

Настройку визуальных эффектов мы можем разделить на два этапа:

- написание кода в скрипт-файл в **BoxScript.cs**;
- настройка свойств Particle System в окне Inspector элемента Box.prefab.

Код скрипт-файла **BoxScript.cs**:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class BoxScript : MonoBehaviour
{
    public GameObject BoxObj;
    [Header("Set in Inspector")]
    public GameObject BoxPrefab;
    public float speed = 1f;
    public float timeBetweenBoxDrops = 1f;
    public float leftRightDistance = 10f;
    public float chanceDirections = 0.1f;
```

```

public static float bottomY = 0f;
void Start()
{
    Invoke("DropBox", 2f);
}
private void OnTriggerEnter(Collider other)
{
    ParticleSystem ps = GetComponent<ParticleSystem>();
    var em = ps.emission;
    em.enabled = true;
    Renderer rend;
    rend = GetComponent<Renderer>();
    rend.enabled = false;
}
void DropBox()
{
    Vector3 myVector =
    new Vector3(DirCoord.transform.position.x, 8.0f, 0.0f);
    GameObject Box = Instantiate<GameObject>(BoxPrefab);
    Box.transform.position =
        DirCoord.transform.position + myVector;
    Invoke("DropBox", timeBetweenBoxDrops);
}
void Update()
{
    Vector3 posBox = transform.position;
    posBox.x += speed * Time.deltaTime;
    transform.position = posBox;
    if (posBox.x < -leftRightDistance)
    {
        speed = Mathf.Abs(speed);
    }
    else if (posBox.x > leftRightDistance)
    {
        speed = -Mathf.Abs(speed);
    }
    if (transform.position.y < bottomY)
    {
        Destroy(BoxObj);
    }
}
void FixedUpdate()
{
    if (UnityEngine.Random.value < chanceDirections)
    {
        speed *= -1;
    }
    if (transform.position.y < bottomY)
    {
        Destroy(BoxObj);
    }
}
}

```

В скрипт-файле основные функциональные блоки выполняют следующие действия:

- создается переменная **bottomY**, которая будет указывать, на каком расстоянии Y нужно будет удалять объекты **Box**;
- метод **OnTriggerEnter (Collision Other)** начинает работу, когда происходит пересечение с объектом, который работает как **Trigger**. В нашем случае таким объектом-триггером будет выступать плоскость **Ground**. При срабатывании триггера запускается проигрывание **Particle System (em.enabled = true)**, и ящик становится невидимым (**rend.enabled = false**);
- внутри метода **Update** происходит уничтожение объекта **Box (Destroy(this.gameObject))**, если ящик падает ниже уровня **bottomY**. Это позволяет избежать накопления ненужных объектов за пределами игровой сцены.

Сохраните сценарий **BoxScript.cs** и подключите его к префабу (шаблону) **Box.prefab** в панели **Project**. Для этого можно просто перетащить файл **BoxScript.cs** на префаб **BoxScript.prefab**.

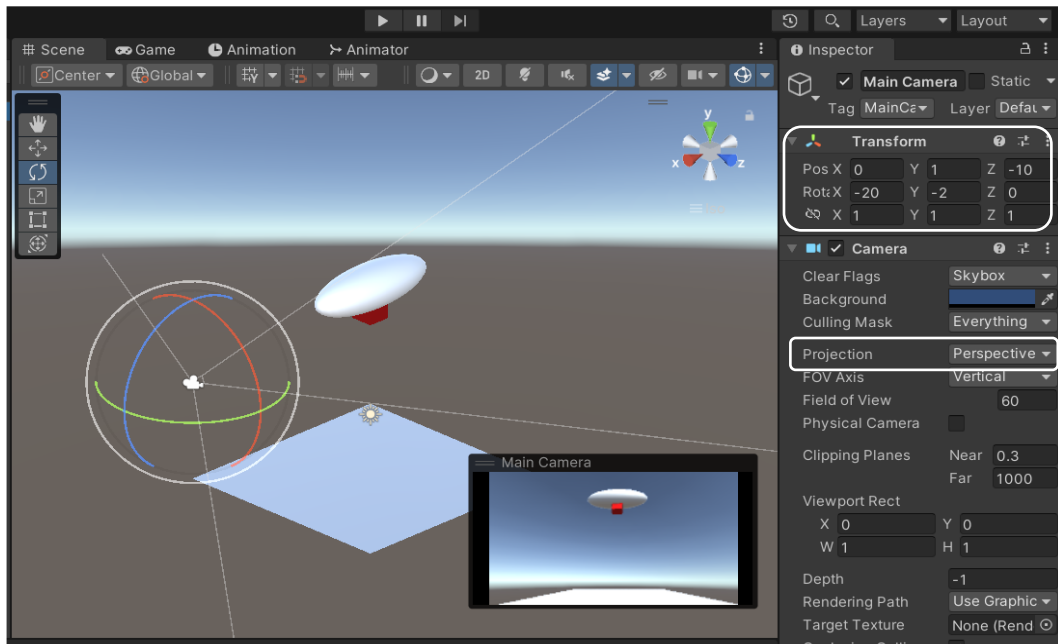
5.5. НАСТРОЙКА КАМЕРЫ И ИГРОВОЙ ОБЛАСТИ

Как и многое в разработке игр, поиск оптимальных настроек камеры – итеративный процесс. Всегда можно настроить камеру таким образом, чтобы получить наиболее подходящий вид игровой области.

Выберите объект **Main Camera** в иерархии объектов. Настройте компонент **Transform**:

- **Position: 0, 1, -10;**
- **Rotation: -20, - 2, 0;**
- **Scale: 1, 1, 1.**

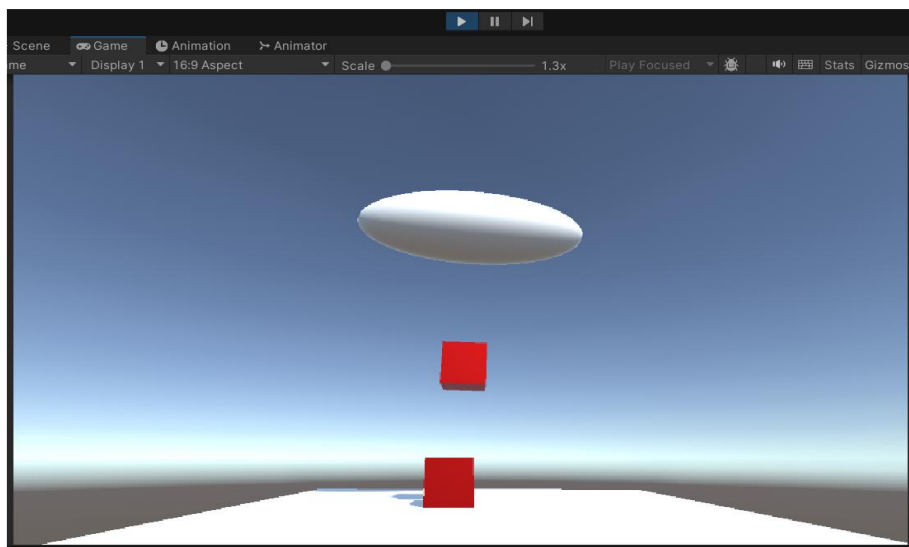
В инспекторе для компонента **Camera** установите в поле **Projection – Perspective**.



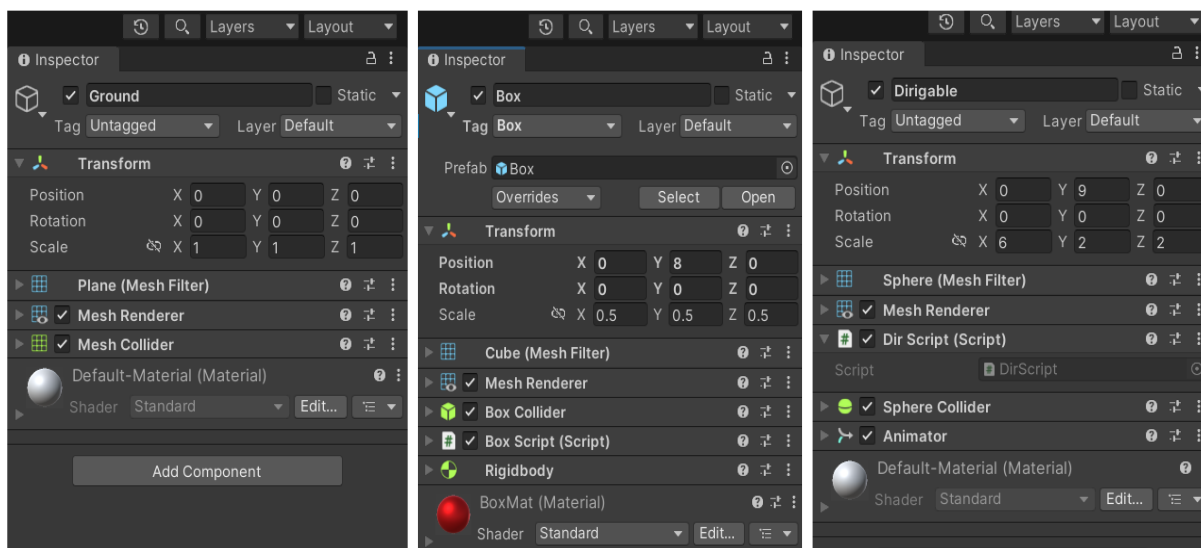
Еще один важный фактор – соотношение сторон панели Game. Вверху панели отображается значение Free Aspect. Выберите из списка 16:9 – стандартное соотношение сторон для большинства современных экранов.



При запуске сцены (кнопка Play) показан вид окна игры в соответствии с выбранными настройками камеры:



Для создания более реалистичной картины установим размеры и координаты наших объектов: поверхности Ground, ящика с подарками Box и дирижабля Dirigable:



Не забудем в скрипте `BoxScript.cs` изменить `public static float bottomY = 0f;`

5.5.1. Комментарии

Таким образом, мы создали заготовку для нашего будущего проекта и даже добавили дирижабль с анимацией. Для того, чтобы более глубоко разобраться в проекте и получить игру, которая будет отличаться от этой, рекомендуем попробовать самостоятельно внести следующие изменения в проект:

- заменить префаб на любой другой;
- поэкспериментировать со значениями `scale` в префабе для изменения размеров игрового объекта.

6. ПРОГРАММИРОВАНИЕ ИГРОВЫХ ОБЪЕКТОВ

При написании сценариев будем использовать пошаговое добавление строк кода, давая описание команд по ходу добавления нового функционала в скрипт-файлы. Как было отмечено ранее, действия игровых объектов описываются в так называемых Script-файлах. Эти файлы содержат код на языке C# и имеют расширение **.cs**. При этом следует отметить, что предварительная компиляция файлов не требуется, Unity берет эту работу на себя в режиме реального времени в процессе работы в среде разработки.

6.1. СКРИПТ-ФАЙЛ **Box** ДЛЯ РАЗРУШЕНИЯ «ЯЩИКА»

Опишем работу падающих «ящиков» с подарками. Дополним сценарий **BoxScript.cs**.

Различные визуальные эффекты можно взять из Asset Store.

Откройте сценарий **DirScript.cs**, подключенный к игровому объекту **DirScript.prefab**. Этот игровой объект «дирижабль» сбрасывает «ящики» через равные временные интервалы. В скрипт-файл **BoxScript.cs** добавим следующий функционал при столкновении с плоскостью **Ground** создадим эффект разрушения ящика. Будет запускаться **Particle System** – компонент, который может имитировать генерацию частиц разного вида (используется при добавлении взрывов, эффектов дыма, молний, тумана и т. д.). В нашем случае будем использовать **Particle System** для имитации разрушения.

Генерируемые объекты **Box** все время существуют на сцене после создания. Со временем их количество будет увеличиваться, поэтому будет правильнее удалять объекты **Box** после того, как они будут уходить за пределы сцены.

Настройку визуальных эффектов мы можем разделить на два этапа:

- написание кода в скрипт-файл в **BoxScript.cs**;
- настройка свойств **Particle System** в окне Inspector элемента **BoxScript.prefab**.

Приступим к написанию кода. Для этого добавим в скрипт-файл **BoxScript.cs** следующие строки кода:

```
public class BoxScript : MonoBehaviour
{
    public GameObject BoxObj;

    [Header("Set in Inspector")]
    public GameObject BoxPrefab;
    public GameObject Dirigable;
    public float speed = 1f;
    public float timeBetweenBoxDrops = 1f;
    public float leftRightDistance = 10f;
    public float chanceDirections = 0.1f;
```

```
public static float bottomY = 0f;
```

```
private void OnTriggerEnter(Collider other)
{
    ParticleSystem ps = GetComponent<ParticleSystem>();
    var em = ps.emission;
    em.enabled = true;
    Renderer rend;
    rend = GetComponent<Renderer>();
    rend.enabled = false;
}
```

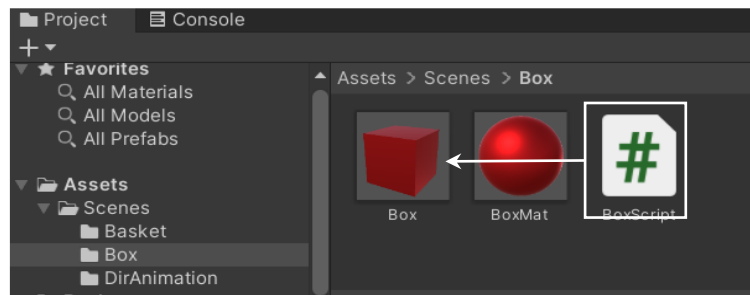
В скрипт-файле основные функциональные блоки выполняют следующие действия.

Создается переменная **bottomY**, которая будет указывать, на каком расстоянии Y нужно будет удалять объекты **Box**.

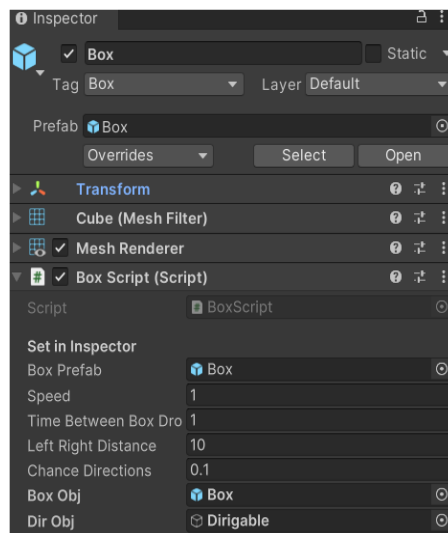
Метод **OnTriggerEnter (Collision Other)** начинает работу, когда происходит пересечение с объектом, который работает как **Trigger**. В нашем случае таким объектом-триггером будет выступать плоскость **Ground**. При срабатывании триггера запускается проигрывание **Particle System** (**em.enabled = true**), и «ящик» становится невидимым (**rend.enabled = false**).

Внутри метода **Update** происходит уничтожение объекта **Box** (**Destroy(this.gameObject)**), если ящик падает ниже уровня **bottomY**. Это позволяет избежать накопления ненужных объектов за пределами игровой сцены.

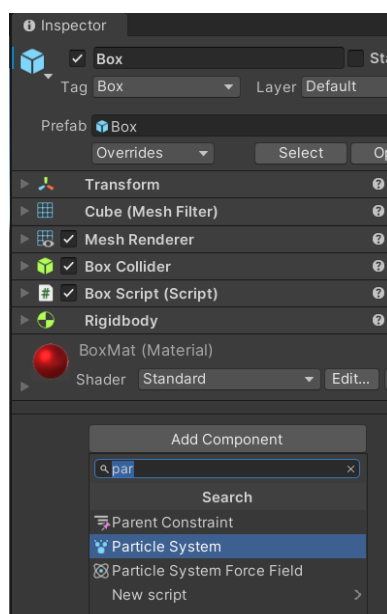
Сохраните сценарий **BoxScript.cs** и подключите его к префабу (шаблону) **BoxScript.prefab** в панели **Project**. Для этого просто перетащите файл **BoxScript.cs** на префаб **BoxScript.prefab**:



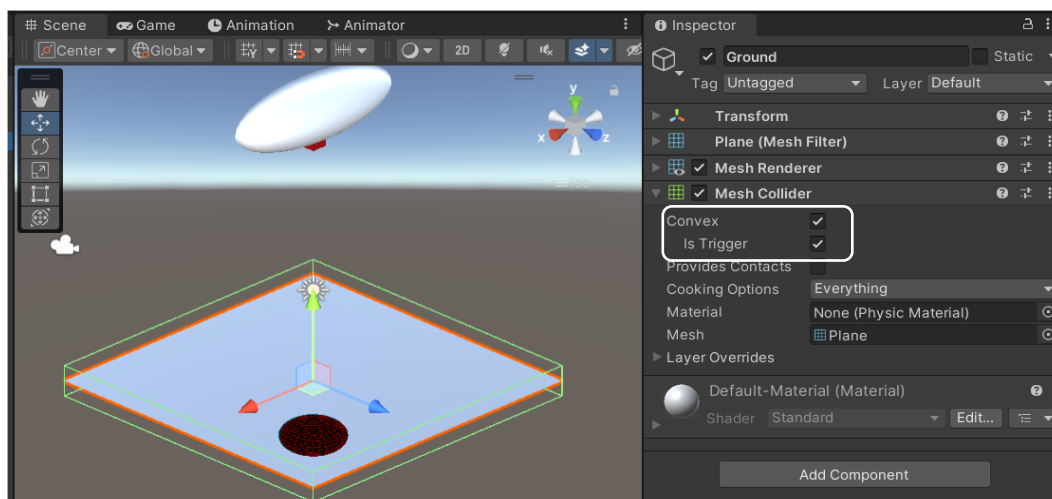
Можно убедиться, что в окне Inspector появится компонент с подключенным Script-файлом.



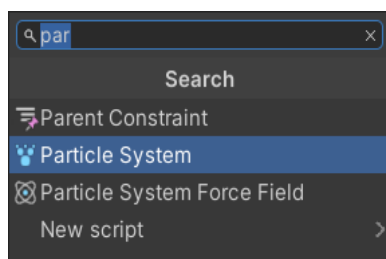
Выберем Box.prefab и добавим компонент Particle System:



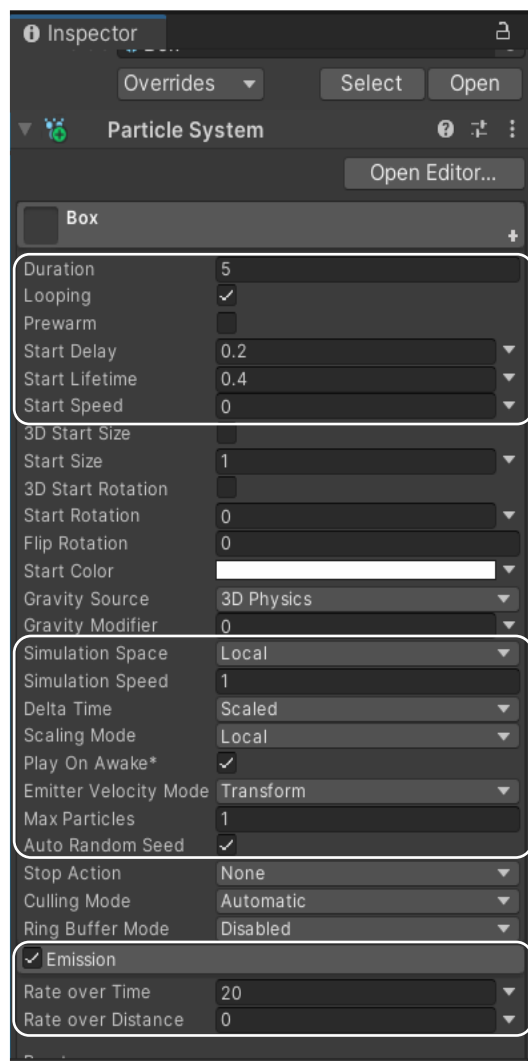
Теперь настроим объект **Ground**, чтобы он работал в качестве триггера. Сделать это можно так: в окне Hierarchy выберите объект **Ground** и после этого в окне Inspector поставьте две галочки в компоненте Mesh Collider – Convex, Is Trigger:



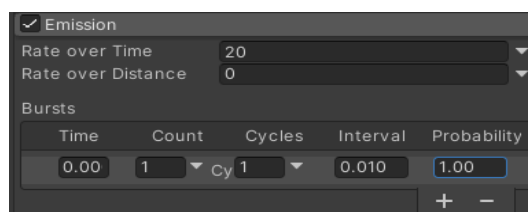
Нажимая Add Component, добавим Particle System.



Изменим настройки **Particle System**, чтобы они имели внешний вид, похожий на разрушение. Для этого выберите элемент **Box**, в окне Inspector разверните компонент **Particle System**. Он содержит большое количество дополнительных настроек, ниже выделены основные, на которые следует обратить внимание. Настройте компонент так, как показано на рисунке ниже.



Поставьте галочку возле Emission и введите параметры, показанные на рисунке ниже (чтобы добавить новую линию с параметрами «всплесков» нажмите «+»).



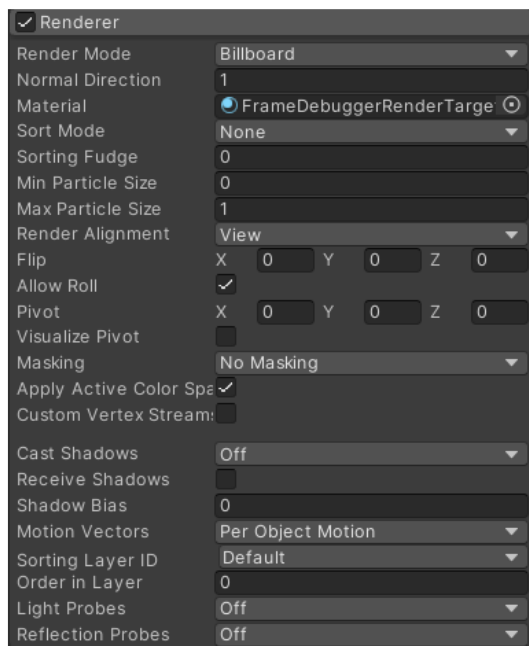
Теперь снимите галочку с Emission. Это делается для того, чтобы при запуске сцены этот параметр был выключен и объект не излучал Particles с эффектами разрушения.

Поставьте галочку напротив «Size over Lifetime» и выберите вид кривой, которая растет от минимума слева до максимума справа. Эта кривая означает, что элемент Particle будет

распространяться от центра в разные стороны, как это происходит во время взрыва. Существует несколько типовых видов кривых, которые позволяют создавать совершенно разные эффекты.



Последний элемент, с которым мы поработаем внутри Particle System – это Renderer, в нем назначается текстура частицы, которая генерируется при запуске. Выберем в качестве текстуры материал с названием FrameDebuggerRenderTargetD. Позже можно найти различные текстуры и поэкспериментировать с ними.



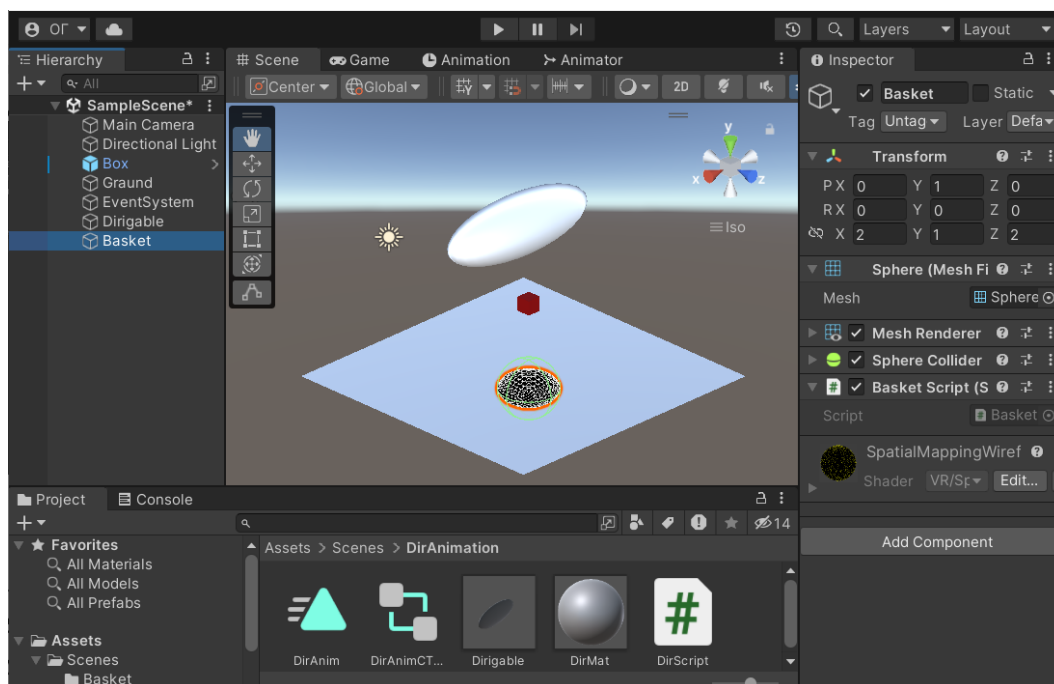
Теперь запускаем сцену и проверяем, что при столкновении «ящика» с плоскостью создается эффект разрушения. Но если быть точным, то на самом деле происходит следующее:

- при столкновении объекта **Box** с **Ground** срабатывает триггер, после этого «ящик» перестает отображаться (`rend.enabled = false`);
- запускается работа **Particle System**, имеющей визуальное оформление взрыва (`em.enabled = true`);
- объект **Box** уничтожается при падении ниже уровня `y = bottomY`.

Таким образом, описан функционал объекта **Box**, создан эффект разрушения, скрыты и удалены объекты при наступлении различных событий.

6.2. СОЗДАНИЕ ДВИЖУЩЕЙСЯ «КОРЗИНЫ» BASKET

Падающие «ящики» с подарками будем ловить «корзиной». Как обычно, создаем 3D-объект «сфера», даём ей имя **Basket** и задаём свойства, приведённые ниже.



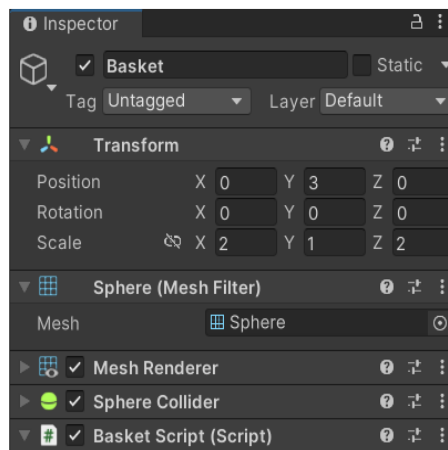
Обращаем внимание, что корзина **Basket** и ящик **Box** должны находиться в одной плоскости, т.е. в Transform Position значения X и Z должны совпадать.

В нашем случае:

Box Position: X: 0; Y: 8; Z: 0,

Basket Position: X: 0; Y: 1; Z: 0.

Установим размеры и координаты корзины **Basket**:



Напишем код для перемещения игрового объекта **Basket** вслед за указателем мыши. При желании можно изменить интерфейс ввода, адаптированный для клавиатуры, Touch Screen мобильных устройств (смартфонов, планшетов) и изменить тем самым механику игры.

Создадим скрипт **BasketScript.cs**, который будет отвечать за перемещение «корзины» **Basket** вслед за курсором мыши и ловлю объектов **Box** в «корзину» **Basket**. Этот метод **OnCollisionEnter** следует добавить ниже метода **Update()**:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class BasketScript : MonoBehaviour
{
    void Update()
    {
        Vector3 mousePos2D = Input.mousePosition;
        mousePos2D.z = -Camera.main.transform.position.z;
        Vector3 mousePos3D = Camera.main.ScreenToWorldPoint(mousePos2D);
        Vector3 pos = this.transform.position;
        pos.x = mousePos3D.x;
        this.transform.position = pos;
    }
    private void OnCollisionEnter(Collision coll)
    {
        GameObject Collided = coll.gameObject;
        if (Collided.tag == "Box")
        {
            Destroy(Collided);
        }
    }
}
```

Скриншот листинга.

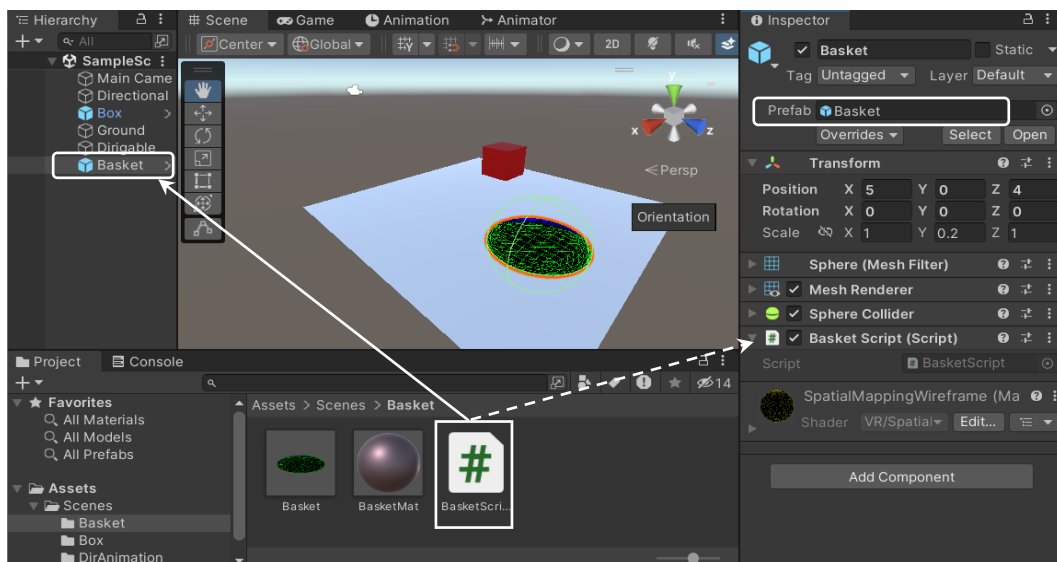
```
BasketScript.cs* [X] BoxScript.cs
BasketScript

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class BasketScript : MonoBehaviour
{
    void Update()
    {
        Vector3 mousePos2D = Input.mousePosition;
        mousePos2D.z = -Camera.main.transform.position.z;
        Vector3 mousePos3D = Camera.main.ScreenToWorldPoint(mousePos2D);
        Vector3 pos = this.transform.position;
        pos.x = mousePos3D.x;
        this.transform.position = pos;
    }

    private void OnCollisionEnter(Collision coll)
    {
        GameObject Collided = coll.gameObject;
        if (Collided.tag == "Box")
        {
            Destroy(Collided);
        }
    }
}
```

Подключим скрипт `BasketScript.cs` к шаблону `BasketScript Prefab`:

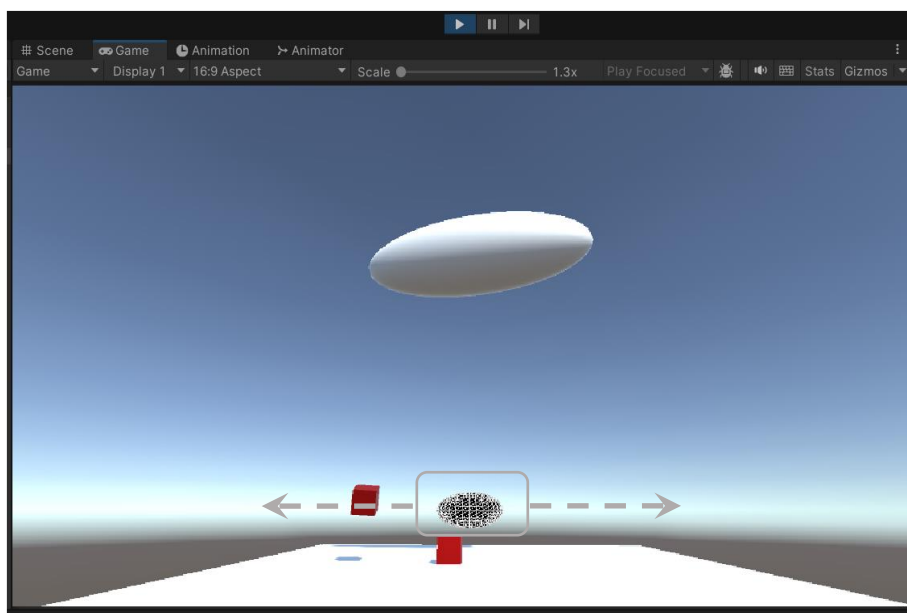


Метод `OnCollisionEnter` срабатывает (вызывается) каждый раз, когда с коллайдером `Basket` что-то сталкивается. В локальную переменную `Collided` записывается ссылка на игровой объект (`coll.gameObject`), столкнувшийся с корзиной.

Далее следует проверка, если этот объект имеет тег **Box**, то он уничтожается **Destroy(Collided)**.

Сохраните сценарий, запустите игру (**Play**) и проверьте, что «ящики» с подарками **Box** ловятся (точнее уничтожаются) при взаимодействии с «корзиной» **Basket**.

Теперь игра выглядит так.



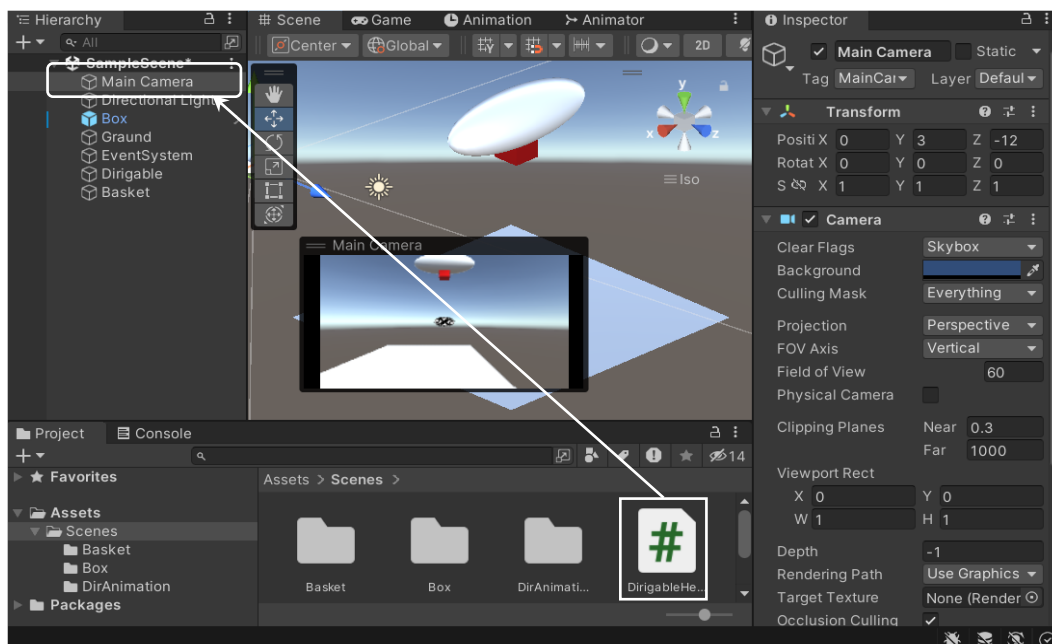
6.2.1. Комментарии

Можно самостоятельно расширить функционал и правила игры. Добавляйте только те элементы, которые можно реализовать с незначительными временными затратами. Помните, что внесение даже самых маленьких дополнений в игру может кардинально изменить её восприятие и привести к неработоспособности или некорректной работе игры в самых непредвиденных местах.

6.3. СКРИПТ-ФАЙЛ СЦЕНАРИЯ ИГРЫ

Практически во всех играх существует сценарий, который управляет игрой в целом: запуском и перезапуском сцены, сохранением, загрузкой и т.д. Мы создадим сценарий, который будет называться **DirigableHelp.cs**. Подключать сценарии, управляющие игрой, следует к тем объектам, которые гарантированно присутствуют в любой сцене, например, к **Main Camera**.

Создадим сценарий **DirigableHelp.cs** и подключим его (можно так же, как и ранее – перетаскиванием) к элементу **Main Camera** в окне **Hierarchy**:



Кроме управления игрой в сценарии будут создаваться три экземпляра **Basket** – главного персонажа, который будет ловить летящие вниз «ящики» с подарками. Условно игрок будет окружен «корзинами» **Basket**, символизирующими количество жизней.

Напишем код скрипта **DirigableHelp.cs**, который будет создавать три экземпляра шаблона **Basket**, располагая их на экране внутри друг друга.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DirigableHelp : MonoBehaviour
{
    public GameObject BasketPrefab;
    public int numBasket = 3;
    public float BasketBottomY = 2.0f;
    public float BasketRadius = 0.7f;
    public List<GameObject> basketList;

    void Start()
    {
        basketList = new List<GameObject>();
        for (int i = 1; i <= numBasket; i++)
```

```

    {
        GameObject tBasketGo =
            Instantiate<GameObject>(BasketPrefab);
        tBasketGo.transform.position =
            new Vector3(3.0f * i, BasketButtomY, 0f);
        tBasketGo.transform.localScale =
            new Vector3(BasketRadius * i,
                        BasketRadius * i,
                        BasketRadius * i);
        basketList.Add(tBasketGo);
    }
}
void Update()
{ }
}

```

Скриншот листинга

```

MainMenu.cs  DirigableHelp.cs  BoxScript.cs  DirScript.cs
bl
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

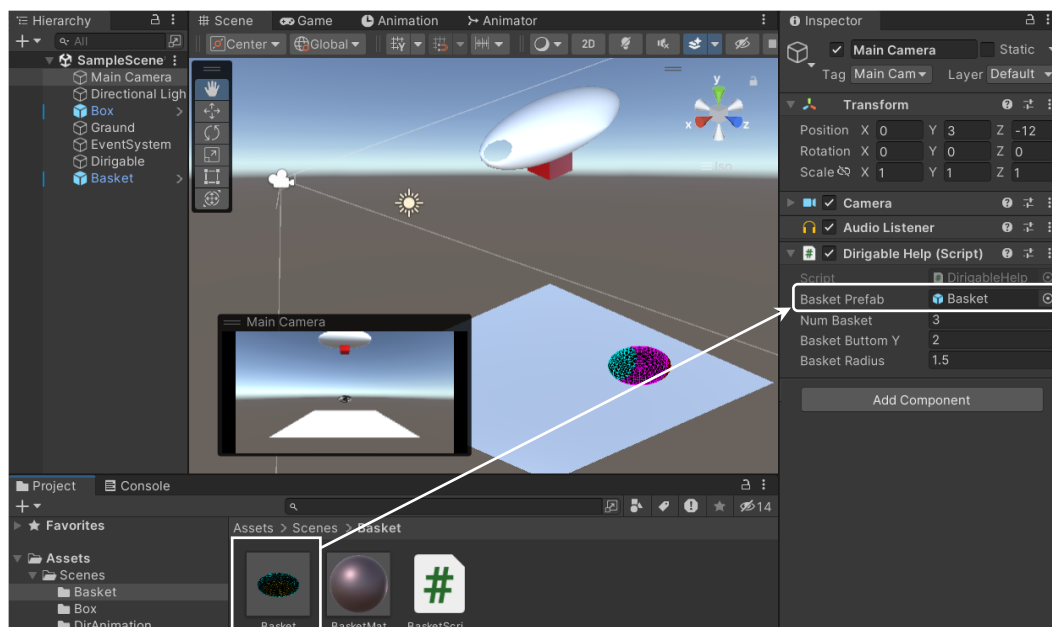
public class DirigableHelp : MonoBehaviour
{
    public GameObject BasketPrefab;
    public int numBasket = 3;
    public float BasketButtomY = 2.0f;
    public float BasketRadius = 0.7f;
    public List<GameObject> basketList;

    void Start()
    {
        basketList = new List<GameObject>();
        for (int i = 1; i <= numBasket; i++)
        {
            GameObject tBasketGo =
                Instantiate<GameObject>(BasketPrefab);
            tBasketGo.transform.position =
                new Vector3(3.0f * i, BasketButtomY, 0f);
            tBasketGo.transform.localScale =
                new Vector3(BasketRadius * i,
                            BasketRadius * i,
                            BasketRadius * i);
            basketList.Add(tBasketGo);
        }
    }

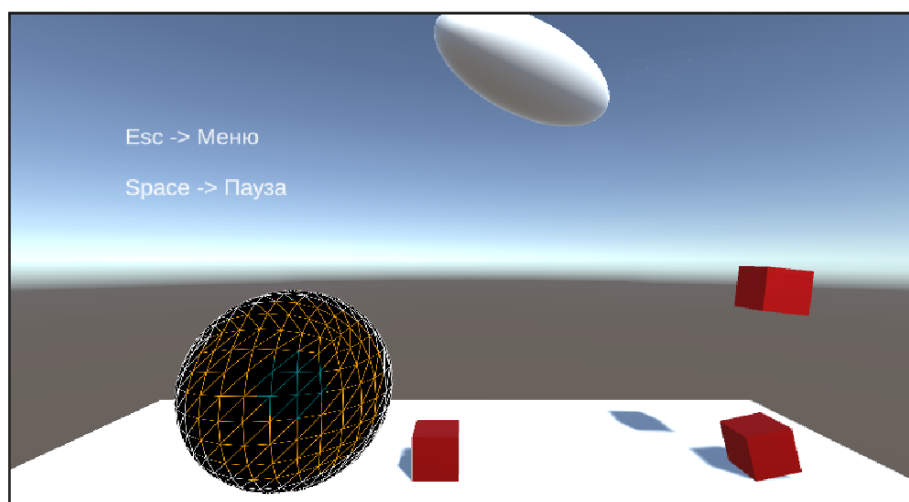
    void Update()
    { }
}

```

Чтобы подключить **Basket** к сценарию, выберите Main Camera в Inspector для поля Basket Prefab, установите шаблон «корзины» **Basket**:



Нажмите кнопку Play и убедитесь, что написанный код создает три экземпляра **Basket** (они разного размера и цвета).

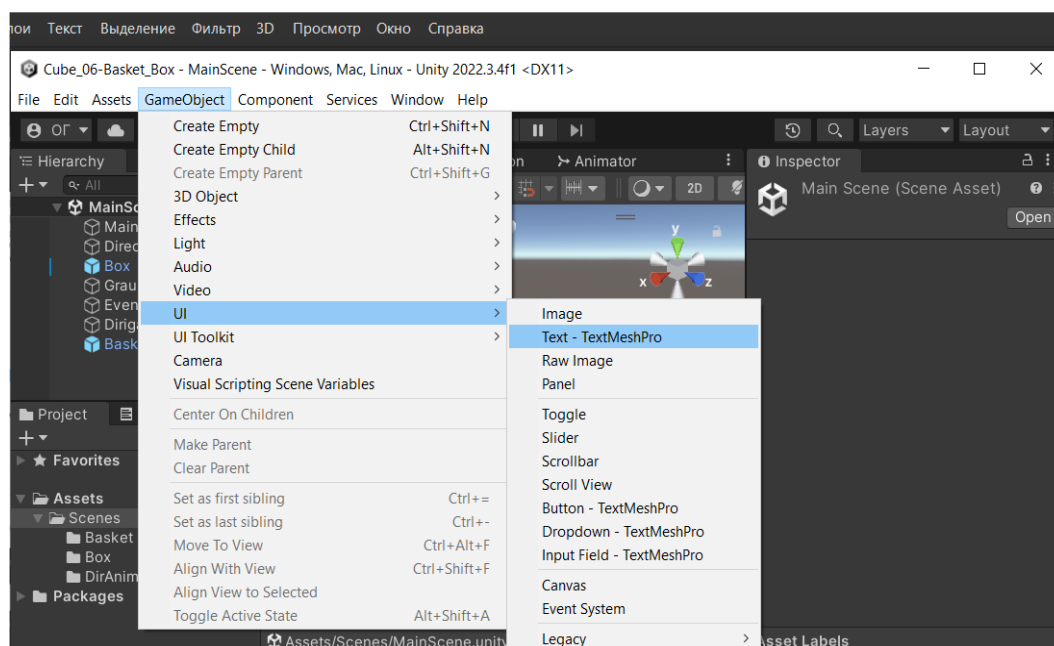


6.4. СОЗДАНИЕ СЧЕТЧИКА ОЧКОВ

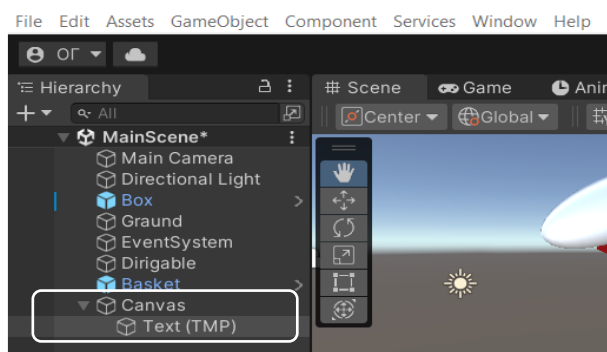
Задача счетчика очков заключается в том, чтобы дать пользователю обратную связь о результатах его действий или достижениях в игре.

Переименуем нашу главную сцену SampleScene в MainScene.

Дважды кликнем по MainScene в панели Project, и в меню Unity выберем пункт GameObject → UI → Text-TextMeshPro.



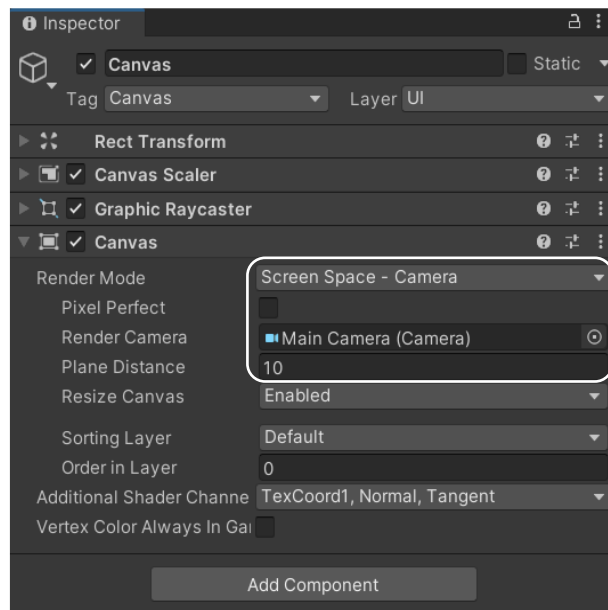
После этого на сцену добавится элемент графического интерфейса пользователя – Canvas:



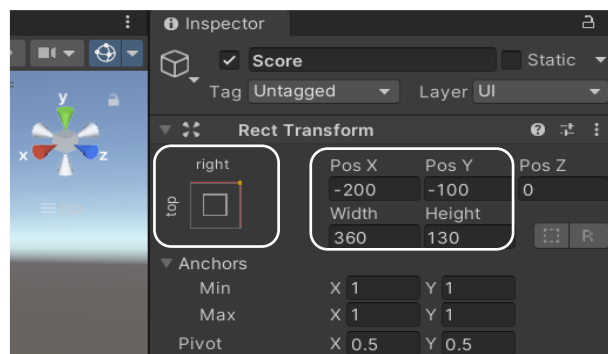
Дважды кликнем по Canvas, чтобы уменьшить его масштаб и увидеть весь объект целиком. Объект Canvas – это двумерный холст, на котором размещаются элементы графического интерфейса.

Переименуем дочерний объект Canvas Text (TMP) в Score.

В окне Hierarchy выделим Canvas и в окне Inspector развернём компонент Canvas. Установим в поле Render Mode значение Screen Space – Camera, а в качестве Render Camera выберите Main Camera (Camera). В поле Plane Distance установим значение, равное 10.

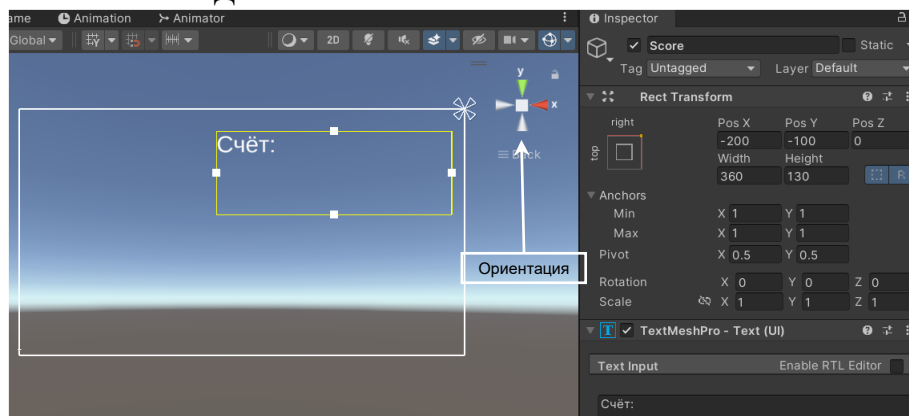


Настроим компонент RectTransform объекта Score так, как показано ниже. Якоря Anchors нужно установить в положение top – right и задать значения отступа сверху и справа. Таким образом, якорь как бы указывает на начальную точку, относительно которой нужно задавать координаты X и Y (Pos X, Pos Y).

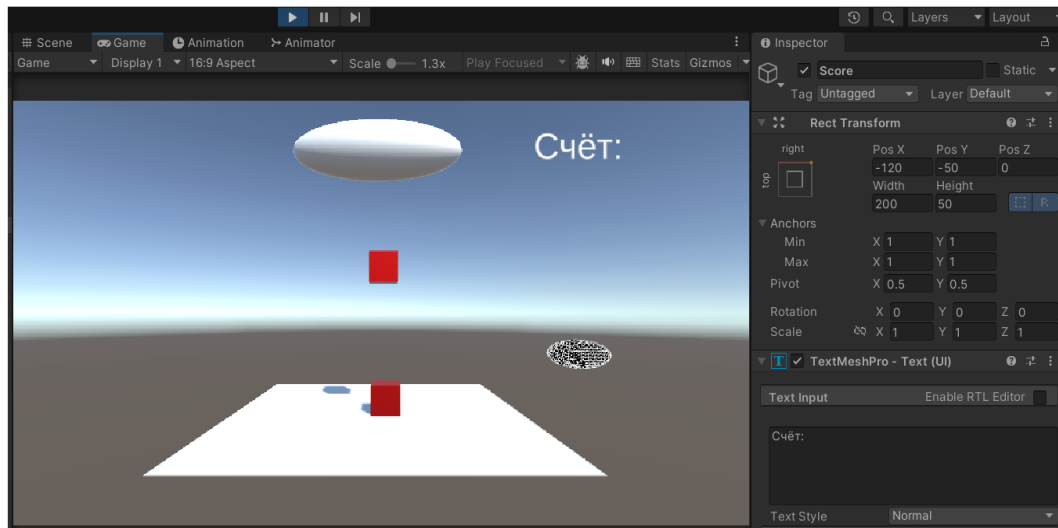


В компоненте Text можно задать надпись, которая будет отображаться на холсте, а также настройки шрифтов.

Изменяя ориентацию, можем добиться того, чтобы изображение выглядело так.



После проделанных операций на сцене можно увидеть созданный холст (Canvas), с расположенным на нем текстовым полем с надписью «Счёт:».



6.5. ДОБАВЛЕНИЕ ОЧКОВ ЗА ПОЙМАННЫЕ ОБЪЕКТЫ

Добавление очков должно происходить в том случае, когда объект **Box** (ящик с подарками) пересекается с **Basket** (корзиной), т.е. считается пойманным. Для того чтобы определять событие столкновения **Box** и **Basket**, можно использовать сценарии **BoxScript.cs** и **BasketScript.cs**.

В сценарии **BasketScript.cs** уже есть метод **private void OnCollisionEnter (Collision coll)**. Пока он отвечает только за то, что уничтожает **Box** после пересечения плоскости «земли». Будем использовать этот же метод для начисления очков за пойманный **Box**.

Добавьте в сценарий **BasketScript.cs** выделенные строки:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

public class BasketScript : MonoBehaviour
{
    public int score_int = 0;
    public TextMeshProUGUI score_Txt;

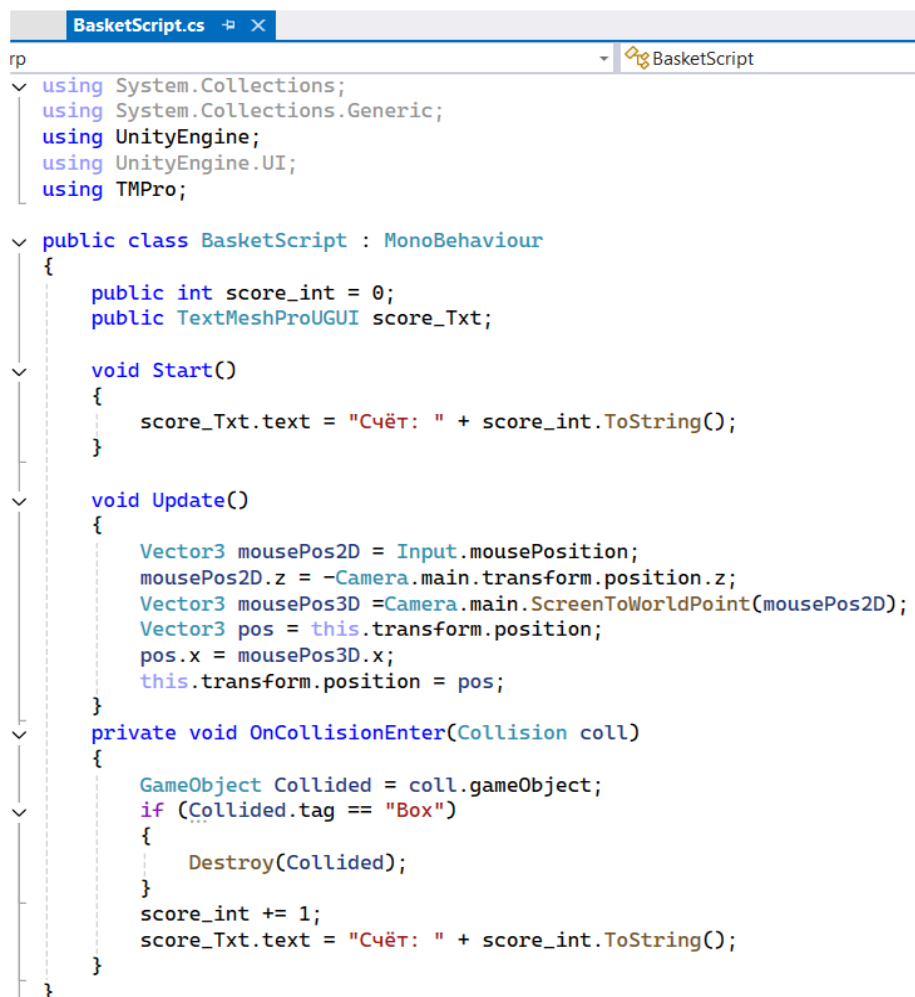
    void Start()
    {
        score_Txt.text = "Счёт: " + score_int.ToString();
    }
}
```

```

void Update()
{
    Vector3 mousePos2D = Input.mousePosition;
    mousePos2D.z = -Camera.main.transform.position.z;
    Vector3 mousePos3D = Camera.main.ScreenToWorldPoint(mousePos2D);
    Vector3 pos = this.transform.position;
    pos.x = mousePos3D.x;
    this.transform.position = pos;
}
private void OnCollisionEnter(Collision coll)
{
    GameObject Collided = coll.gameObject;
    if (Collided.tag == "Box")
    {
        Destroy(Collided);
    }
    score_int += 1;
    score_Txt.text = "Счет: " + score_int.ToString();
}
}

```

Скриншот листинга.



```

BasketScript.cs
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class BasketScript : MonoBehaviour
{
    public int score_int = 0;
    public TextMeshProUGUI score_Txt;

    void Start()
    {
        score_Txt.text = "Счет: " + score_int.ToString();
    }

    void Update()
    {
        Vector3 mousePos2D = Input.mousePosition;
        mousePos2D.z = -Camera.main.transform.position.z;
        Vector3 mousePos3D = Camera.main.ScreenToWorldPoint(mousePos2D);
        Vector3 pos = this.transform.position;
        pos.x = mousePos3D.x;
        this.transform.position = pos;
    }

    private void OnCollisionEnter(Collision coll)
    {
        GameObject Collided = coll.gameObject;
        if (Collided.tag == "Box")
        {
            Destroy(Collided);
        }
        score_int += 1;
        score_Txt.text = "Счет: " + score_int.ToString();
    }
}

```

Описание новых команд листинга:

- `Using UnityEngine.UI` подключается библиотека, доступная внутри движка Unity, позволяющая использовать интерфейс пользователя `.UI (User Interface)`;
- переменная `score_int` типа `int`, хранит счетчик пойманных объектов `Box`;
- `TextMeshProUGUI score_Txt` – текстовая часть игрового объекта `Score`;
- `score_Txt.text = "Счёт: " + score_int.ToString();` преобразует число счетчика пойманных объектов типа `int` в строку и записывает в переменную с именем `score`. Обратите внимание на то, что только благодаря строке в начале сценария `using UnityEngine.UI;` можно получать доступ к компоненту `Text` в коде на C#;
- `score += 1;` увеличивает значение на 1 при вызове функции `OnCollisionEnter(Collision coll)`, т.е. с каждым пойманным «ящиком».

6.6. УВЕДОМЛЕНИЕ О ТОМ, ЧТО «ЯЩИК» `Box` НЕ БЫЛ ПОЙМАН

На данном этапе, если «ящик» `Box` не был пойман, то он удаляется автоматически. Но чтобы можно было завершить раунд и удалить все объекты, упавшие мимо, ящик `Box` должен также уведомлять сценарий `DirigableHelp.cs` о случившемся событии (падении на плоскость `Plane`). Для этого нужно сделать так, чтобы один сценарий вызывал функцию в другом сценарии.

Внесите следующие изменения в `void Update()` сценария `BoxScript.cs`:

```
void Update()
{
    Vector3 posBox = transform.position;
    posBox.x += speed * Time.deltaTime;
    transform.position = posBox;
    if (posBox.x < -leftRightDistance)
    {
        speed = Mathf.Abs(speed);
    }
    else if (posBox.x > leftRightDistance)
    {
        speed = -Mathf.Abs(speed);
    }
}
```



```

        if (transform.position.y < bottomY)
        {
            Destroy(this.gameObject);
        }
        DirigableHelp apScript=Camera.main.GetComponent<DirigableHelp>();
        apScript.BoxDestroyed();
    }
}

```

Скриншот листинга с выделенным новым фрагментом кода показан ниже.

```

BasketScript.cs  BoxScript.cs
bl
BoxScript

public class BoxScript : MonoBehaviour
{
    [Header("Set in Inspector")]
    public GameObject BoxPrefab;
    public float speed = 1f;
    public float timeBetweenBoxDrops = 1f;
    public float leftRightDistance = 10f;
    public float chanceDirections = 0.1f;

    public static float bottomY = 0.0f;

    void Start()...
    private void OnTriggerEnter(Collider other)...
    void DropBox()...
    void Update()
    {
        Vector3 posBox = transform.position;
        posBox.x += speed * Time.deltaTime;
        transform.position = posBox;

        if (posBox.x < -leftRightDistance)
        {
            speed = Mathf.Abs(speed);
        }
        else if (posBox.x > leftRightDistance)
        {
            speed = -Mathf.Abs(speed);
        }
        if (transform.position.y < bottomY)
        {
            Destroy(this.gameObject);
            DirigableHelp apScript = Camera.main.GetComponent<DirigableHelp>();
            apScript.BoxDestroyed();
        }
    }
}

```

Описание новых команд листинга:

– `DirigableHelp apScript = Camera.main.GetComponent<DirigableHelp>();` в локальную переменную `apScript`

записывается ссылка на компонент **DirigableHelp (Script)** главной камеры **Main Camera**;

– после этого появляется возможность напрямую обращаться к переменным и методам экземпляра класса **DirigableHelp.cs**, подключенного к главной камере;

– **apScript.BoxDestroyed()**; это вызов метода **BoxDestroyed()** класса **DirigableHelp**. Но метода пока не существует, давайте создадим его.

Откройте сценарий **DragonPicker.cs**, добавьте в него новый метод **BoxDestroyed()**:

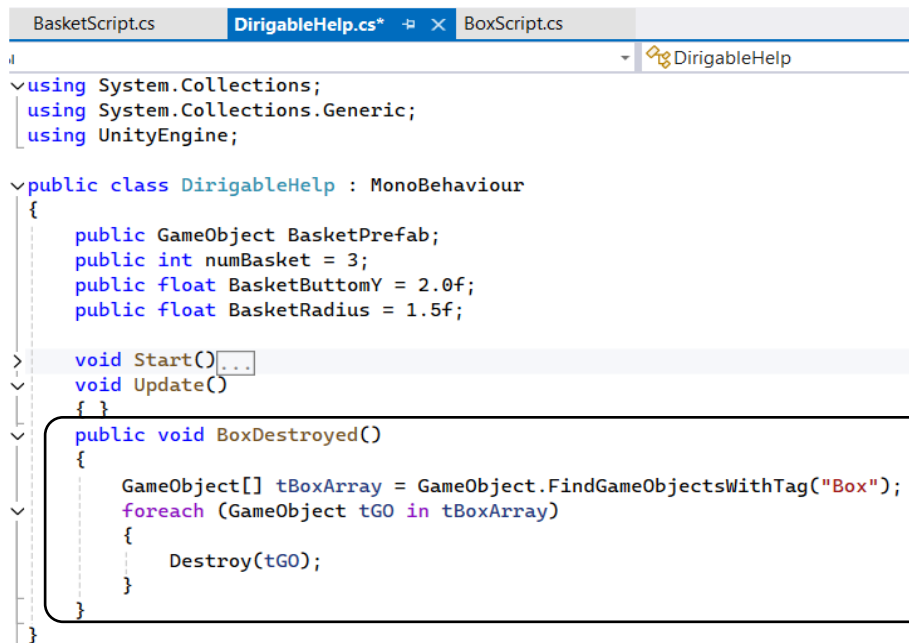
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DirigableHelp : MonoBehaviour
{
    public GameObject BasketPrefab;
    public int numBasket = 3;
    public float BasketBottomY = 2.0f;
    public float BasketRadius = 1.5f;

    void Start()
    { }
    void Update()
    { }

    public void BoxDestroyed()
    {
        GameObject[] tBoxArray=GameObject.FindGameObjectsWithTag("Box");
        foreach (GameObject tGO in tBoxArray)
        {
            Destroy(tGO);
        }
    }
}
```

Скриншот кода приведен ниже.



Описание новых команд:

- метод **BoxDestroyed** имеет тип **public** (т.е. является общедоступным), чтобы его можно было вызывать из других классов, например, таких как **BoxScript.cs**;
- строки кода внутри созданного метода возвращают массив всех существующих игровых объектов **Box** и с помощью цикла **for each** выполняется обход всех найденных объектов с их последующим уничтожением.

6.7. УНИЧТОЖЕНИЕ BASKET ПОСЛЕ ПОТЕРИ «ЯЩИКА» BOX

После потери **Box** должна удаляться одна «корзина» **Basket**. После потери трех «корзин» игра должна останавливаться. Для этого сценарий **DragonPicker.cs** должен содержать следующие строки кода (приведем листинг целиком, с указанием новых строк кода):

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
```

```

public class DirigableHelp : MonoBehaviour
{
    public GameObject BasketPrefab;
    public int numBasket = 3;
    public float BasketBottomY = 2.0f;
    public float BasketRadius = 1.5f;
    public List<GameObject> basketList;

    void Start()
    {
        basketList = new List<GameObject>();
        for (int i = 1; i <= numBasket; i++)
        {
            GameObject tBasketGo =
                Instantiate<GameObject>(BasketPrefab);
            tBasketGo.transform.position =
                new Vector3(0, BasketBottomY, 0);
            tBasketGo.transform.localScale = new Vector3(1*i,1*i,1*i);
            basketList.Add(tBasketGo);
        }
    }

    void Update()
    {
    }

    public void BoxDestroyed()
    {
        GameObject[] tBoxArray =
            GameObject.FindGameObjectsWithTag("Box");
        foreach (GameObject tGO in tBoxArray)
        {
            Destroy(tGO);
        }

        int basketIndex = basketList.Count - 1;
        GameObject tBasketGo = basketList[basketIndex];
        basketList.RemoveAt(basketIndex);
        Destroy(tBasketGo);
    }
}

```

Скриншот кода.

```
BasketScript.cs  DirigableHelp.cs*  BoxScript.cs
DirigableHelp

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class DirigableHelp : MonoBehaviour
{
    public GameObject BasketPrefab;
    public int numBasket = 3;
    public float BasketBottomY = 2.0f;
    public float BasketRadius = 1.5f;
    public List<GameObject> basketList;

    void Start()
    {
        basketList = new List<GameObject>();
        for (int i = 1; i <= numBasket; i++)
        {
            GameObject tBasketGo = Instantiate<GameObject>(BasketPrefab);
            tBasketGo.transform.position = new Vector3(0, BasketBottomY, 0);
            tBasketGo.transform.localScale = new Vector3(1 * i, 1 * i, 1 * i);
            basketList.Add(tBasketGo);
        }
    }

    void Update()
    { }

    public void BoxDestroyed()
    {
        GameObject[] tBoxArray = GameObject.FindGameObjectsWithTag("Box");
        foreach (GameObject tGO in tBoxArray)
        {
            Destroy(tGO);
        }
        int basketIndex = basketList.Count - 1;
        GameObject tBasketGo = basketList[basketIndex];
        basketList.RemoveAt(basketIndex);
        Destroy(tBasketGo);
    }
}
```

Описание новых команд листинга в приведённом примере:

- подключаем библиотеку **SceneManager** для возможности работы со сценами, она потребуется в дальнейшем;
- определяется новый список типа **List<GameObject>**;
- в конце цикла **for** добавляется каждый созданный **BasketPrefab** в список **basketList**;
- уничтожается одна «корзина» с каждым очередным срабатыванием метода **BoxDestroyed()**.

Теперь, если запустить игру и потерять все три корзины **Basket**, то программа завершится с исключением

IndexOutOfRangeException, так как на данный момент в игре не прописано, что требуется сделать программе в случае потери всех «корзин».

Чтобы исключение не возникало, в метод **BoxDestroyed()** сценария **DirigableHelp.cs** следует добавить строки, возвращающие игру в исходное состояние, когда игрок теряет все три «корзины» (что в нашей концепции игры символизирует жизни):

```
public void BoxDestroyed()
{
    GameObject[] tBoxArray =
        GameObject.FindGameObjectsWithTag("Box");
    foreach (GameObject tGO in tBoxArray)
    {
        Destroy(tGO);
    }
    int basketIndex = basketList.Count - 1;
    GameObject tBasketGo = basketList[basketIndex];
    basketList.RemoveAt(basketIndex);
    Destroy(tBasketGo);
    if (basketList.Count == 0)
    {
        SceneManager.LoadScene("MainScene");
    }
}
```

Так как листинг **DirigableHelp.cs** уже достаточно большой, приведем только скриншот метода **BoxDestroyed**, в который были добавлены новые строки кода.

```
public void BoxDestroyed()
{
    GameObject[] tBoxArray = GameObject.FindGameObjectsWithTag("Box");
    foreach (GameObject tGO in tBoxArray)
    {
        Destroy(tGO);
    }
    int basketIndex = basketList.Count - 1;
    GameObject tBasketGo = basketList[basketIndex];
    basketList.RemoveAt(basketIndex);
    Destroy(tBasketGo);

    if (basketList.Count == 0)
    {
        SceneManager.LoadScene("MainScene");
    }
}
```

Проверьте, что после потери всех трех «корзин» происходит перезапуск сцены, а счетчик заработанных очков сбрасывается (снова становится равным нулю).

6.7.1. Комментарии

В этой части мы добавили в игру смысл с помощью простого добавления счетчика очков. В качестве дополнительного задания попробуйте сохранять рекорд игрока. Для этого стоит изучить работу методов `PlayerPrefs.Set()` и `PlayerPrefs.Get()`. С помощью данных методов можно запоминать переменные для отдельных пользователей во время игры. При повторном входе в игру эти переменные сохраняют свои значения.

7. ДОРАБОТКА СЦЕН И СОЗДАНИЕ ИГРОВОГО МЕНЮ

7.1. ВОЗМОЖНАЯ ДОРАБОТКА ЛОКАЦИИ

Чтобы локация выглядела более привлекательно, можно добавить в качестве декораций горы или море, облака, простую поверхность заменить на рельефную и т.п. Существует немало доступных источников для скачивания различных персонажей, анимаций и текстур. Не стоит ограничивать фантазию...

Однако правила хорошего тона при разработке требуют четкой иерархии в объектах, сценах, скриптах и т.д. Поэтому следует создавать отдельные папки для хранения объекта, его материала, физических свойств, анимаций. Это позволит иметь логичную структуру объектов и их взаимодействий.

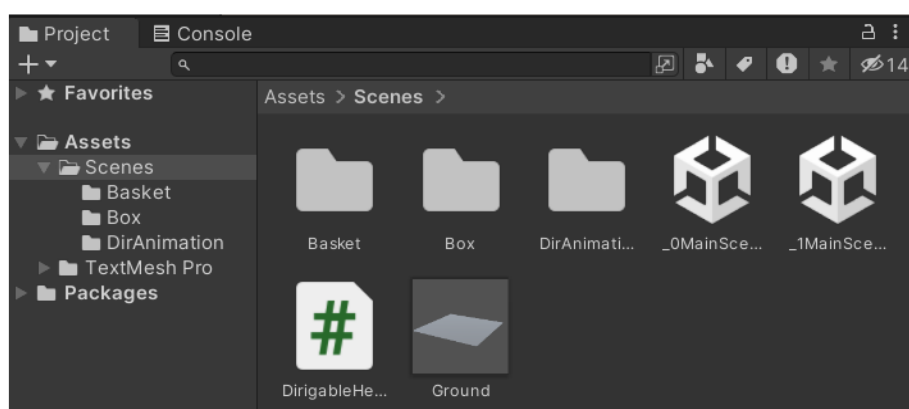
Добавим новую стартовую сцену, на которую будет попадать пользователь сразу после запуска игры.

Переименуем текущую MainScene в _1MainScene. Так мы обозначим, что игровая сцена смещается по иерархии загрузки на уровень ниже.

Добавить новую сцену можно, создав копию текущей сцены, для этого выделите объект _1MainScene и нажмите сочетание клавиш Ctrl+D (или Command+D). Переименуйте созданную копию объекта в _0MainScene, это будет имя новой созданной сцены. После этого в папке должны находиться две сцены:

_0MainScene – копия сцены, в которой мы создадим стартовое меню;

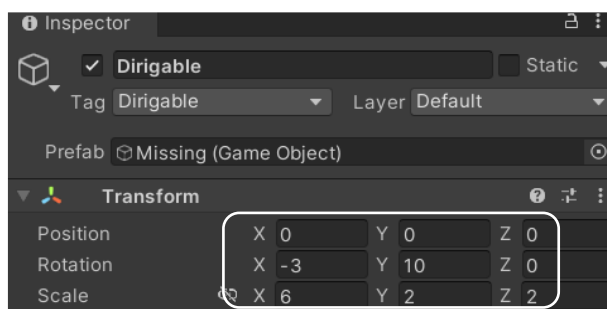
_1MainScene – основная сцена, с которой мы ранее работали.



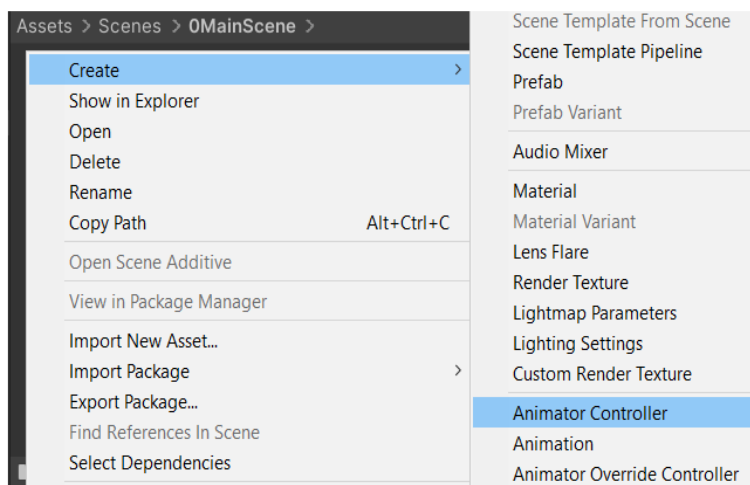
Далее мы настроим стартовую сцену _0Scene таким образом, чтобы на ней было удобно разместить элементы меню навигации.

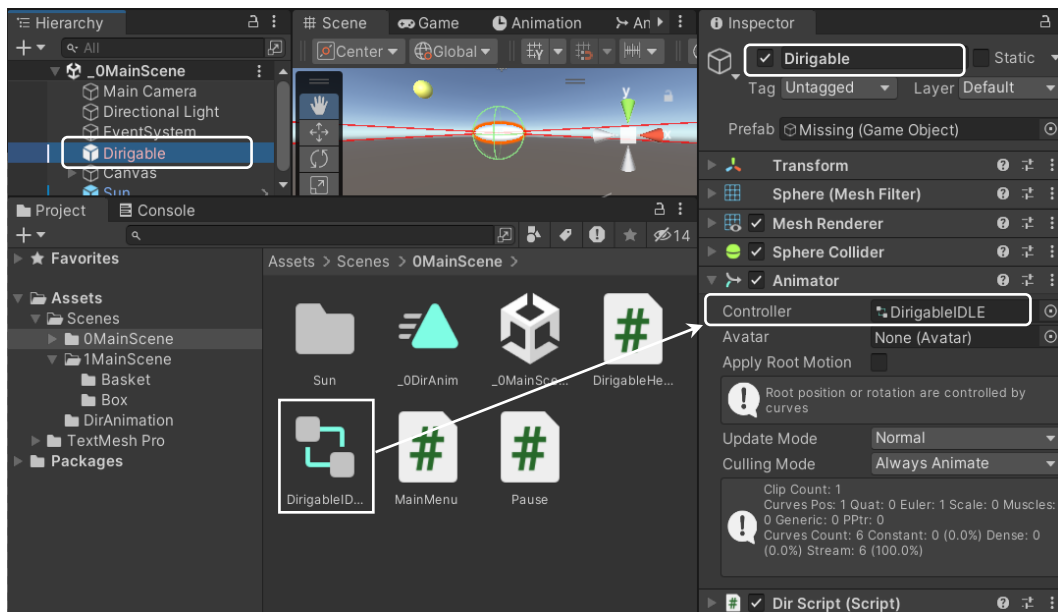
Для начала внесем некоторые изменения в имеющуюся структуру сцены:

- удалим объект Ground, для этого в окне Hierarchy кликните по нему правой кнопкой мыши и в контекстном меню нажмите Delete;
- удалим объект Score, который находится внутри холста Canvas;
- выберем объект Main Camera и в окне Inspector удалим компонент с подключенным скрипт-файлом DirigableHelp.cs (кликните по трем точкам справа от названия компонента и выберите Remove Component), в этой сцене он нам не потребуется;
- далее выберем объект Dirigable и также изменим в инспекторе настройки Transform, как показано на рисунке ниже:



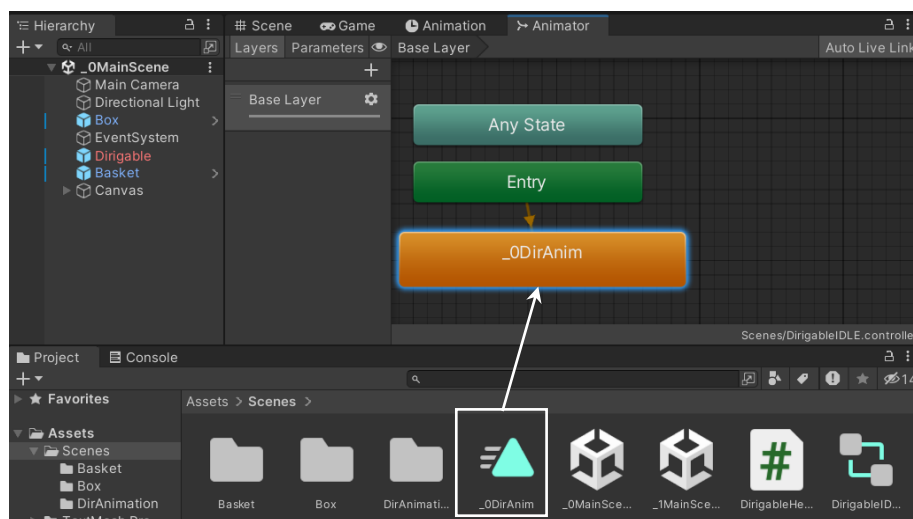
Изменим анимацию дирижабля на стартовой сцене. Для этого создадим отдельный контроллер. Кликнем правой кнопкой мыши внутри папки Scenes и выберем из контекстного меню Create → Animator Controller, контроллеру дадим имя DirigableIDLE.



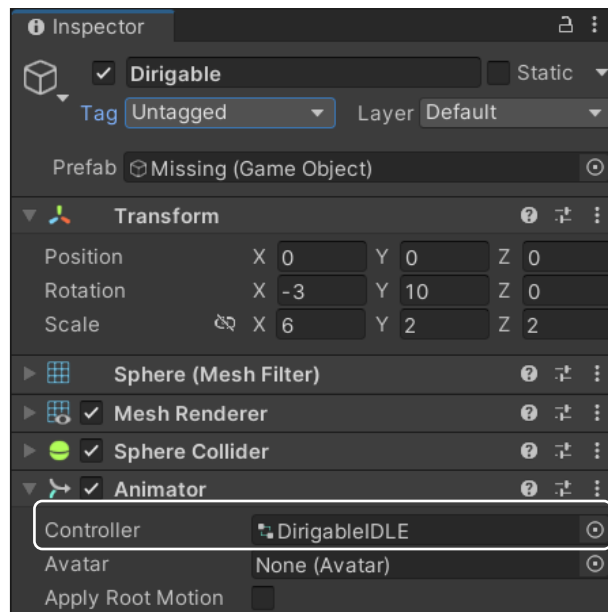


В папке Assets – Animations найдем подходящую анимацию.

В папке Scene откройте контроллер DirigableIDLE и перетяните в него только что добавленную в папку анимацию _0DirAnim. Это значит, что мы создали контроллер для состояния простоя дирижабля, внутри контроллера находится анимация дирижабля на поверхности земли:



Теперь выбираем дирижабль Dirigable в окне Hierarchy, и после этого в окне инспектора в правой части в компоненте Animator выбираем другой контроллер DirigableIDLE.



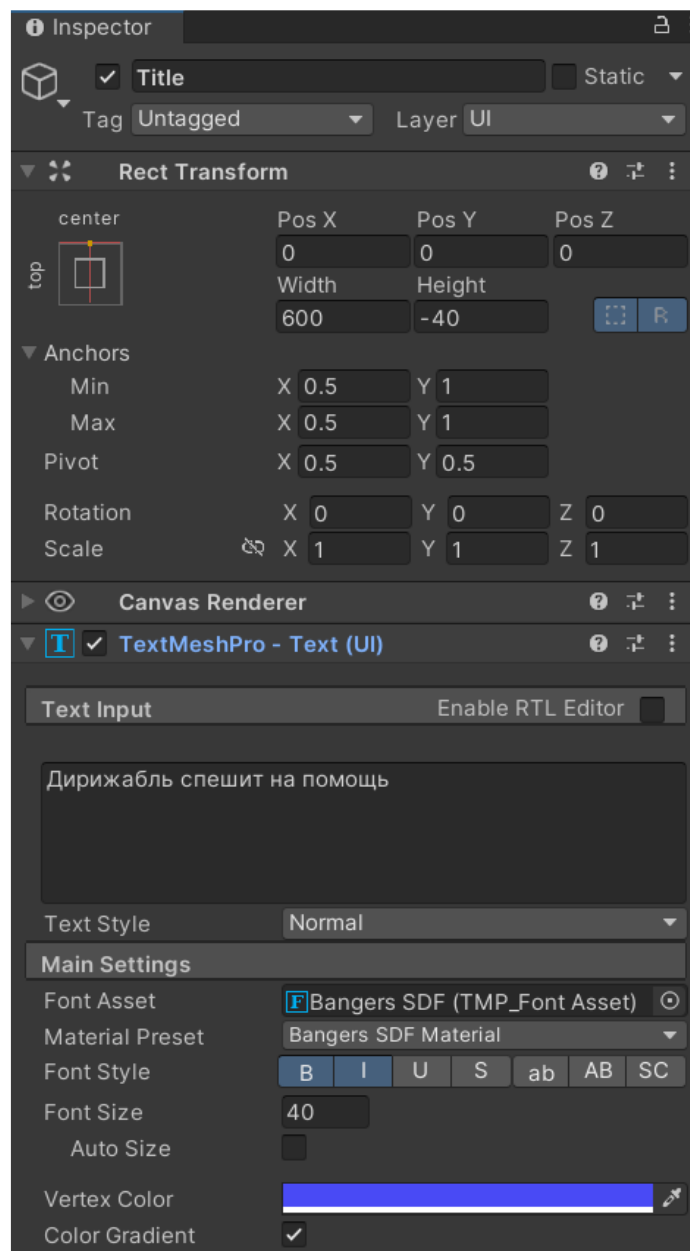
Сохраним сцену _0MainScene и запустим её (Play). Теперь при запуске сцены можно увидеть анимированный дирижабль.

Переключаться между сценами и дорабатывать их можно в любой момент, просто кликнув по ней два раза.

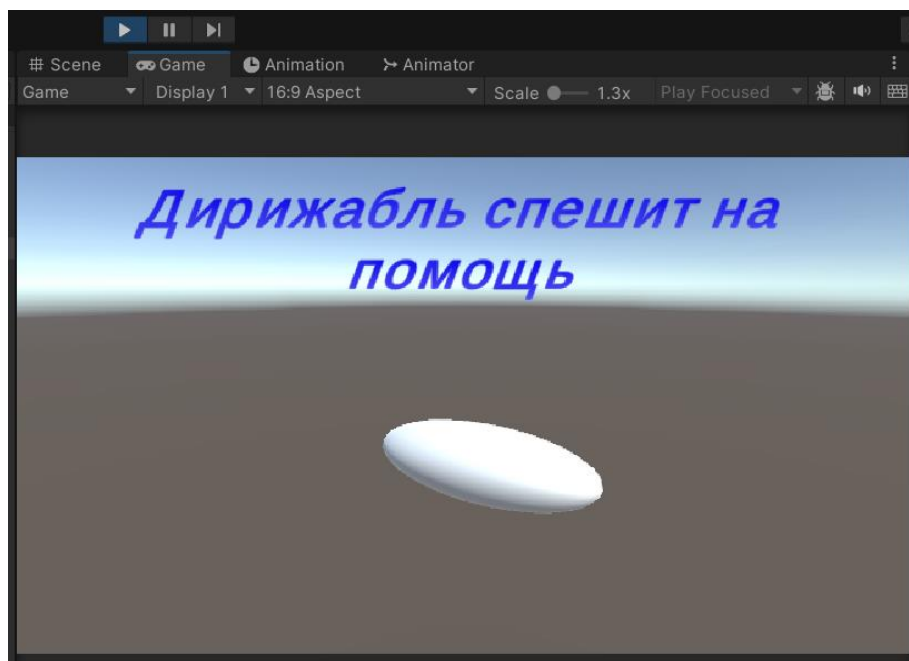
7.2. СОЗДАНИЕ ИГРОВОГО МЕНЮ

Создадим заголовок на сцене с названием игры. Как и раньше, выбираем в верхнем меню GameObject → UI → Text-TextMeshPro. Дадим имя новому объекту Title и так же, как делали ранее, переместим его внутрь Canvas (если этого не произошло автоматически).

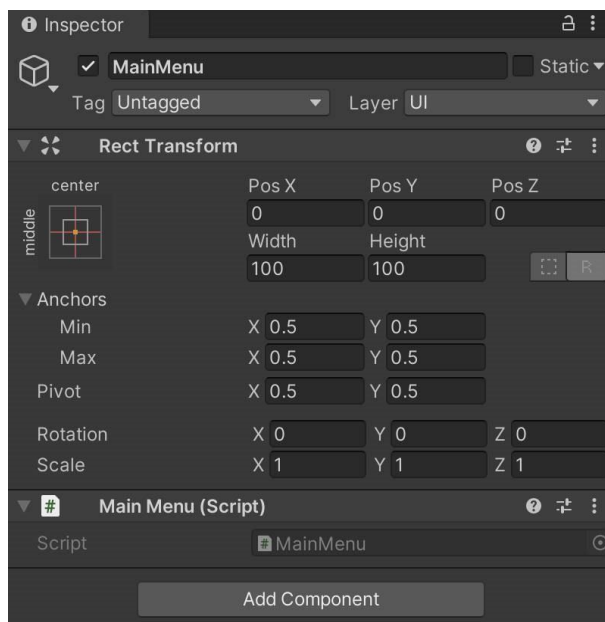
Настройте Title в окне Inspector так, как показано на рисунке.



В итоге получим надпись: «Дирижабль спешит на помощь» вверху по центру экрана. Можно изменить цвет, размер и стиль надписи, а также самостоятельно найти шрифты с текстурами и анимацией.



Для создания меню создайте пустой объект, выбрав в верхнем меню **GameObject** → **Create Empty** (Ctrl+Shift+N) , который будет содержать кнопки переключения между сценами. В панели Hierarchy слева появится пустой объект. Назовите его **MainMenu**, также создайте и подключите к объекту соответствующий скрипт **MainMenu.cs**:



Сделайте элемент **MainMenu**, дочерним для **Canvas**.

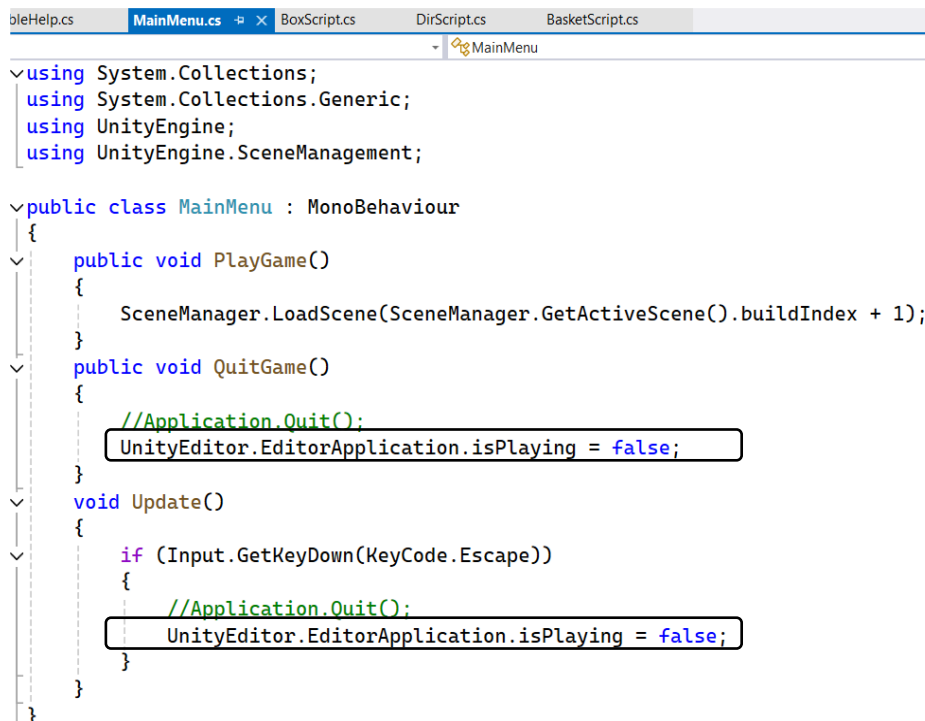
Добавьте в скрипт **MainMenu.cs** следующий код, который будет отвечать за переключение между сценами:

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
using static System.Net.Mime.MediaTypeNames;

public class MainMenu : MonoBehaviour
{
    public void PlayGame()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex+1);
    }
    public void QuitGame()
    {
        //Application.Quit();
        UnityEditor.EditorApplication.isPlaying = false;
    }
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            //Application.Quit();
            UnityEditor.EditorApplication.isPlaying = false;
        }
    }
}

```



```

bleHelp.cs  MainMenu.cs  BoxScript.cs  DirScript.cs  BasketScript.cs
MainMenu

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class MainMenu : MonoBehaviour
{
    public void PlayGame()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
    }
    public void QuitGame()
    {
        //Application.Quit();
        UnityEditor.EditorApplication.isPlaying = false;
    }
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            //Application.Quit();
            UnityEditor.EditorApplication.isPlaying = false;
        }
    }
}

```

Для завершения игры при нажатии кнопки QuitButton добавили возможность завершения игры при нажатии Esc.

Однако Application.Quit() будет работать только из построенного приложения.

В редакторе Unity функция выхода из приложения игнорируется, поэтому нажатие этой кнопки в режиме воспроизведения ничего не даст.

Для того чтобы выйти из игрового режима с помощью скрипта, нужно использовать функцию:

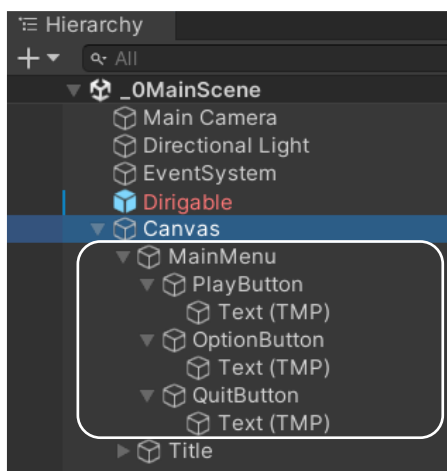
```
UnityEditor.EditorApplication.isPlaying = false;
```

Это имеет тот же эффект, что и выход из игры с помощью `Application.Quit()`, но этот выход работает и в редакторе.

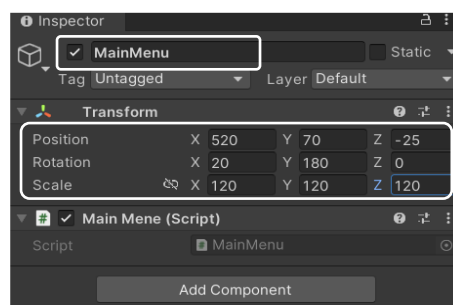
Добавьте в панель Hierarchy элементы UI графического интерфейса: `GameObject → UI → Button` (3 штуки).

Сделайте созданный объект Button дочерним для MainMenu. Обратите внимание, что внутри кнопки Button автоматически создается дочерний элемент Text. Переименуйте созданные кнопки Button в PlayButton, OptionButton и QuitButton.

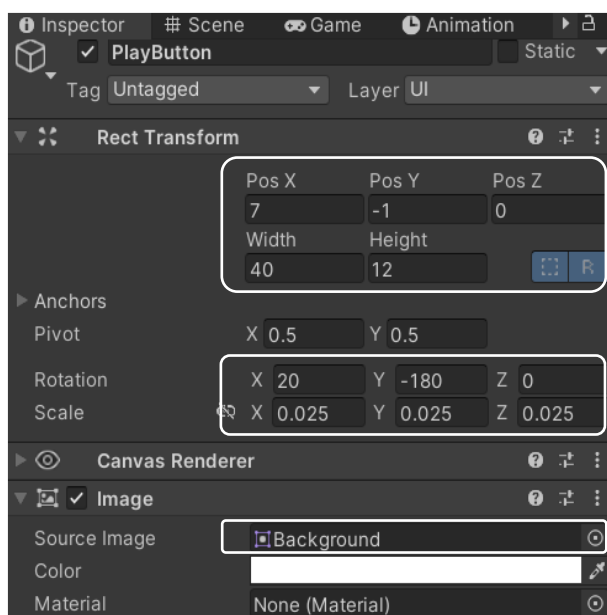
Должна получиться структура кнопок и надписей такая, как показано на рисунке.



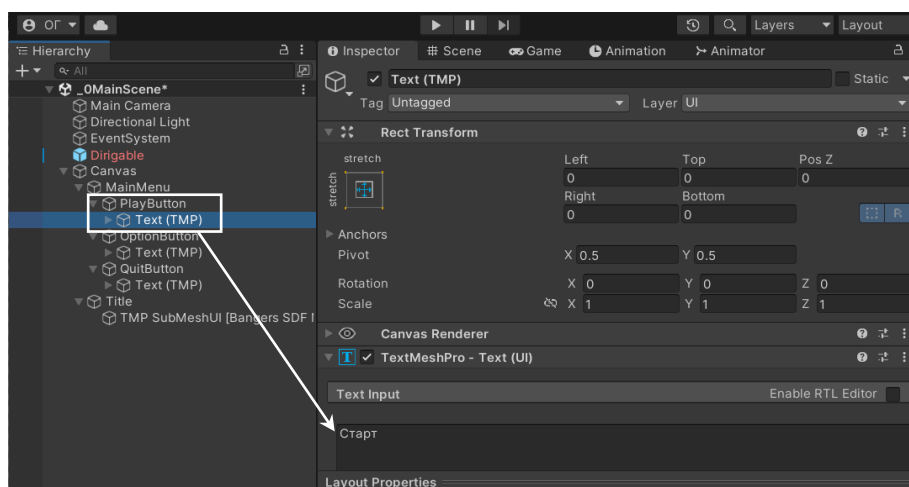
Настроим оформление надписей кнопок и их расположение так, чтобы интерфейс выглядел как стандартное меню с элементами управления. Кнопки имеют стандартные настройки, которые задают их расположение, внешний вид при различных вариантах срабатывания и т.д. В окне Hierarchy выберите элемент MainMenu и установите его параметры Transform.



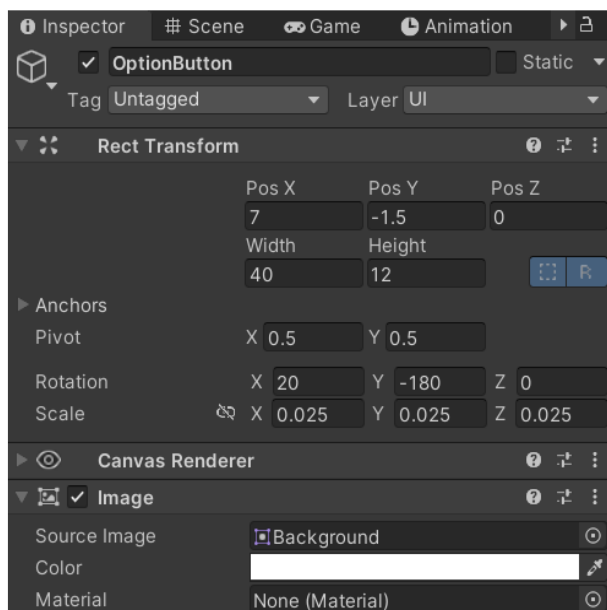
Теперь настроим кнопки. Начнем с PlayButton. Важные настройки находятся во вкладке Button в окне Inspector, эти настройки можно использовать при создании различных эффектов анимации кнопки без дополнительных текстур. Некоторые настройки отвечают за внешний вид при нажатии – наведении – выборе и т.д. Настройте кнопку Button так, как показано на рисунке.



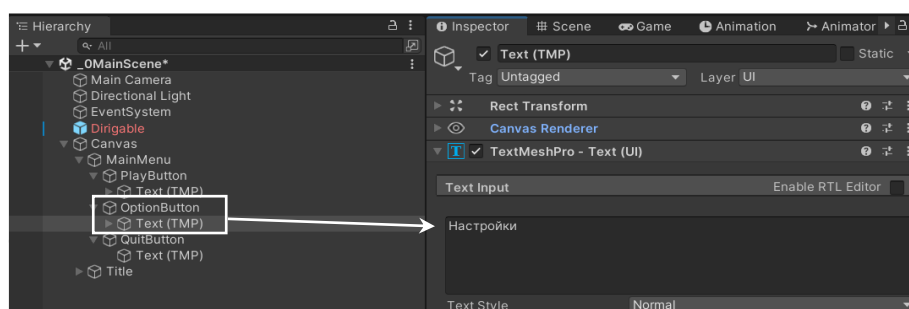
Чтобы изменить надпись на кнопке, разверните элемент PlayButton, кликните по элементу Text внутри. Далее в окне Inspector (для удобства его положение на скриншоте ниже было изменено) найдите компонент Text и напишите в поле ввода текста: «Старт».



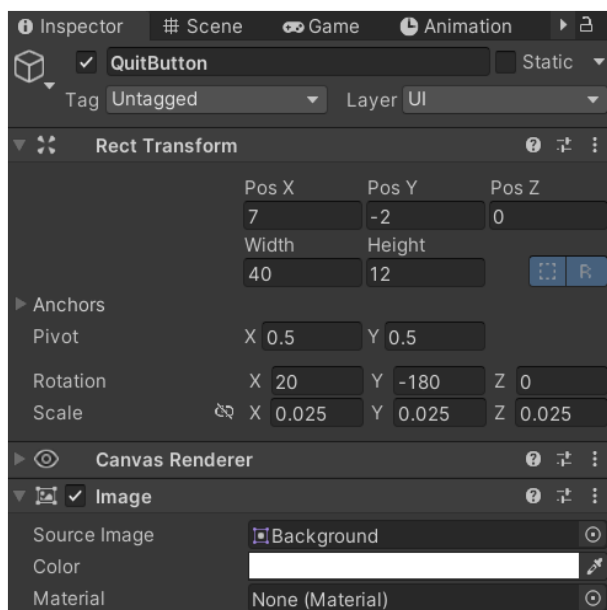
Аналогично ниже приведены настройки для кнопки
OptionButton:



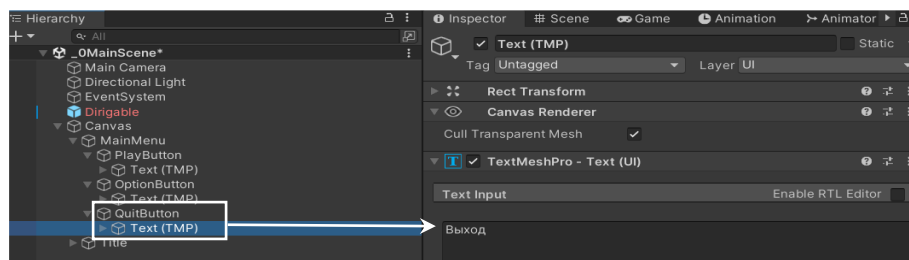
Текстовое поле Text внутри OptionButton:



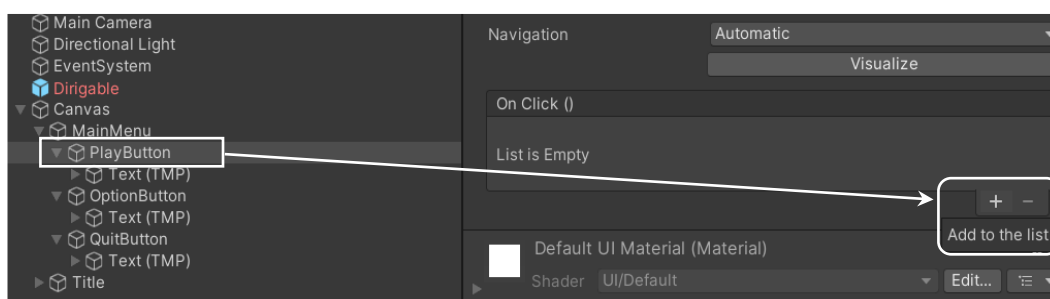
Аналогично ниже приведены настройки для кнопки
QuitButton:



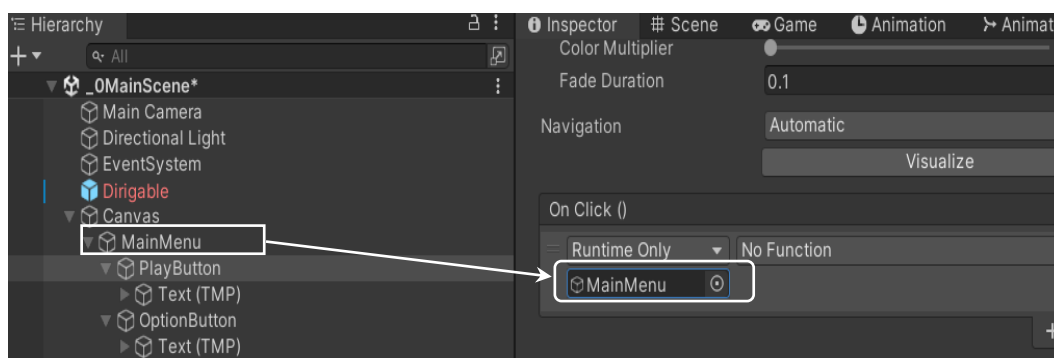
Текстовое поле Text внутри QuitButton:



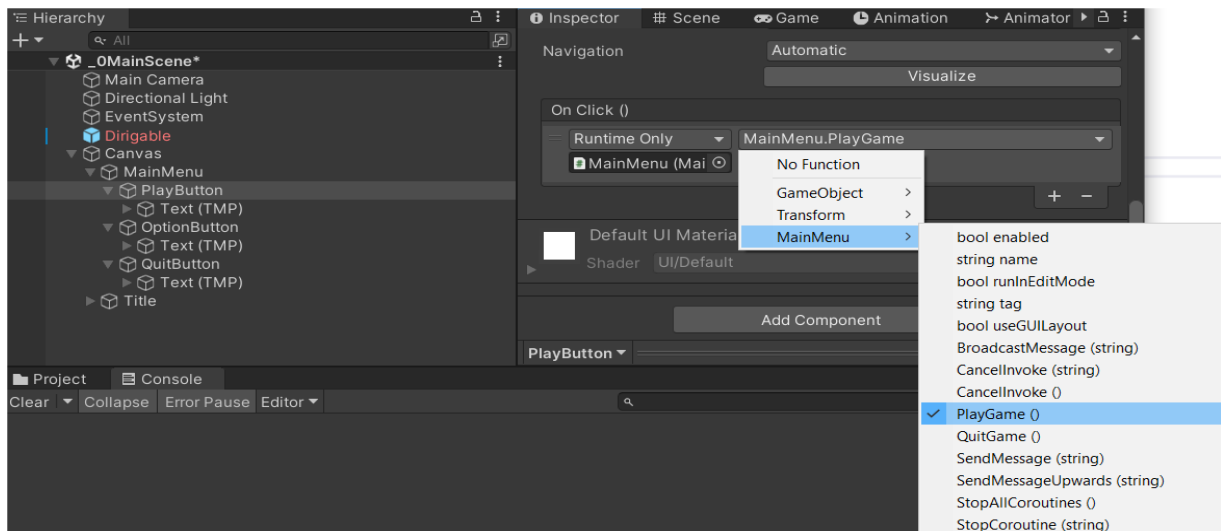
За функционал кнопки отвечает нижняя часть компонента Button внутри окна Inspector – On Click(). Выберите кнопку PlayButton. Чтобы настроить действие, которое будет происходить при срабатывании кнопки, нажмите знак «+» в поле On Click ():



В появившееся пустое поле для игрового объекта добавьте объект из Hierarchy с именем MainMenu. Таким образом, мы сообщаем программе, что при нажатии должна вызываться некоторая часть скрипта, привязанная к прикрепленному игровому объекту:

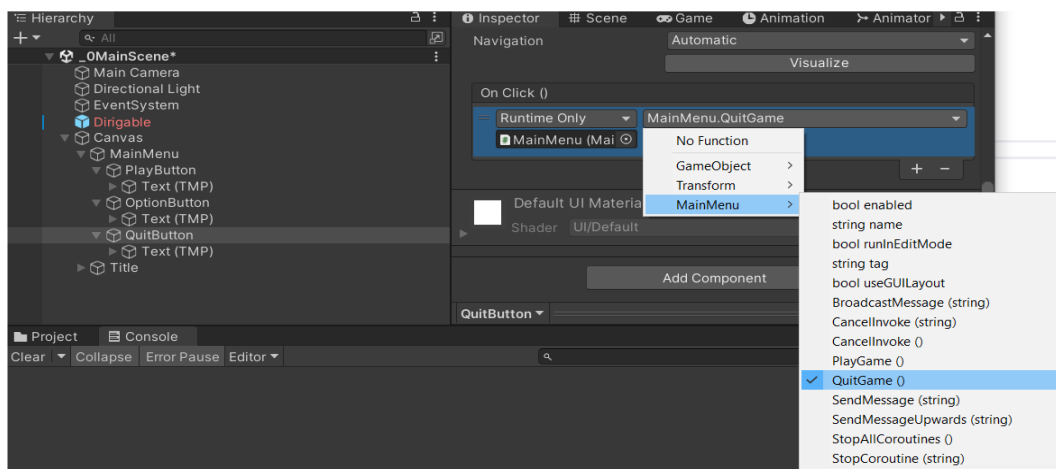


Если все сделано правильно, то теперь из вкладки No Functions стало возможным выбрать скрипт MainMenu и метод PlayGame, отвечающий за запуск игры:

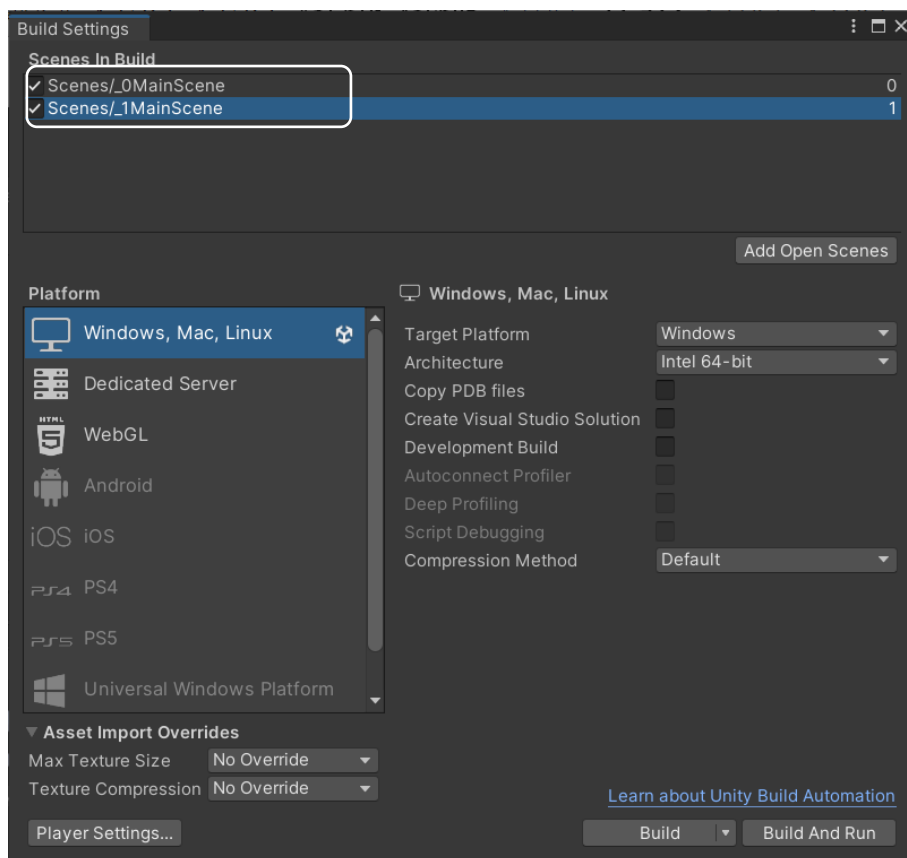


Теперь, при нажатии кнопки PlayGame будет происходить запуск игры.

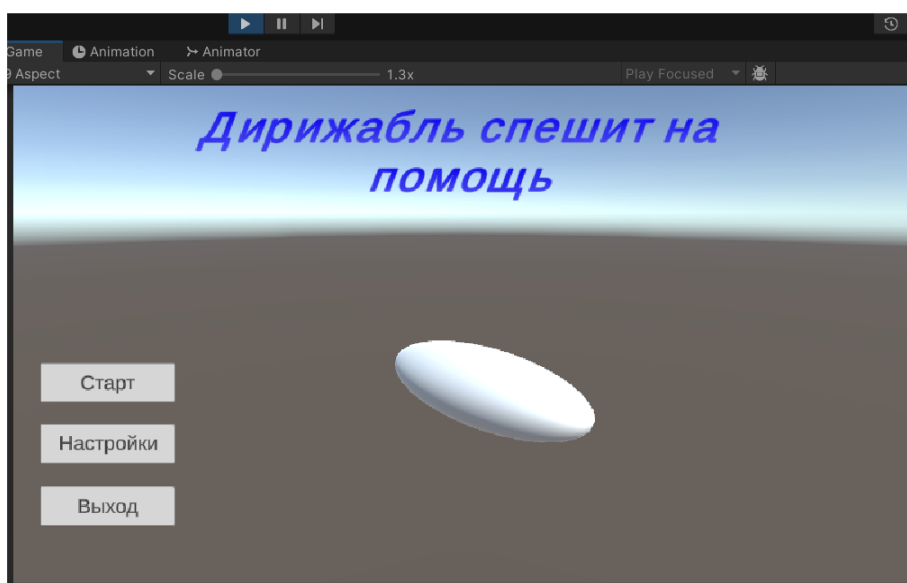
Настройте кнопку QuitButton. Также нужно добавить событие (знак «+»), переместить в пустое поле объект MainMenu, и в поле No Function выбрать метод QuitGame() из скрипта Main Menu:



Теперь для того чтобы переключение сцен работало корректно, нужно настроить Build игры и добавить созданные сцены в сборку. Нажмите File → Build Settings, перетащите созданные сцены _0MainScene и _1MainScene в поле Scenes In Build в порядке их загрузки (первой загружается сцена с меню):

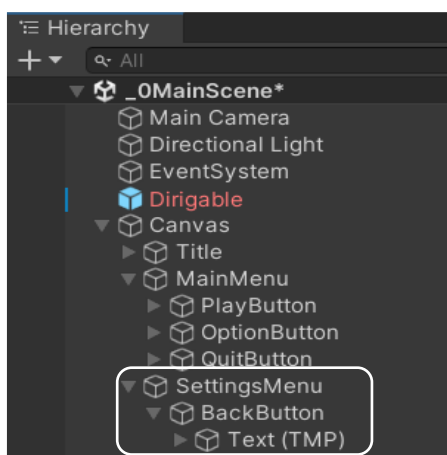


После этого можно закрыть окно Build Settings и проверить корректность работы меню, нажав кнопку Play. После нажатия кнопки Play игра должна запускаться, а при нажатии кнопки Quit – завершать работу (однако в рамках симуляции в Unity она может не отрабатывать).

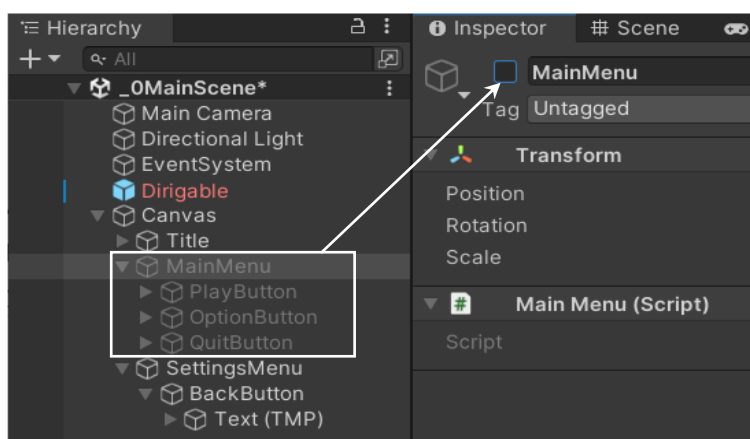


Вернемся к кнопке Option. Перед тем как запрограммировать кнопку Option, нужно создать дополнительные элементы меню. При нажатии на кнопку Option у нас будут появляться дополнительные элементы меню и скрываться элементы главного меню MainMenu.

Самостоятельно создайте дополнительное меню с названием SettingsMenu (можно создать дубликат MainMenu) и удалите из SettingsMenu две верхние кнопки PlayButton и OptionButton. Единственную оставшуюся кнопку переименуйте в BackButton (вернуться назад).

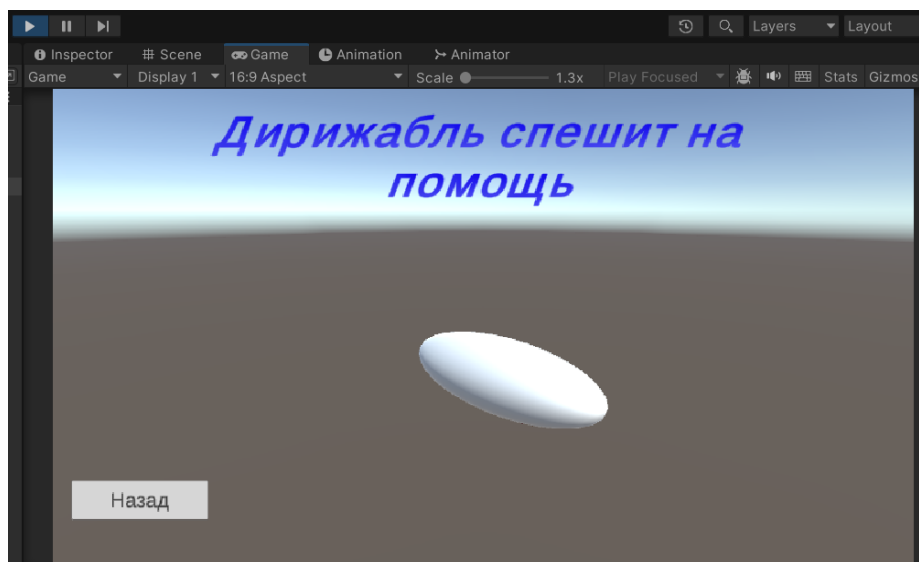


Чтобы объект главного меню MainMenu не мешал при создании нового SettingsMenu в том же окне, можно его временно отключить, сняв галочку напротив названия объекта в окне Inspector:

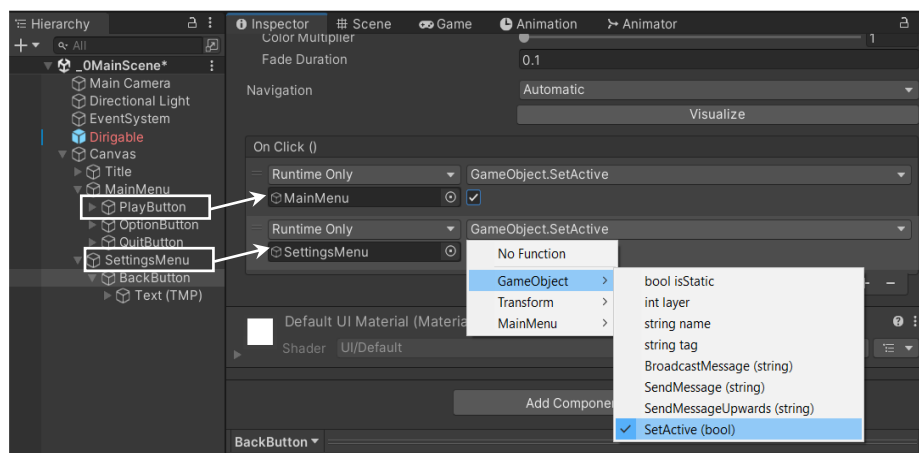


В поле объекта Text внутри кнопки BackButton напишите «Назад» – надпись, которая будет отображаться в меню.

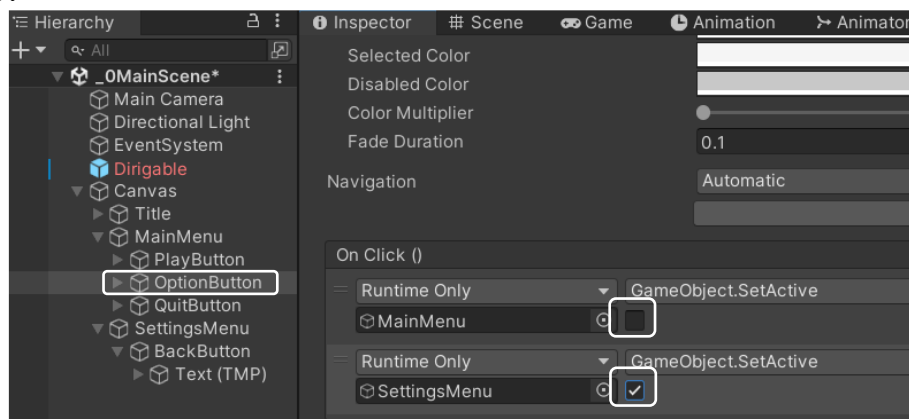
Созданное SettingsMenu должно выглядеть следующим образом:



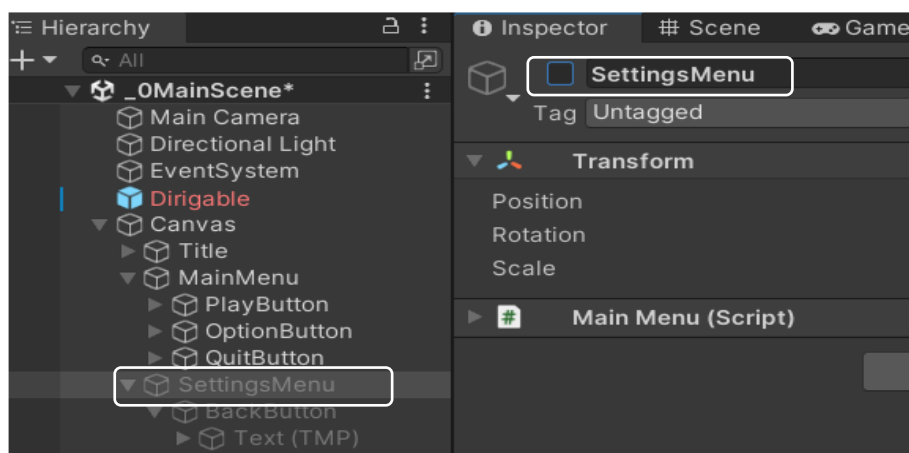
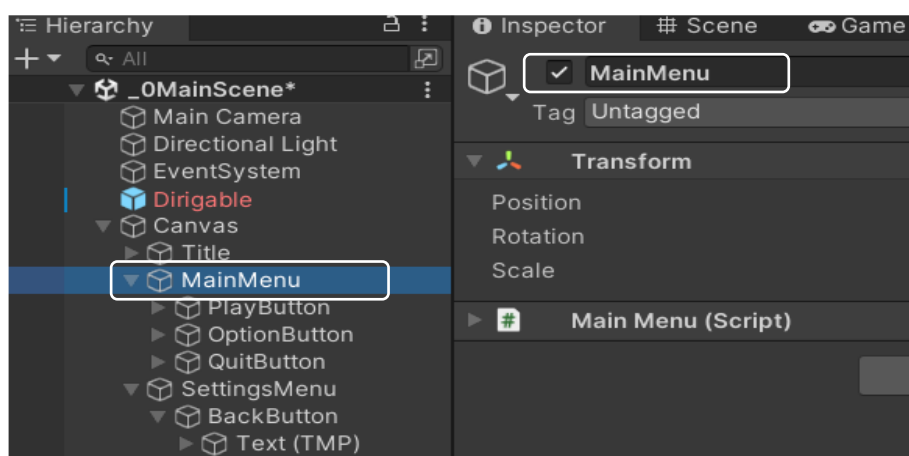
Для программирования кнопок BackButton и OptionButton не обязательно писать дополнительный код. Рассмотрим реализацию переключения элементов меню, просто настроив активацию игровых объектов при нажатии кнопок. Выберите кнопку BackButton и в окне Inspector настройте ее параметры On Click():, как показано на рисунке:



Для `OptionButton` настройка параметра `On Click()` показана на рисунке:



Чтобы интерфейс работал корректно, при старте игры меню `MainMenu` должно быть активировано, а `SettingMenu` – деактивировано. В окне `Inspector` напротив имени `MainMenu` галочка должна быть установлена, а у имени `SettingMenu` – снята.



7.3. ПАУЗА И ВОЗВРАЩЕНИЕ В ГЛАВНОЕ МЕНЮ

Кнопка паузы будет работать следующим образом:

- при нажатии клавиши пробел (Space) игра будет останавливаться;
- на экране будет появляться надпись «Пауза»;
- при повторном нажатии клавиши Space игра продолжится;
- на экране пропадет надпись «Пауза»;
- при нажатии клавиши Backspace (стрелка назад), происходит загрузка стартовой сцены.

Для того чтобы реализовать описанный функционал, нам потребуется скрипт-файл, в котором будут описаны работа сцены и условия появления надписи «Пауза». А также текстовое поле с надписью «Пауза». Перейдем к реализации.

Откройте сцену _1MainScene.

Создайте скрипт-файл с именем **Pause.cs**, откройте его в редакторе кода и напишите код, указанный ниже:

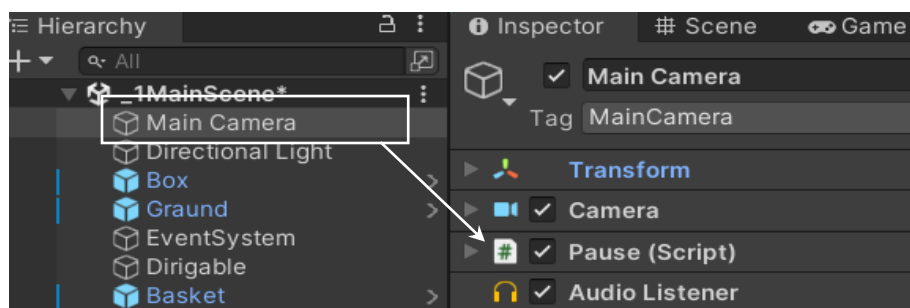
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
public class Pause : MonoBehaviour
{
    private bool paused = false;
    public GameObject panel;
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            if (!paused) {
                Time.timeScale = 0;
                paused = true;
                panel.SetActive(true);
            }
            Else {
                Time.timeScale = 1;
                paused = false;
                panel.SetActive(false);
            }
        }
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex-1);
        }
    }
}
```



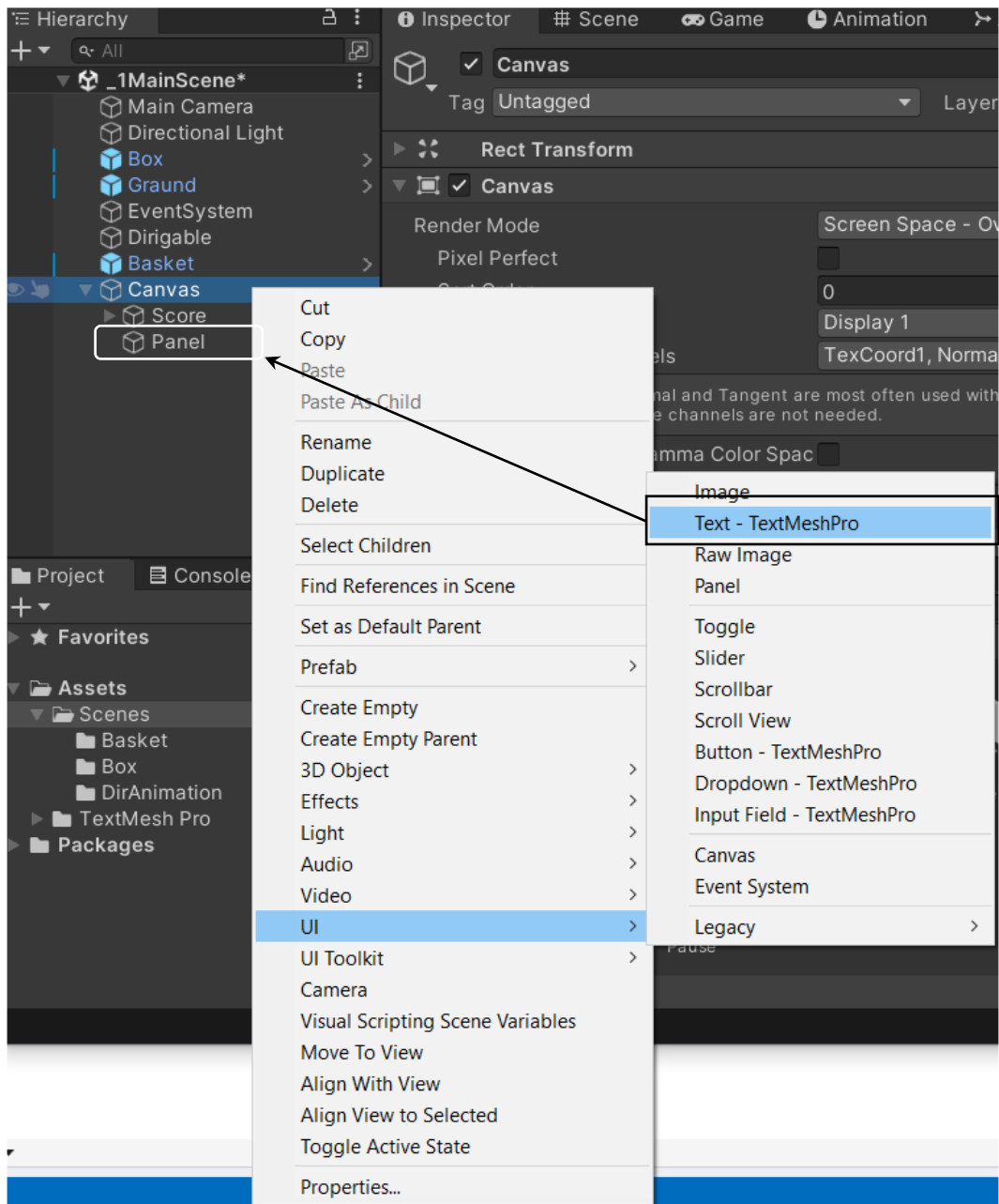
```
Pause.cs
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class Pause : MonoBehaviour
{
    private bool paused = false;
    public GameObject panel;
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            if (!paused)
            {
                Time.timeScale = 0;
                paused = true;
                panel.SetActive(true);
            }
            else
            {
                Time.timeScale = 1;
                paused = false;
                panel.SetActive(false);
            }
        }
        if (Input.GetKeyDown(KeyCode.Escape))
        {
            SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex - 1);
        }
    }
}
```

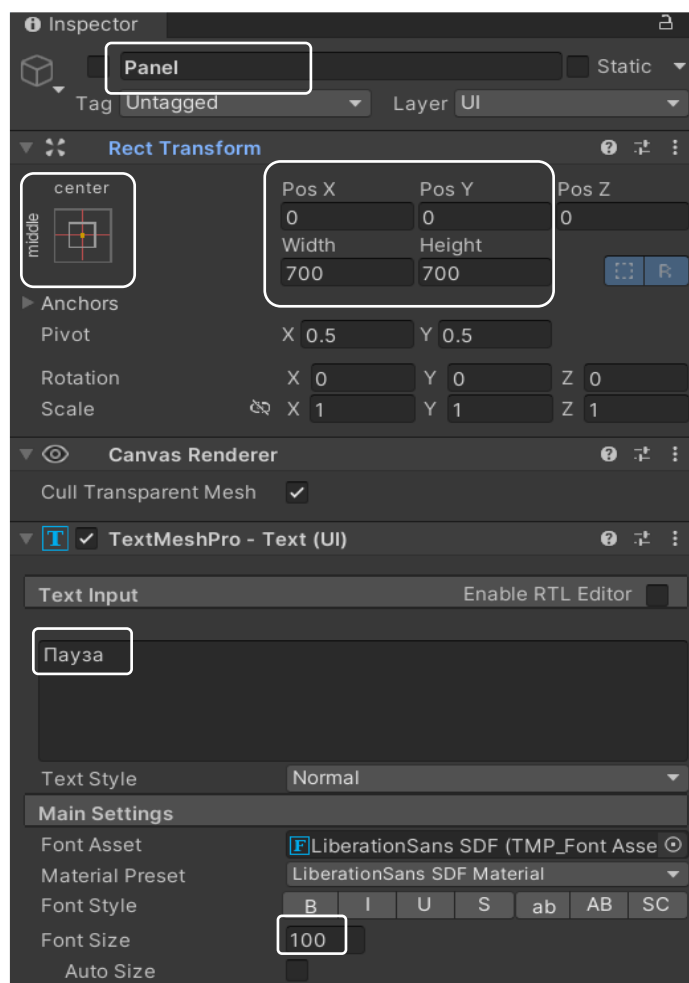
Сохраните скрипт-файл и подключите его к объекту на сцене Main Camera (перетаскиванием, как делали ранее, либо выбрав камеру и через кнопку Add Component в окне Inspector):



Далее добавим объект текстового типа (Text) и переименуем его в Panel:

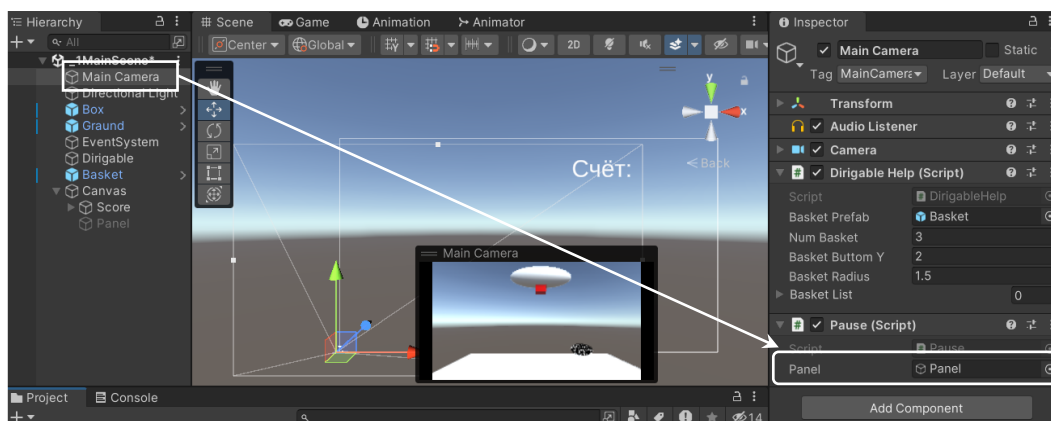


Ниже показаны настройки расположения панели Panel.



Уберите галочку напротив имени Panel в окне Inspector, чтобы при старте сцены надпись не была активна и не отображалась на экране.

Выберите объект Main Camera и переместите в окно переменной Panel в окне Inspector текстовый объект Panel из окна Hierarchy:



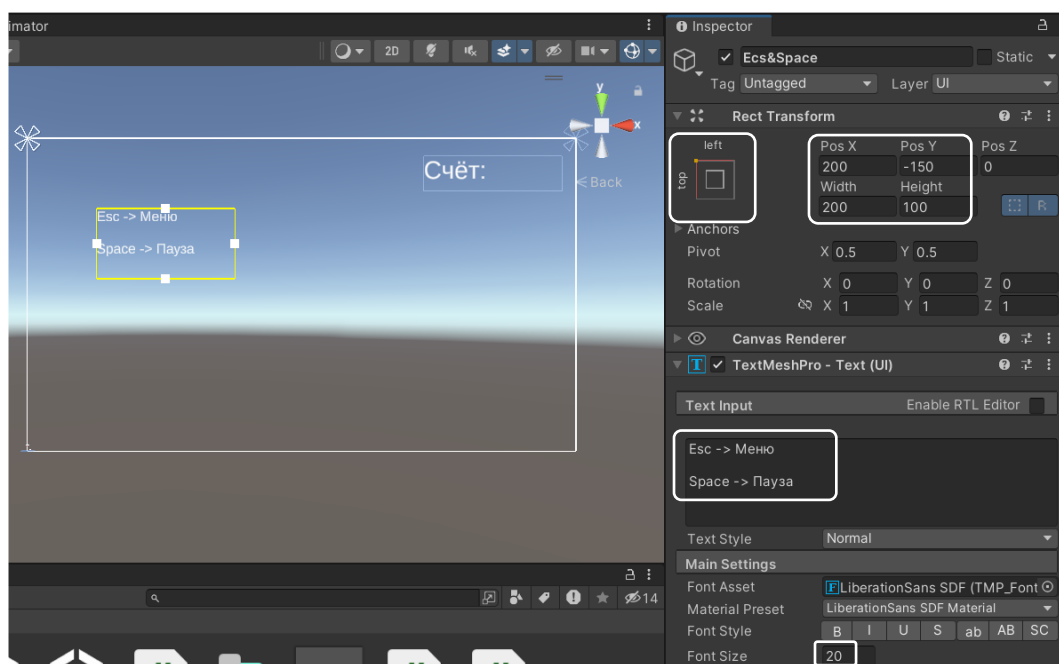
Запустите сцену и проверьте, что она работает корректно, а именно:

- при нажатии клавиши Space игра останавливается, появляется «Пауза», при повторном нажатии – продолжается, «Пауза» исчезает;
- при нажатии клавиши Esc происходит загрузка стартовой сцены с именем _0MainScene.

Чтобы проинформировать пользователя о введенном функционале, разместим элементы графического интерфейса в левой верхней части окна с игрой.

В окне Hierarchy создайте дубликат текстового объекта Score и переименуйте его в Ecs&Space.

Настройте параметры Rect Transform с положением и размером объекта Ecs&Space:



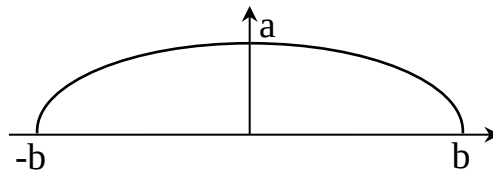
7.4. СОЗДАНИЕ ДВИЖЕНИЯ БЕЗ АНИМАЦИИ

Оживим нашу начальную сцену движением «солнца».

Движения без анимации можно получить, изменяя в коде координаты объекта в пространстве. Для этого достаточно задавать новые координаты для объекта:

```
transform.position = new Vector3(x0, y0,z0);
```

Пусть наше «солнце» будет двигаться по параболе.

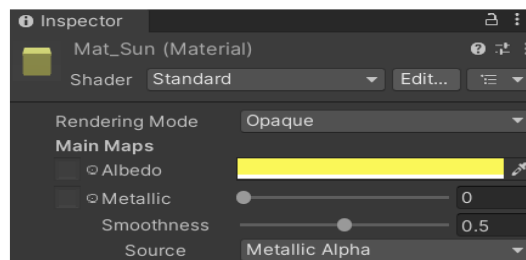


В этом случае функция будет выглядеть так:

$$f(x) = -\frac{a}{b^2}(x^2 - b^2) \text{ или } f(x) = -\frac{a}{b^2}(x - b)(x + b).$$

То есть нам необходимо обеспечить изменение аргумента x от $-b$ до b .

Как обычно создадим объект сфера, дадим ему имя Sun и добавим материал.



Также создадим скрипт **SunScript.cs**, в котором обеспечим изменение координат:

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class SunScript : MonoBehaviour
{
    float a = 7f, b = 10f, hx_SunPos = 0.01f;
    float R_Sun = 2f, kR = 4f/3f;
    float x_SunPos, y_SunPos, z_SunPos = 0.0f;
```

```

void Start()
{
    x_SunPos = -b;
    // Солнце движется по параболе
    y_SunPos = -a / (b * b) * (x_SunPos - b) * (x_SunPos + b);

    transform.localPosition =
        new Vector3(x_SunPos, y_SunPos, z_SunPos);
    transform.localScale =
        new Vector3(R_Sun * kR, R_Sun * kR, R_Sun * kR);
}

void Update()
{
    // Солнце движется по параболе
    y_SunPos = -a / (b * b) * (x_SunPos - b) * (x_SunPos + b);
    transform.localPosition =
        new Vector3(x_SunPos, y_SunPos, z_SunPos);

    x_SunPos += hx_SunPos; // * Time.deltaTime;
    if (x_SunPos > Math.Abs(b)) {
        hx_SunPos = -hx_SunPos;
        transform.localScale =
            new Vector3(R_Sun / kR, R_Sun / kR, R_Sun / kR);
    };
    if (x_SunPos < -Math.Abs(b)) {
        hx_SunPos = -hx_SunPos;
        transform.localScale =
            new Vector3(R_Sun * kR, R_Sun * kR, R_Sun * kR);
    }
}
}

```

И, соответственно, скриншот.

```
DirScript.cs SunScript.cs* X
лы SunScript
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class SunScript : MonoBehaviour
{
    float a = 7f, b = 10f, hx_SunPos = 0.01f;
    float R_Sun = 2f, kR = 4f/3f;
    float x_SunPos, y_SunPos, z_SunPos = 0.0f;

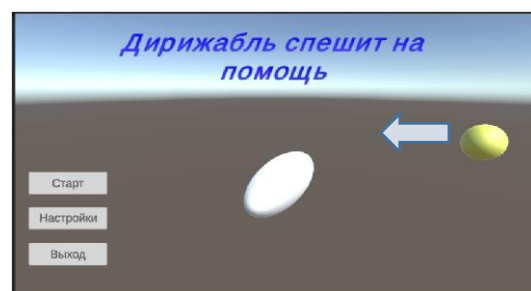
    void Start()
    {
        x_SunPos = -b;
        // Солнце движется по параболе
        y_SunPos = -a / (b * b) * (x_SunPos - b) * (x_SunPos + b);

        transform.localPosition =
            new Vector3(x_SunPos, y_SunPos, z_SunPos);
        transform.localScale =
            new Vector3(R_Sun * kR, R_Sun * kR, R_Sun * kR);
    }

    void Update()
    {
        // Солнце движется по параболе
        y_SunPos = -a / (b * b) * (x_SunPos - b) * (x_SunPos + b);
        transform.localPosition =
            new Vector3(x_SunPos, y_SunPos, z_SunPos);

        x_SunPos += hx_SunPos * Time.deltaTime;
        if (x_SunPos > Math.Abs(b)) {
            hx_SunPos = -hx_SunPos;
            transform.localScale =
                new Vector3(R_Sun / kR, R_Sun / kR, R_Sun / kR);
        };
        if (x_SunPos < -Math.Abs(b)) {
            hx_SunPos = -hx_SunPos;
            transform.localScale =
                new Vector3(R_Sun * kR, R_Sun * kR, R_Sun * kR);
        }
    }
}
```

Выглядит это примерно так:



7.4.1. Комментарии

В листинге переменные имеют такой смысл:

- **a** – максимальное значение **y** «солнца»;
- **b**, **hx_SunPos** обеспечивают изменение **x_SunPos** из диапазона **[-b, b]** с шагом **hx_SunPos**, при выходе из диапазона **hx_SunPos** меняет знак;
- **R_Sun** – радиус, **kR** – коэффициент для изменения размера «солнца»;
- изменение **x_SunPos** учитывает **Time.deltaTime** – встроенная переменная, которая будет изменять свое значение при разной частоте кадров;
- **y_SunPos** – значение функции $f(x)$;
- **x_SunPos**, **y_SunPos**, **z_SunPos** – координаты, которые изменяются в **Vector3(x_SunPos, y_SunPos, z_SunPos)** и обеспечивают движение «солнца»;
- **R_Sun*kR** увеличивает, а **R_Sun/kR** уменьшает радиус «солнца» в **transform.localScale**.

7.5. ИЗМЕНЕНИЕ МАТЕРИАЛА В СКРИПТЕ

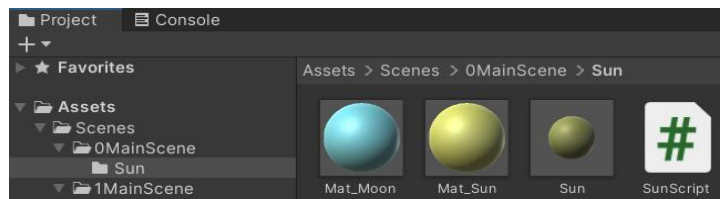
Все параметры **Material**, которые видны в инспекторе при просмотре материала, доступны через сценарий, что дает возможность изменить или оживить работу материала во время выполнения.

Это позволяет изменять числовые значения **Material**, менять цвета и динамически менять местами текстуры во время игры (набор функций, доступных для управления материалами с помощью скрипта, приведён в прил. 1).

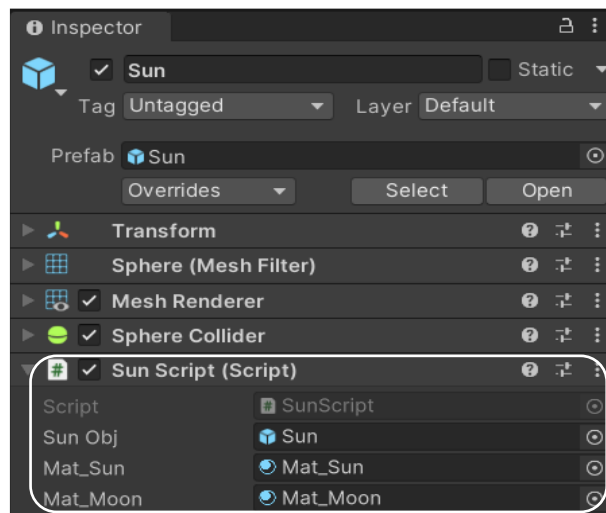
Важно отметить, что эти функции устанавливают только те свойства, которые доступны для текущего шейдера по материалу.

Сделаем так, чтобы слева направо двигалось солнце, а справа налево – например, «луна».

Для этого создадим материал для «луны», кликнув внутри **Scenes** правой кнопкой мыши и выбрав **Create → Material**, назовём его **Mat_Moon**:



Обязательно скрипту **SunScript.cs** назначаем созданные материалы.



Для того чтобы материал изменялся в зависимости от направления, в скрипт **SunScript.cs** внесем следующие изменения:

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class SunScript : MonoBehaviour
{
    float a = 7f, b = 10f, hx_SunPos = 0.01f;
    float R_Sun = 2f, kR = 4f / 3f;
    float x_SunPos, y_SunPos, z_SunPos = 0.0f;

    public GameObject SunObj;
    public Material Mat_Sun;
    public Material Mat_Moon;

    void Start()
    {
        SunObj.GetComponent<MeshRenderer>().material = Mat_Sun;

        x_SunPos = -b;
        // Солнце движется по параболе
        y_SunPos = -a / (b * b) * (x_SunPos - b) * (x_SunPos + b);
```

```

        transform.localPosition =
            new Vector3(x_SunPos, y_SunPos, z_SunPos);
        transform.localScale =
            new Vector3(R_Sun * kR, R_Sun * kR, R_Sun * kR);
    }

    void Update()
    {
        // Солнце движется по параболе
        y_SunPos = -a / (b * b) * (x_SunPos - b) * (x_SunPos + b);
        transform.localPosition =
            new Vector3(x_SunPos, y_SunPos, z_SunPos);

        x_SunPos += hx_SunPos; // * Time.deltaTime;
        if (x_SunPos > Math.Abs(b))
        {
            hx_SunPos = -hx_SunPos;
            transform.localScale =
                new Vector3(R_Sun / kR, R_Sun / kR, R_Sun / kR);
            SunObj.GetComponent<MeshRenderer>().material = Mat_Moon;
        };
        if (x_SunPos < -Math.Abs(b))
        {
            hx_SunPos = -hx_SunPos;
            transform.localScale =
                new Vector3(R_Sun * kR, R_Sun * kR, R_Sun * kR);
            SunObj.GetComponent<MeshRenderer>().material = Mat_Sun;
        }
    }
}

```

7.5.1. Комментарии

В листинге переменные имеют такой смысл:

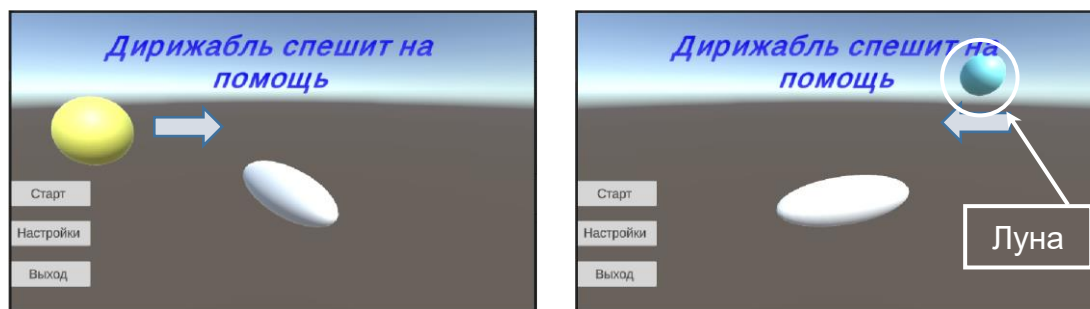
- описание объекта «солнце» – **SunObj**;
- описание соответствующих материалов **Mat_Sun** и **Mat_Moon**;
- в зависимости от направления движения – это будет или «солнце», или «луна»:

```

SunObj.GetComponent<MeshRenderer>().material =
    Mat_Sun | Mat_Moon.

```

Теперь наша первая сцена выглядит примерно так:



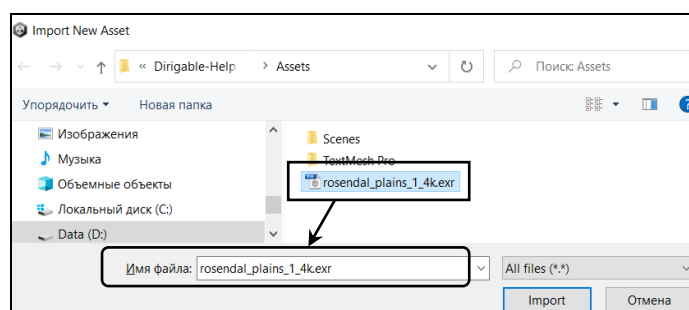
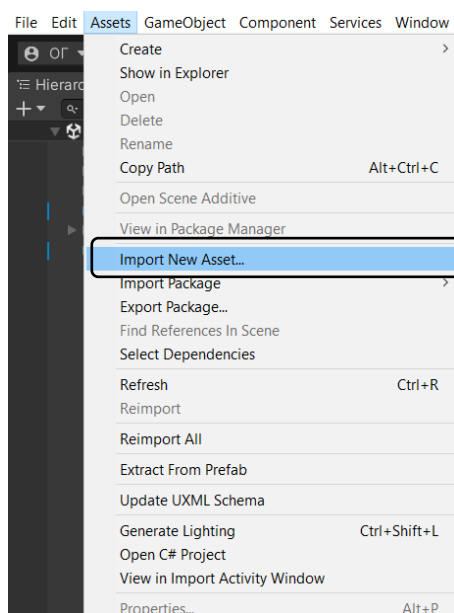
7.6. ИЗМЕНЕНИЕ SKYBOX В UNITY

Для изменения Skybox сцены понадобятся специальные текстуры, одна из них – HDR (High Dynamic Range). Они содержат больше информации и полезны для освещения сцен.

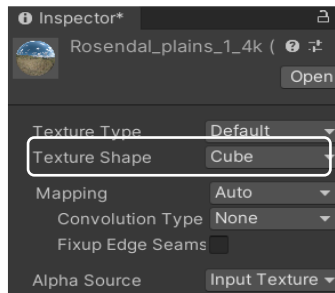
Существует немало ресурсов, содержащих самые разные текстуры, их несложно найти. Воспользуемся бесплатным ресурсом со страницы <https://polyhaven.com/hdri>

Загрузим, например, `rosendal_plains_1_4k.exr`.

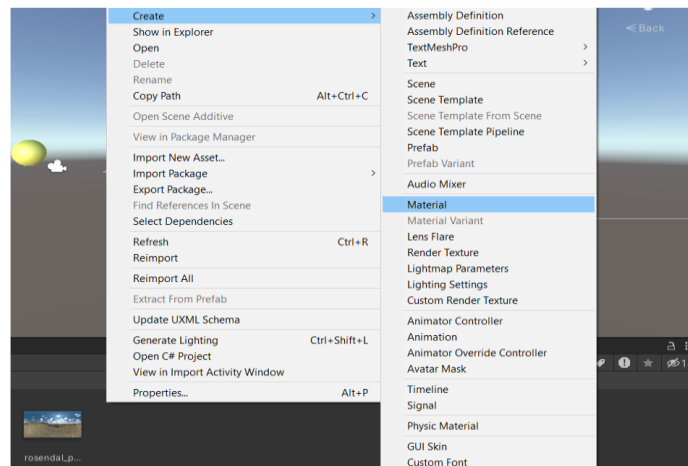
Импортируем HDR-текстуру в Unity из соответствующей папки (Assets → Import New Asset...).



В инспекторе установим для параметра Texture Shape значение Cube.

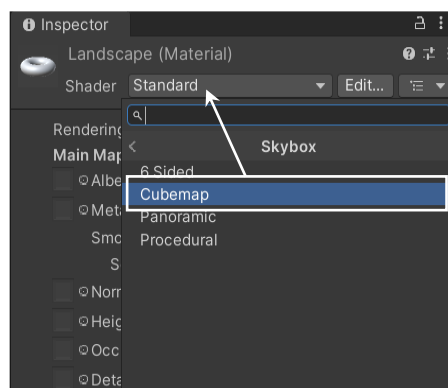


Кликнув внутри Scenes правой кнопкой мыши и выбрав Create → Material, создадим его.

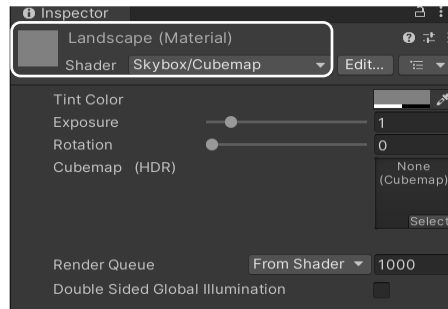


Дадим ему имя Landscape, а затем кликаем по нему мышкой, чтобы увидеть свойства в инспекторе.

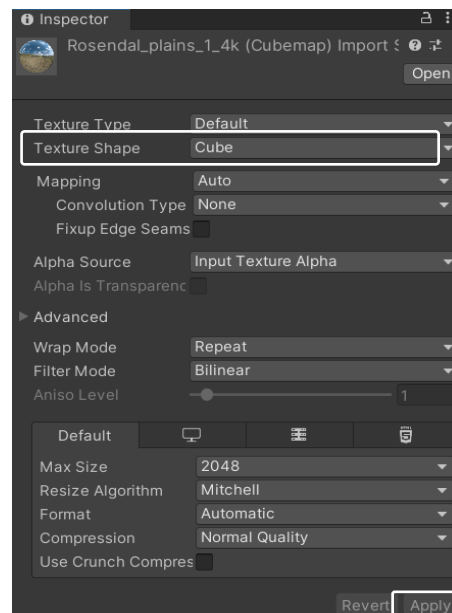
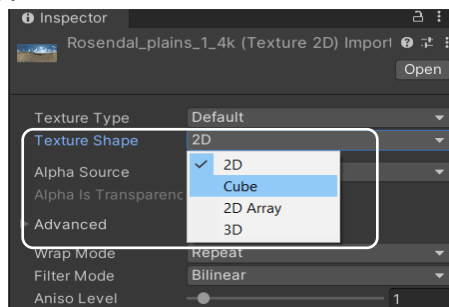
В заголовке инспектора для Landscape значение поля Shader (по умолчанию – Standard) меняем его на Cubemap.



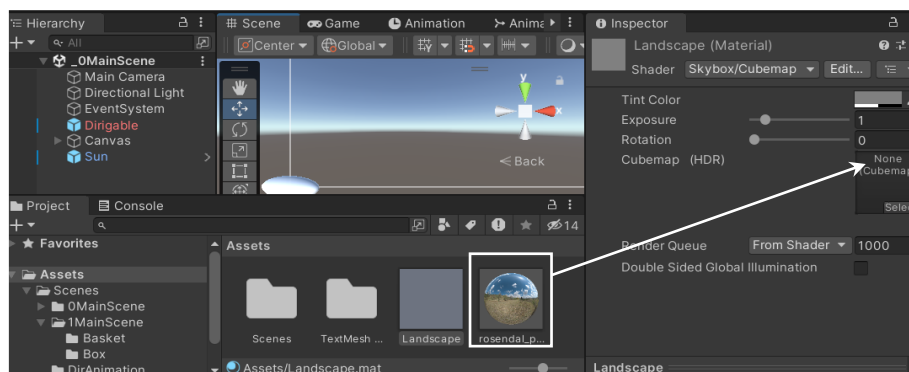
Теперь у материала изменились параметры.



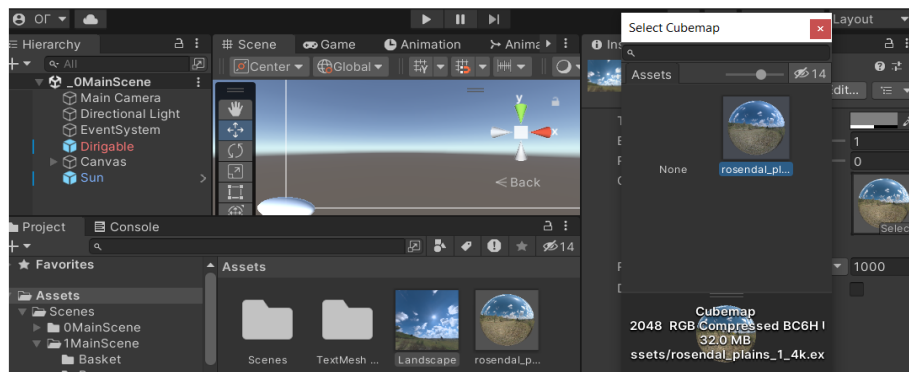
Внесём некоторые коррективы в изображение. Выбираем импортированную HDR-текстуру и изменяем поле Texture Shape на Cube.



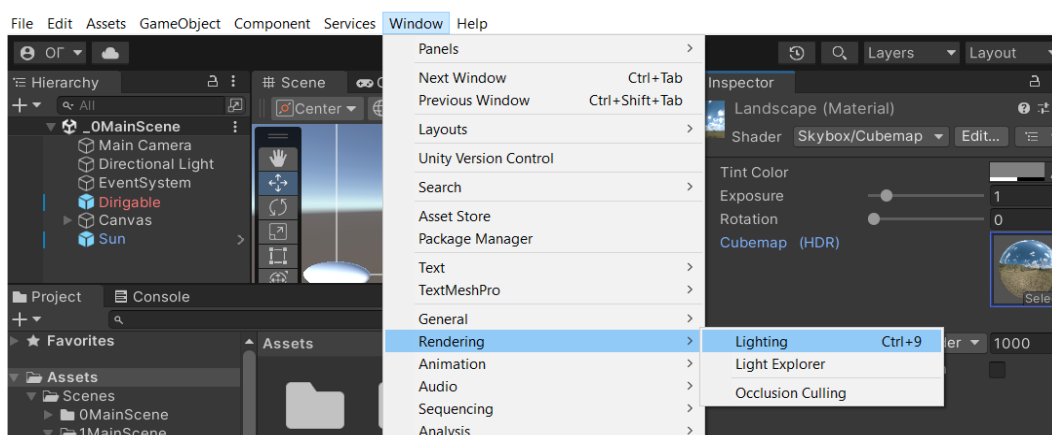
Это позволяет использовать текстуру для материала Landscape. Можем перетащить текстуру прямо в поле None.



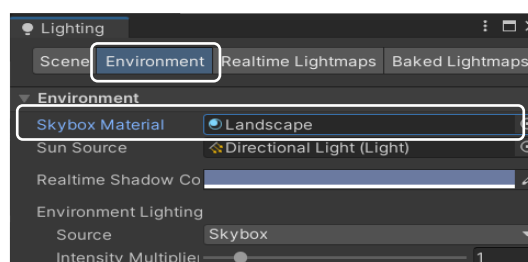
Или нажать кнопку Select и выбрать её во всплывающем окне.



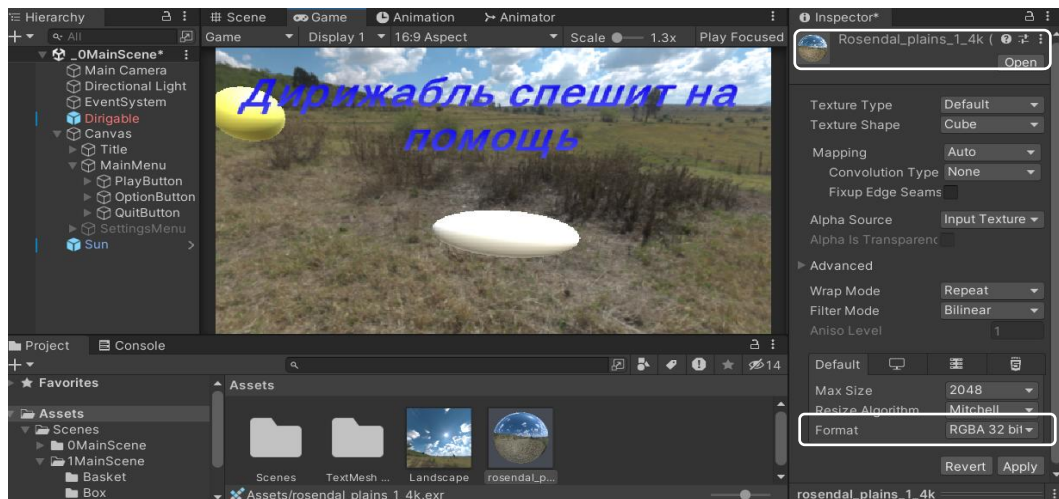
Для применения настроенного материала Landscape к сцене нужно открыть вкладку Lighting (Window → Rendering → Lighting Ctrl+9).



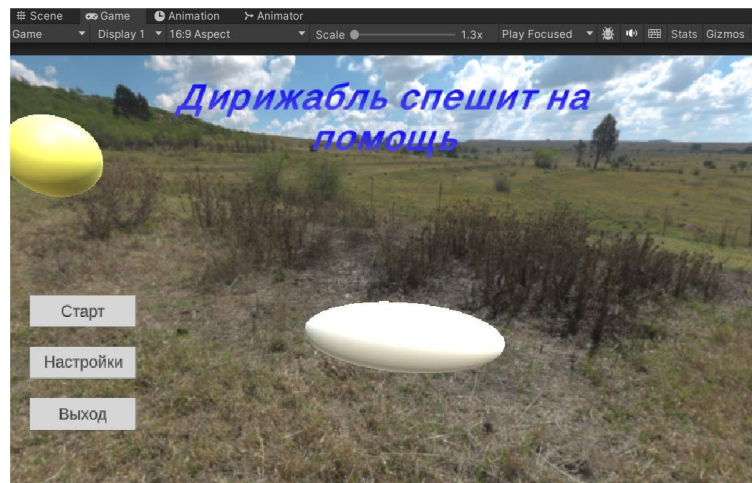
Внутри окна Lighting переходим в раздел Environment и в поле Skybox Material назначаем материал Landscape.



Для изменения качества изображения можно в инспекторе изменить его сжатие на RGB 16, RGB 24 или RGBA 32. Это приведет к увеличению качества, но также и объёма изображения.



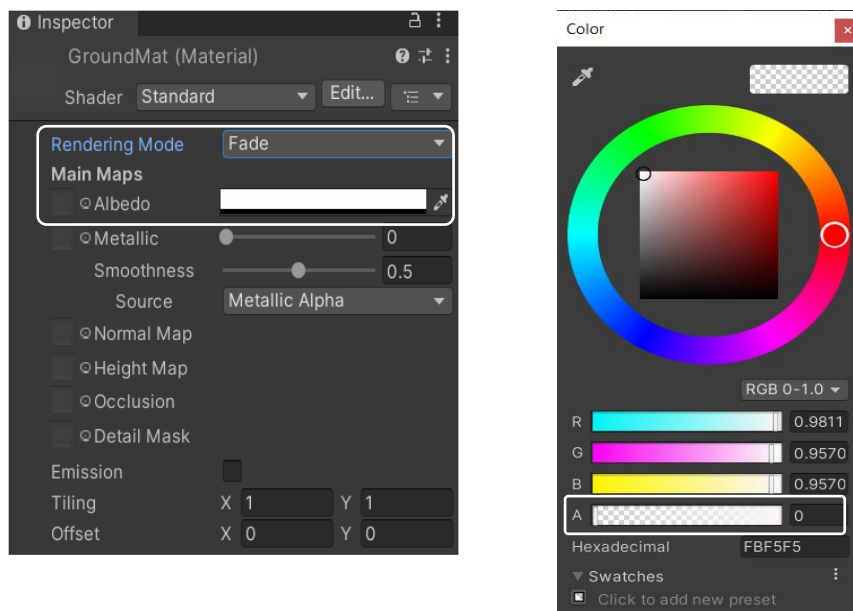
Теперь первая сцена `_0MainScene` выглядит так.



Аналогичные действия делаем для второй сцены `_1MainScene`.

При этом «землю» нужно сделать прозрачной. Для этого создадим материал `GroundMat` и установим для режима `Rendering Mode` значение `Transparent` (прозрачный) или `Fade` (исчезающий).

В пункте Albedo выбираем окно Color и устанавливаем ползунок A. Значение 0 означает полную прозрачность, а 1 – полную непрозрачность.



Так же изменим цвет текста поля Score. В итоге вот что получилось...



8. ИГРОВОЙ ПРОТОТИП «БОЛЬШОЙ ГЛАЗ»

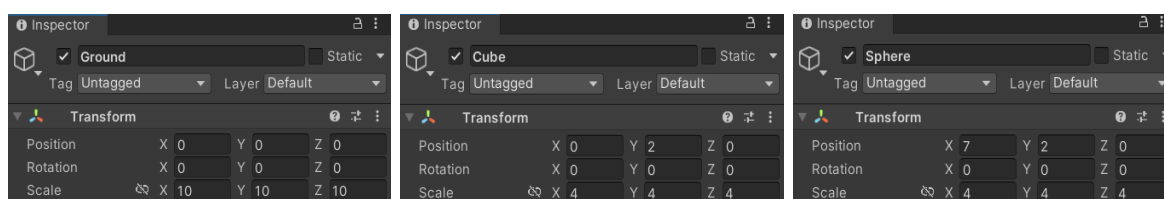
Создадим робота «Большой Глаз», который будет показывать то, что видит...

8.1. СОЗДАНИЕ «ОКРУЖЕНИЯ»

Робот будет рассматривать кубики и шарики, которые разбросаны на «Земле».

Вначале создадим игровой объект «Земля». Для этого на верхней панели быстрого доступа в среде Unity выберем GameObject → 3D Object → Plane.

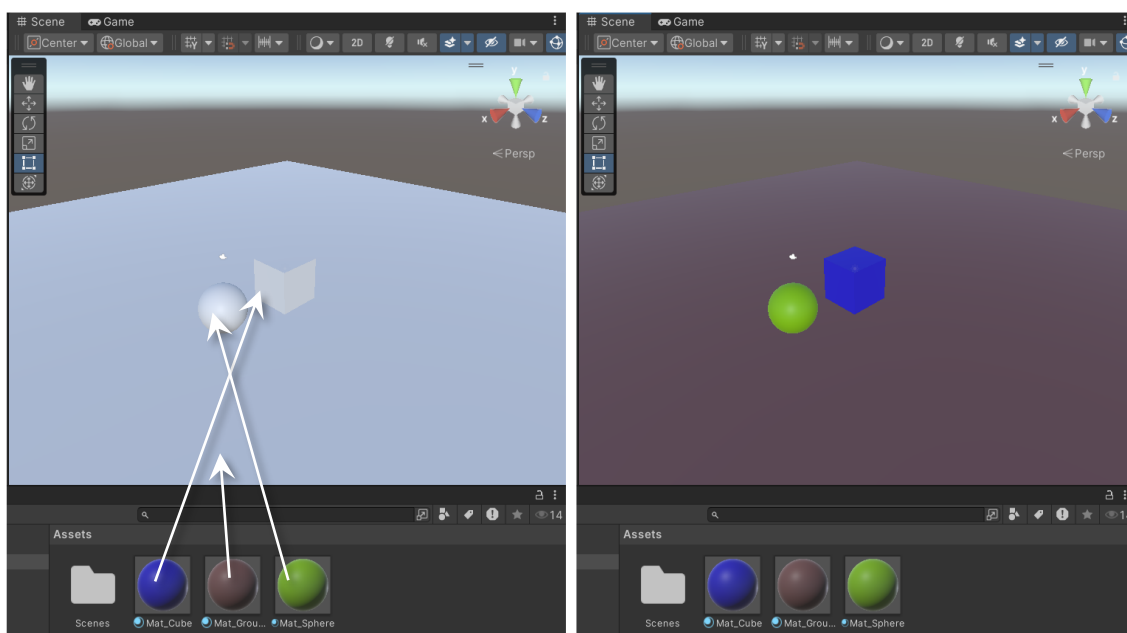
Объект «Земля» назовем Ground, а также создадим кубики и шарики и установим такие значения:



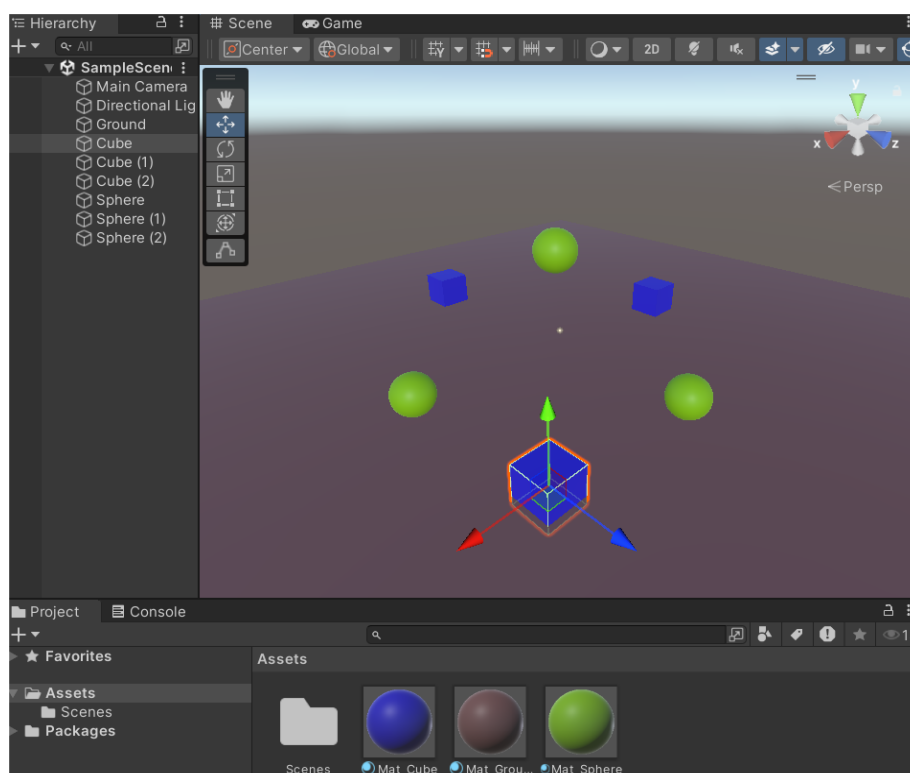
Далее создадим материалы для наших объектов, чтобы их раскрасить. Кликнув внутри Scenes правой кнопкой мыши и выбрав Create → Material, назовём их Mat_Cube, Mat_Ground и Mat_Sphere:



Перетаскивая созданные материалы на объекты, получим такие цвета.

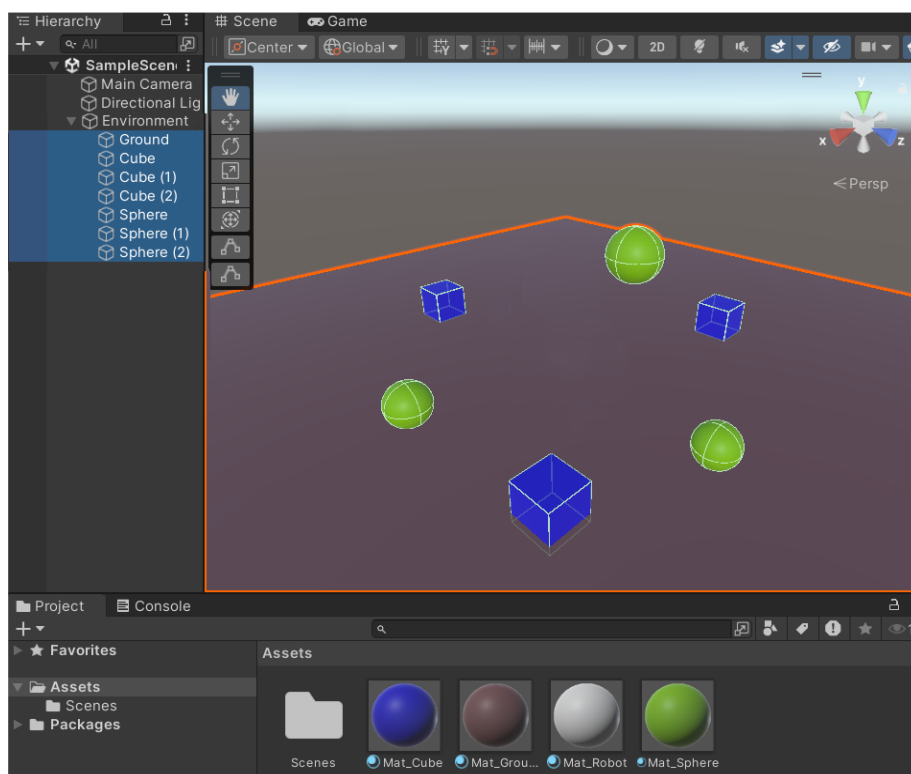


Копии кубика и шарика (Ctrl+D) расположим по кругу.

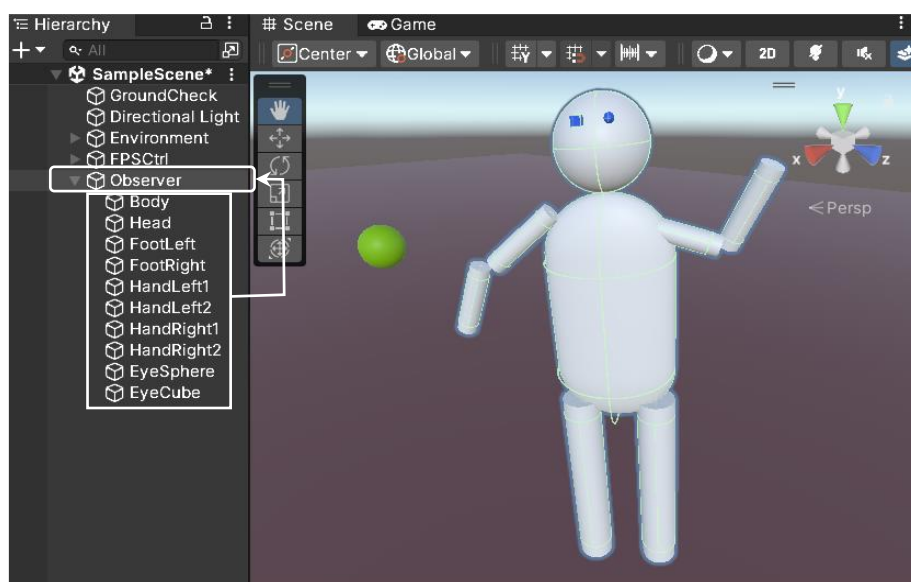


Создадим пустой объект GameObject (в окне Hierarchy выбираем Create Empty) и назовём его Environment (Окружение).

Все наши кубики, шарики и «Землю» поместим в этот объект. Для этого выделяем их (нажимая Shift) и помещаем в Environment.



Так как «кубики» и «шарики» одинаковые, то создадим ещё «наблюдателя» для того, чтобы была, так сказать, «точка отсчёта».



Части тела «наблюдателя» создаем из «подручных» средств: сферы, капсулы, цилиндров. Это странный наблюдатель: у него

один глаз сферический, а другой – кубический (видимо, установлен какой-то датчик).

В пустой объект `GameObject`, назовём его `Observer` (Наблюдатель), поместим в него все части «тела».

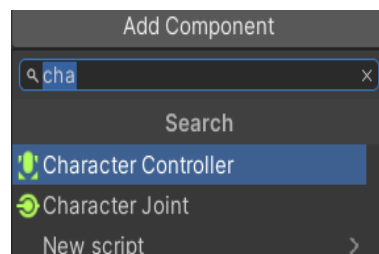
У этого наблюдателя такие параметры.

Объект	Transform	X	Y	Z
Body	Position	0	3	0
	Rotation	0	0	0
	Scale	3	3	1
Head	Position	0	7	0
	Rotation	0	0	0
	Scale	2	2	2
FootLeft	Position	1	-2.5	0
	Rotation	0	0	0
	Scale	1	3	1
FootRight	Position	-1	-2.5	0
	Rotation	0	0	0
	Scale	1	3	1
HandLeft1	Position	-1.5	5	0
	Rotation	-22	-6	-70
	Scale	0.5	1	0.5
HandLeft2	Position	-3	5.5	0
	Rotation	17	1	21
	Scale	0.5	1	0.5
HandRight1	Position	2	5	0
	Rotation	-22	22	-132
	Scale	0.5	1	0.5
HandRight2	Position	3.5	3.5	0
	Rotation	11	-11	22
	Scale	0.5	1	0.5
EyeSphere	Position	-0.3	7.5	1
	Rotation	0	0	0
	Scale	0.2	0.2	0.2
EyeCube	Position	0.3	7.5	1
	Rotation	0	4	0
	Scale	0.2	0.2	0.2

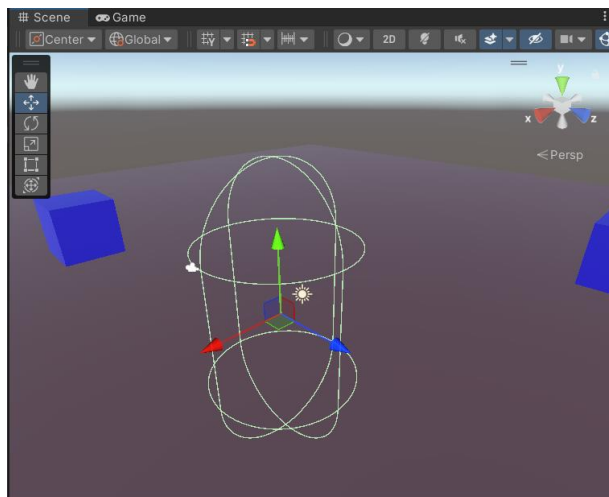
8.2. СОЗДАНИЕ «РОБОТА»

Прежде чем создать робота, создадим FPS¹-контроллер. Как и раньше, создадим пустой объект GameObject (в окне Hierarchy выбираем Create Empty) и назовём его FPSCtrl.

Добавим ему компонент Character Controller (позволяет выполнять движения, ограниченные столкновениями, без необходимости иметь дело с жестким телом. На него не действуют никакие силы, и он будет перемещаться только при вызове функции Move. Он будет выполнять перемещения, но будет ограничен столкновениями).

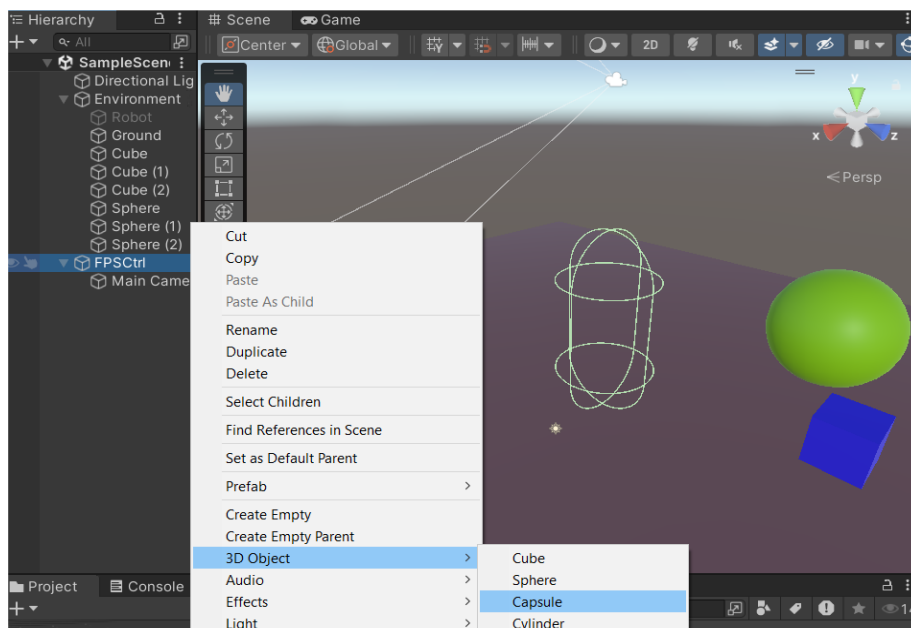


Появилась прозрачная капсула.



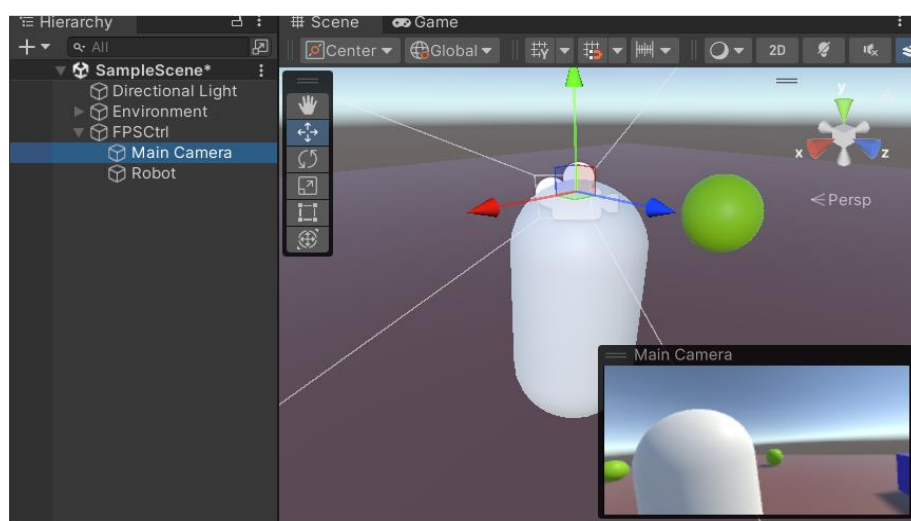
Поместим в контроллер FPSCtrl капсулу (FPSCtrl → правая кнопка мыши → 3D Object → Capsule) и переименуем эту капсулу в Robot. Это и будет наш робот.

¹ FPS (First-person shooter) – жанр компьютерных игр, в которых игровой процесс основывается на действиях (наблюдение, стрельба и т.п.) от первого лица таким образом, чтобы игрок воспринимал происходящее глазами главного героя.



Создадим материал для «робота» (внутри Scenes правой кнопкой мыши → Create → Material), назовём его Mat_Robot.

Переместим камеру в верхнюю часть капсулы, т.е. туда, где находится «голова робота».



Отметим особенности обзора. Для перемещения взгляда вверх-вниз нужно изменять координату X камеры, а для поворота влево-вправо – координату Y FPS-контроллера.

Перемещаем (перетаскиваем) камеру на контроллер, теперь она должна перемещаться вместе с контроллером и взгляд «робота» можно перемещать по всем трём координатам.

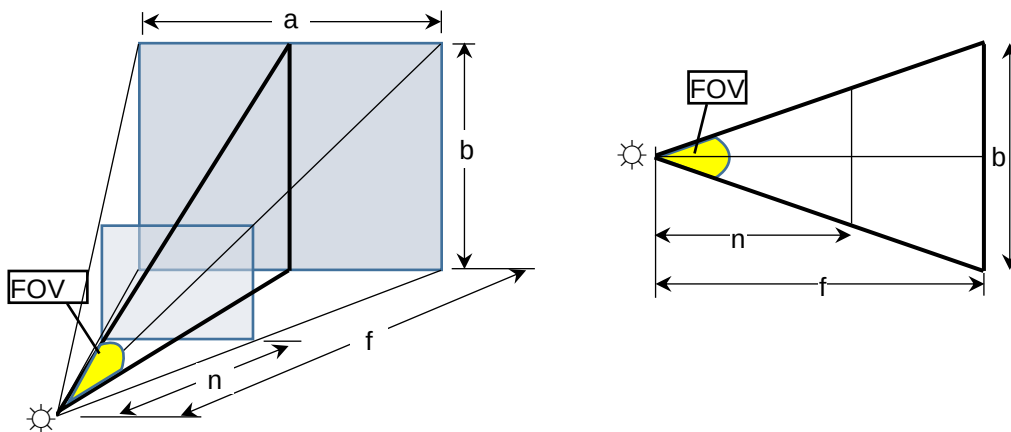
8.3. КАМЕРЫ

Сцена Unity представляет игровые объекты в трехмерном пространстве. Поскольку экран зрителя двумерный, Unity необходимо захватить вид и «выровнять» его для отображения. Это делается с помощью камер.

Настройки компонента Camera определяют размер и форму области, попадающей в поле зрения. При соответствующих настройках этих параметров камера может отображать на экране то, что она в данный момент «видит». По мере перемещения и вращения игрового объекта соответственно перемещается и вращается отображаемый вид.

Камера в реальном мире, как и человеческий глаз, «видит» мир таким образом, что объекты кажутся тем меньше, чем дальше они находятся от точки обзора. Этот хорошо известный эффект *перспективы* широко используется в искусстве и компьютерной графике и важен для создания реалистичных сцен. Камера, которая не уменьшает размера объектов с расстоянием, называется *ортографической*, и у камер Unity есть опция для этого.

Как перспективные, так и ортографические камеры имеют ограничение на то, как далеко они могут «видеть» из своего текущего положения. Ограничение определяется плоскостью, перпендикулярной направлению камеры вперед (Z). Это *дальняя плоскость отсечения*, поскольку объекты, находящиеся на большем расстоянии от камеры, «отсекаются». Также имеется соответствующая *ближняя плоскость отсечения* рядом с камерой – видимый диапазон расстояний равен расстоянию между двумя плоскостями.

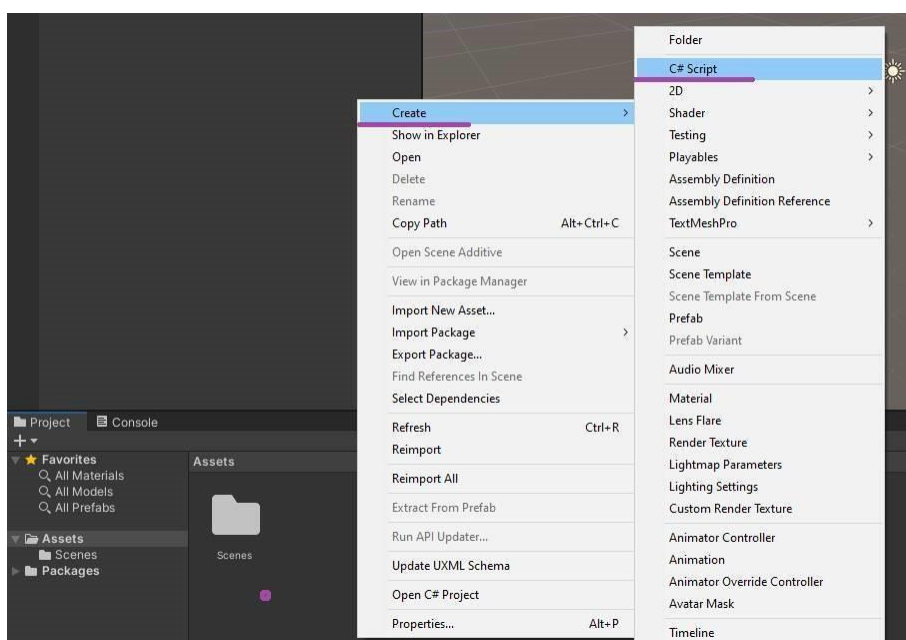


При использовании перспективы кажется, что объекты уменьшаются в размерах по мере увеличения расстояния до камеры. Это означает, что ширина и высота видимой части сцены увеличивается с увеличением расстояния. Таким образом, объем обзора перспективной камеры представляет собой не прямоугольник, а пирамидальную форму с вершиной в положении камеры и основанием в дальней плоскости отсечения. То есть такая усеченная форма пирамиды известна как *усеченный конус*. Поскольку его высота непостоянна, усеченное изображение определяется отношением его ширины к высоте (известным как *соотношение сторон*) и углом между верхом и низом при вершине (известным как *поле зрения* или FOV²).

8.4. СКРИПТ-ФАЙЛ ROBOTSCRIPT.CS

Создадим новый C# Script-файл и назовём его `RobotScript.cs`.

Кликните правой кнопкой мыши внутри папки Assets и выберите из контекстного меню `Create → C# Script`.



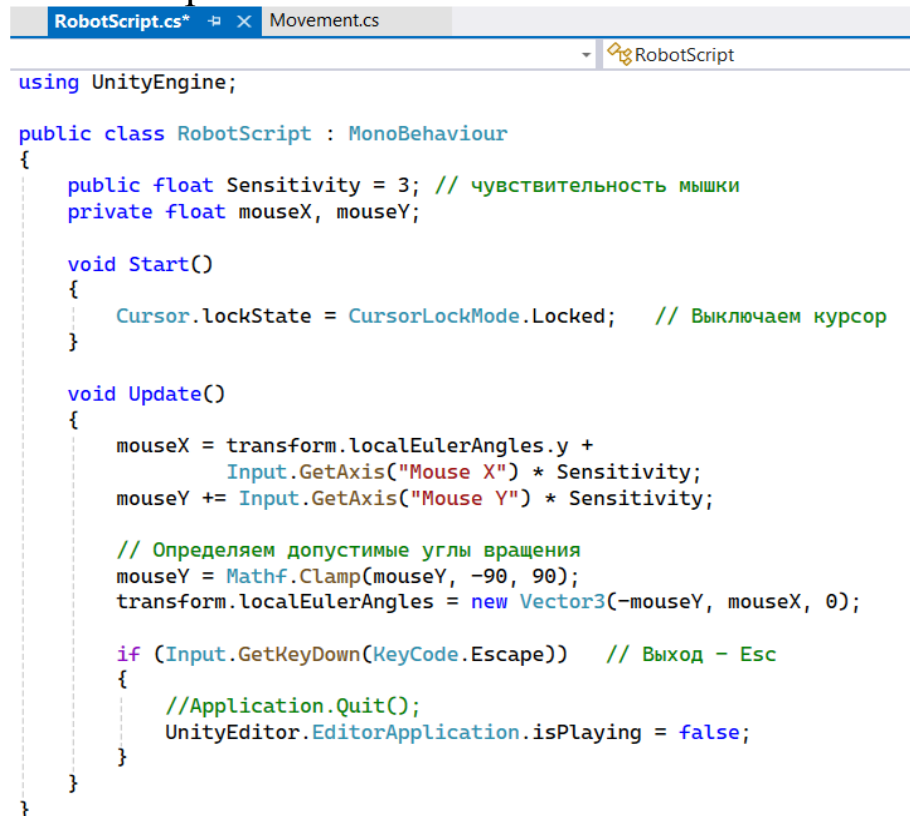
² Field Of View (поле зрения) – это конусообразная проекция зрачка глаза или виртуальной камеры. Всё, что находится внутри этой конусообразной проекции, можно увидеть, не поворачивая головы.

Код RobotScript.cs:

```
using UnityEngine;

public class RobotScript : MonoBehaviour
{
    public float Sensitivity = 3; // чувствительность мышки
    private float mouseX, mouseY;
    void Start()
    {
        Cursor.lockState = CursorLockMode.Locked; // Выключаем курсор
    }
    void Update()
    {
        mouseX = transform.localEulerAngles.y +
            Input.GetAxis("Mouse X") * Sensitivity;
        mouseY += Input.GetAxis("Mouse Y") * Sensitivity;
        // Определяем допустимые углы вращения
        mouseY = Mathf.Clamp(mouseY, -90, 90);
        transform.localEulerAngles = new Vector3(-mouseY, mouseX, 0);
        if (Input.GetKeyDown(KeyCode.Escape)) // Выход - Esc
        {
            //Application.Quit();
            UnityEditor.EditorApplication.isPlaying = false;
        }
    }
}
```

И скриншот из VS Studio.



```
RobotScript.cs* Movement.cs
RobotScript

using UnityEngine;

public class RobotScript : MonoBehaviour
{
    public float Sensitivity = 3; // чувствительность мышки
    private float mouseX, mouseY;

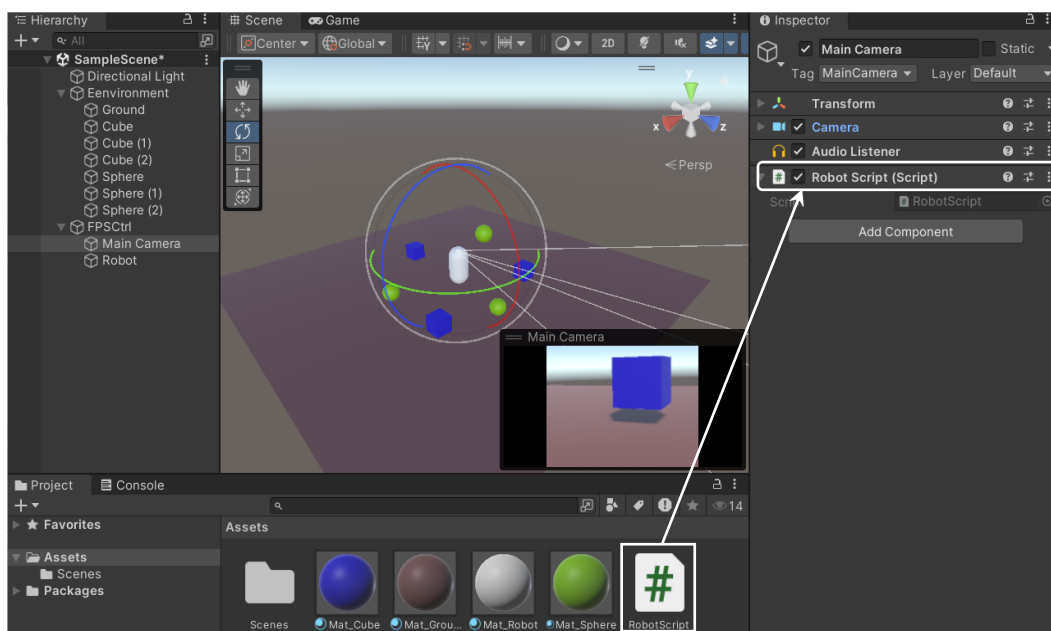
    void Start()
    {
        Cursor.lockState = CursorLockMode.Locked; // Выключаем курсор
    }

    void Update()
    {
        mouseX = transform.localEulerAngles.y +
            Input.GetAxis("Mouse X") * Sensitivity;
        mouseY += Input.GetAxis("Mouse Y") * Sensitivity;

        // Определяем допустимые углы вращения
        mouseY = Mathf.Clamp(mouseY, -90, 90);
        transform.localEulerAngles = new Vector3(-mouseY, mouseX, 0);

        if (Input.GetKeyDown(KeyCode.Escape)) // Выход - Esc
        {
            //Application.Quit();
            UnityEditor.EditorApplication.isPlaying = false;
        }
    }
}
```

Скрипт-файл **RobotScript.cs** перетаскиваем на камеру, а FPS-контроллеру назначаем Robot Body.



8.4.1. Комментарии

В Script-файле **RobotScript.cs** переменные имеют следующий смысл:

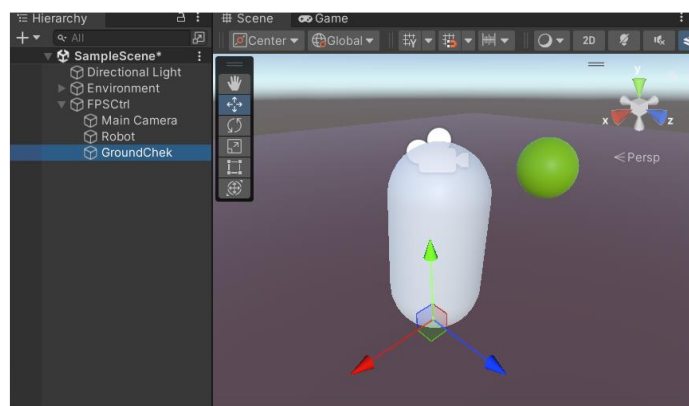
- **robotBody** – тело робота, которое будем вращать;
- **Sensivity = 100f** – чувствительность мыши;
- **xRotation = 0f** – поворот камеры вверх-вниз;
- **mouseX, mouseY** – произведение соответствующих координат мыши, чувствительности мыши **Sensivity** и временного интервала **Time.deltaTime**. Это встроенная переменная, которая будет изменять свое значение при разной частоте кадров;
- **Quaternion.Euler(xRotation, 0f, 0f)** вращает объект **robotBody**, используется для хранения трехмерной ориентации, а также для описания относительного поворота от одной ориентации к другой.

8.5. СКРИПТ-ФАЙЛ MOVEMENT.CS

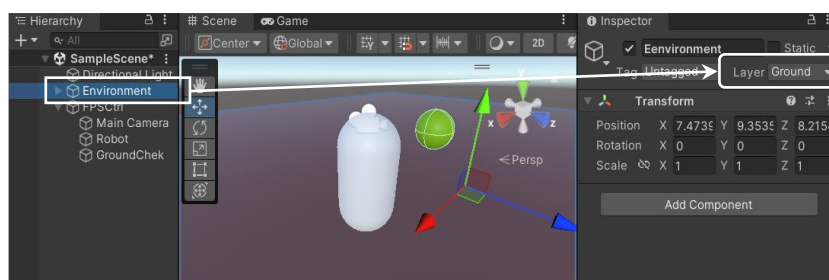
Скрипт-файл будет обеспечивать движение и контроль текущего положения робота.

Для этого создадим пустой объект **GameObject** (в окне **Hierarchy** выбираем **Create Empty**) и назовём его **GroundCheck**.

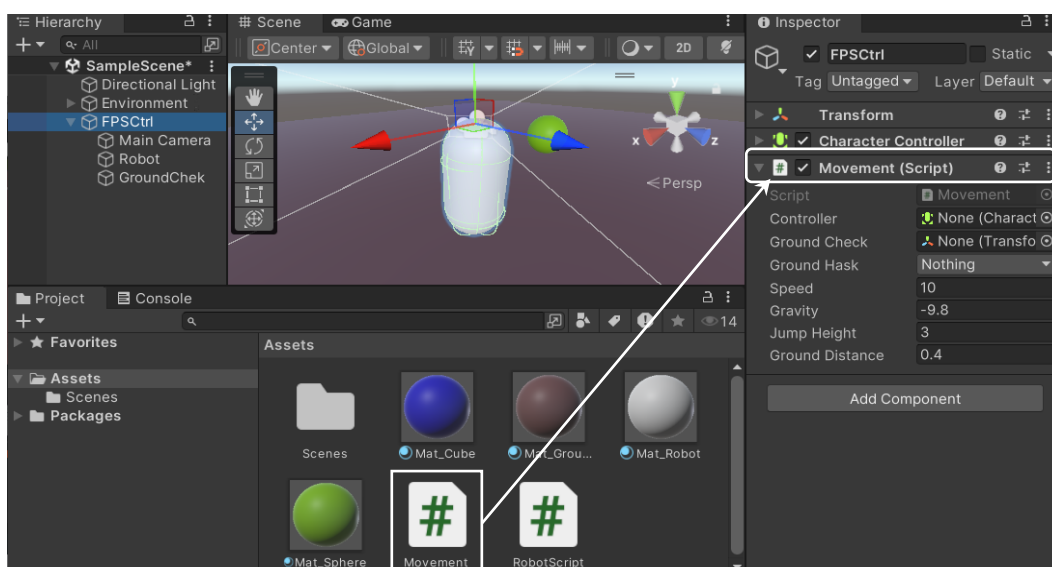
Отправим его в нижнюю часть капсулы («ноги робота»), чтобы проверить, на чём он стоит.



Установим для окружения Environment слой Ground, теперь каждый объект у нас имеет слой земли.

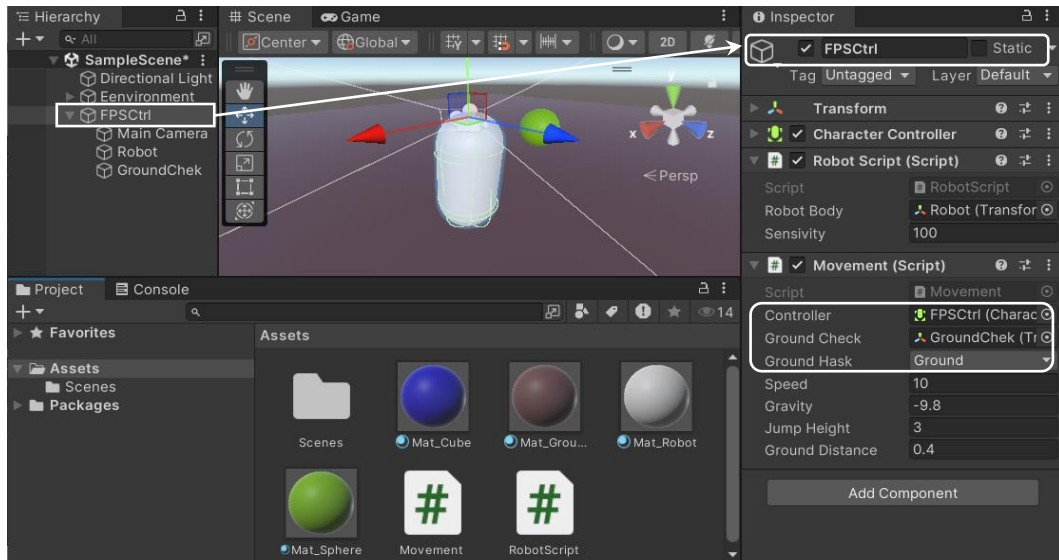


Создаём скрипт Movement.cs и прикрепляем его к контроллеру FPSCtrl.



Скрипту Movement.cs устанавливаем поля:

- Controller – FPSCtrl;
- Ground Check – GroundCheck;
- Ground Hask – Ground.



Собственно, код скрипта Movement.cs:

```
using UnityEngine;

public class Movement : MonoBehaviour
{
    public CharacterController controller;
    public Transform groundCheck;
    public LayerMask groundHask;
    public float speed = 1f;
    public float gravity = -9.8f;
    public float jumpHeight = 5f;
    Vector3 velocity;

    void Start()
    {
        velocity.y = 2;
    }
    void Update()
    {
        // Считываем оси движения
        float moveX = Input.GetAxis("Horizontal");
        float moveZ = Input.GetAxis("Vertical");

        if (Input.GetKeyDown(KeyCode.Space)) // Пробел – прыжок
        {
            velocity.y = Mathf.Sqrt(jumpHeight * -4f * gravity);
        }
    }
}
```

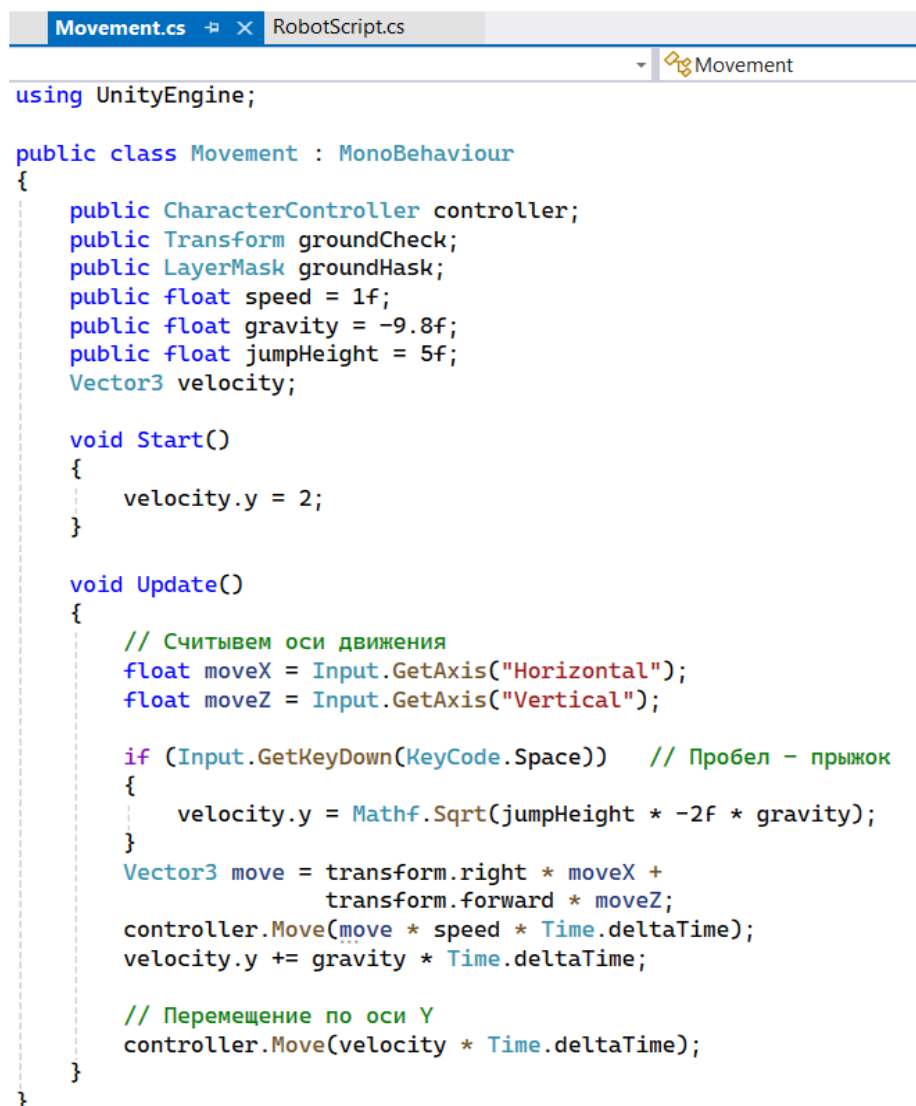
```

        Vector3 move = transform.right * moveX +
                        transform.forward * moveZ;
        controller.Move(move * speed * Time.deltaTime);
        velocity.y += gravity * Time.deltaTime;

        // Перемещение по оси Y
        controller.Move(velocity * Time.deltaTime);
    }
}

```

Скриншот скрипта Movement.cs.



```

using UnityEngine;

public class Movement : MonoBehaviour
{
    public CharacterController controller;
    public Transform groundCheck;
    public LayerMask groundMask;
    public float speed = 1f;
    public float gravity = -9.8f;
    public float jumpHeight = 5f;
    Vector3 velocity;

    void Start()
    {
        velocity.y = 2;
    }

    void Update()
    {
        // Считываем оси движения
        float moveX = Input.GetAxis("Horizontal");
        float moveZ = Input.GetAxis("Vertical");

        if (Input.GetKeyDown(KeyCode.Space)) // Пробел - прыжок
        {
            velocity.y = Mathf.Sqrt(jumpHeight * -2f * gravity);
        }
        Vector3 move = transform.right * moveX +
                        transform.forward * moveZ;
        controller.Move(move * speed * Time.deltaTime);
        velocity.y += gravity * Time.deltaTime;

        // Перемещение по оси Y
        controller.Move(velocity * Time.deltaTime);
    }
}

```

И последний штрих: пусть наш «наблюдатель» осматривается. Для это создадим простенький скрипт `ObserverScript.cs`. и перетаскиваем его на объект `Observer`.

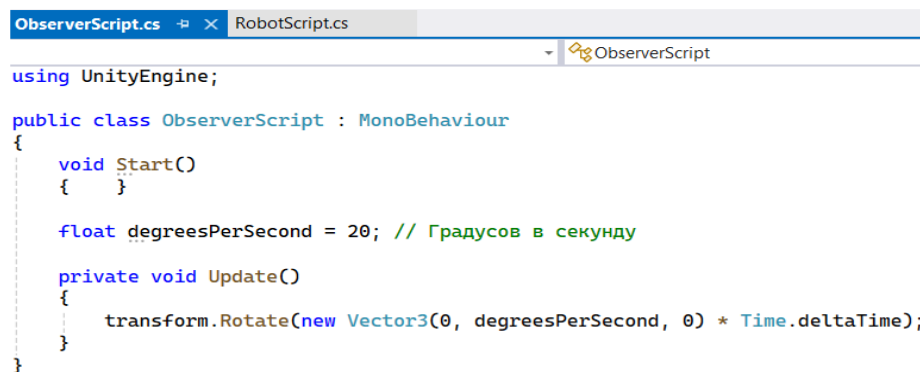
```
using UnityEngine;

public class ObserverScript : MonoBehaviour
{
    void Start()
    {
    }

    float degreesPerSecond = 20; // Градусов в секунду

    private void Update()
    {
        transform.Rotate(new Vector3(0, degreesPerSecond, 0)
            * Time.deltaTime);
    }
}
```

Скриншот скрипта `ObserverScript.cs`.



```
ObserverScript.cs [x] RobotScript.cs
using UnityEngine;

public class ObserverScript : MonoBehaviour
{
    void Start()
    {
    }

    float degreesPerSecond = 20; // Градусов в секунду

    private void Update()
    {
        transform.Rotate(new Vector3(0, degreesPerSecond, 0) * Time.deltaTime);
    }
}
```

8.5.1. Комментарии

В Script-файле `Movement.cs` переменные имеют следующий СМЫСЛ:

- `speed = 1f` – скорость;
- `gravity = -9.8f` – гравитация;
- `jumpHeight = 5f` – высота прыжка;
- Условие для прыжка – если нажата клавиша «пробел»
`if (Input.GetButtonDown(KeyCode.Space))`
 - в этом случае рассчитываем скорость по оси Y согласно влиянию гравитации и заданной высоте (физическая формула)
`velocity.y = Mathf.Sqrt(jumpHeight*(-2f)*gravity);`
 - создаем вектор смещения вправо-влево и смещение вперед-назад для каждой из осей:
`Vector3 move = transform.right * moveX +
transform.forward * moveZ;`

– используя контроллер, перемещаемся на новые координаты, учитывая скорость

```
controller.Move(move * speed * Time.deltaTime);
```

– к скорости по Y прибавляем гравитацию (тем самым уменьшая скорость, робот будет падать)

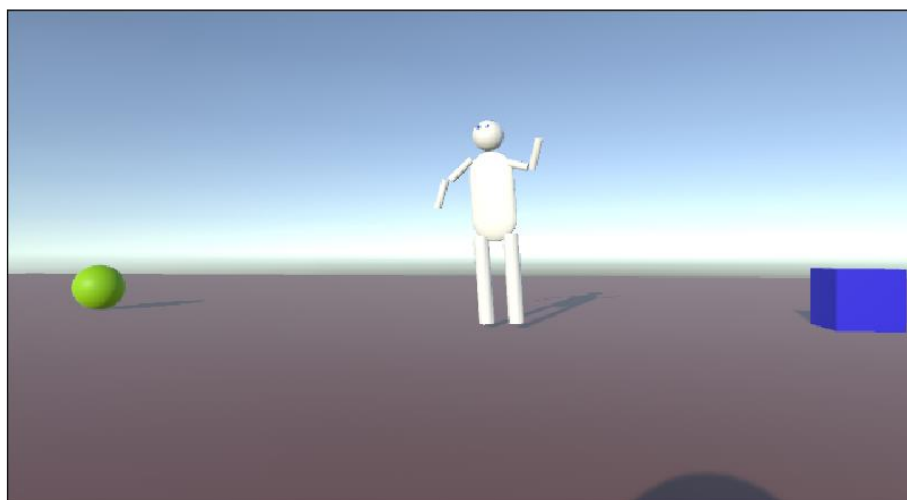
```
velocity.y += gravity * Time.deltaTime;
```

– перемещение по оси Y

```
controller.Move(velocity * Time.deltaTime).
```

В Script-файле `ObserverScript.cs` переменная `degreesPerSecond` задает скорость вращения (градусы в секунду).

В итоге может получиться примерно так...



8.6. РЕАЛИСТИЧНЫЕ КООРДИНАТЫ ОБЪЕКТОВ

Управление роботом осуществляется стрелками и мышкой.

Для координат используемых объектов можно использовать, например, такие значения.

Объект	Transform	X	Y	Z
Ground	Position	0	0	0
	Rotation	0	0	0
	Scale	10	10	10
Cube	Position	25	1	25
	Rotation	0	0	0
	Scale	2	2	2
Cube (1)	Position	-7	1	22
	Rotation	0	0	0
	Scale	2	2	2
Cube (2)	Position	22	1	-11
	Rotation	0	0	0
	Scale	2	2	2
Sphere	Position	-9	1	-7

Объект	Transform	X	Y	Z
	Rotation	0	0	0
	Scale	2	2	2
Sphere (1)	Position	22	1	7
	Rotation	0	0	0
	Scale	2	2	2
Sphere (2)	Position	15	1	33
	Rotation	0	0	0
	Scale	2	2	2
Robot	Position	0	0	0
	Rotation	0	0	0
	Scale	4	4	4
Main Camera	Position	0	4	0
	Rotation	0	-60	0.6
	Scale	1	1	11
GroundChek	Position	15	-1	-18
	Rotation	1	80	3
	Scale	1	1	1

9. ИГРОВОЙ ПРОТОТИП «КОЛОБОК»

«... Я от дедушки ушел,
Я от бабушки ушел,
От тебя, зайца,
Не хитро уйти ...»

С детства помним этот стишок. Пусть наш колобок катится по окружающей среде и преодолевает различные трудности.

9.1. СОЗДАНИЕ ИГРОВОГО ОБЪЕКТА

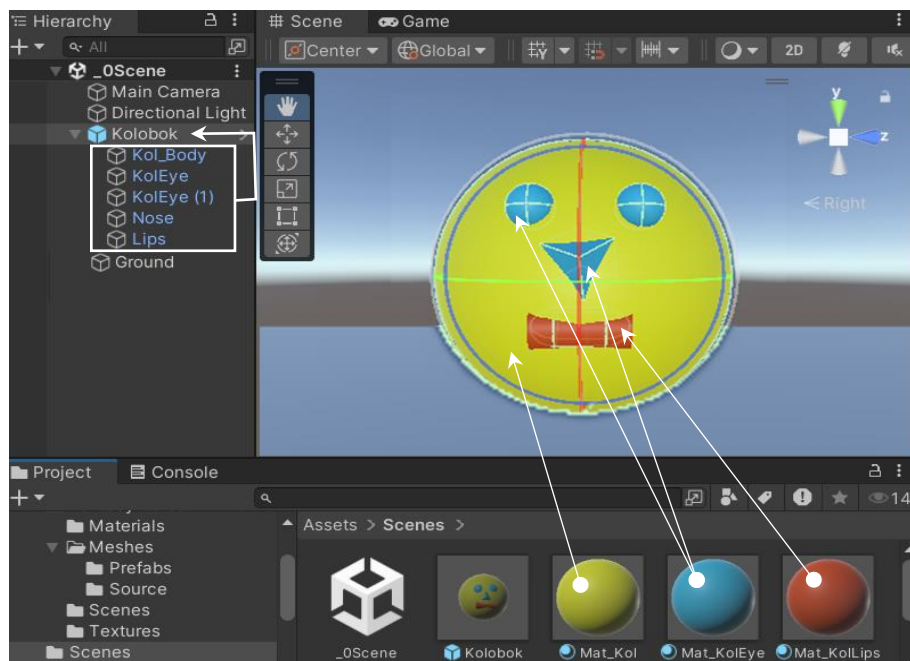
Для начала создадим главного героя. Это несложно – три сферы разного радиуса будут соответственно телом колобка Kol_Body и глазами KolEye, KolEye (1). Нос (Nose) – вершина кубика, губы (Lips) – видимая часть цилиндра. Мы умышленно используем только стандартные примитивы.

Материалы Mat_Kol, Mat_KolEye и Mat_KolLips перетаскиваем на эти объекты.

Глаза, нос и губы у колобка как ориентиры нужны для того, чтобы при вращении были видны изменения.

Как обычно в пустой объект помещаем эти объекты и назовём его Kolobok.

Колобок должен катиться по земле. Как и ранее создадим объект Plane и дадим ему имя Ground.

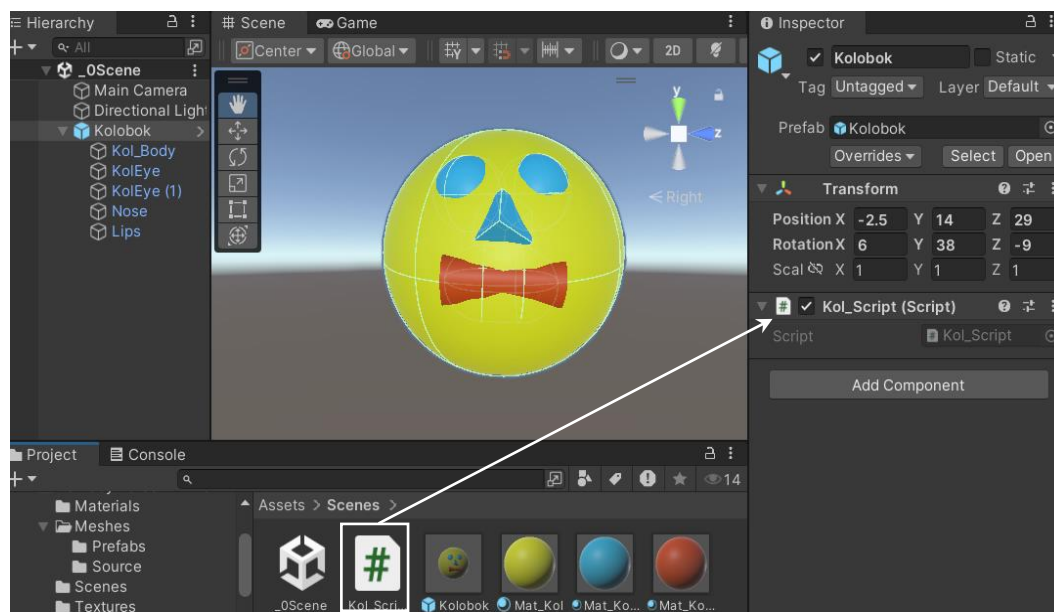


Для наших объектов в инспекторе можно установить, например, такие параметры.

Объект	Transform	X	Y	Z
Ground	Position	0	0	0
	Rotation	0	0	0
	Scale	3	1	1
Main Camera	Position	30	5	-30
	Rotation	8	-44	0
	Scale	1	1	11
Kolobok	Position	0	0	0
	Rotation	-13	-15	4
	Scale	1	1	1
Kol_Body	Position	0	0	0
	Rotation	0	0	0
	Scale	4	4	4
KolEye	Position	0.5	0.8	1.7
	Rotation	0	0	0
	Scale	0.5	0.5	0.5
KolEye (1)	Position	-0.6	0.8	1.7
	Rotation	0	0	0
	Scale	0.5	0.5	0.5
Nose	Position	0	0.2	1.8
	Rotation	-60	7	-145
	Scale	0.5	0.5	0.5
Lips	Position	0	-0.5	1.7
	Rotation	45	-1	90
	Scale	0.5	0.5	0.5

9.2. ВРАЩЕНИЕ И ДВИЖЕНИЕ ИГРОВОГО ОБЪЕКТА

Для того, чтобы колобок катился и вращался, создадим скрипт `Kol_Script.cs` и поместим его на объект `Kolobok`.



В предыдущем проекте наблюдатель у нас вращался вокруг оси `OY`. В нашем случае пусть будет вращение колобка вокруг оси `OX`.

Код скрипта `Kol_Script.cs` описывающий вращение и перемещения колобка, будет выглядеть следующим образом:

```
using UnityEngine;
public class Kol_Script : MonoBehaviour
{
    float speed = 5;          // Определяет силу воздействия на колобка
    float degrSecX = 100,     // Скорость вращения (Градусов в секунду)
        degrSecX_;           // Вспомогательная переменная
    // Переменная для получения ссылки на физическую модель колобка
    private Rigidbody rb;
    void Start() {
        rb = GetComponent<Rigidbody>();
        degrPerSecX_ = degrPerSecX;
    }
    private void Update() {
        if (Input.GetKeyDown(KeyCode.LeftArrow)) {
            // Нажимаем [ -> ] Изменение направления вращения колобка
            degrSecX_ = degrSecX;
        }
        if (Input.GetKeyDown(KeyCode.LeftArrow)) {
            // Нажимаем [ <- ] Изменение направления вращения колобка
            degrSecX_ = -degrSecX;
        }
        transform.Rotate(new Vector3(degrSecX_, 0, 0)*Time.deltaTime);
    }
}
```

```

        // Считываем оси движения
        float moveX = Input.GetAxis("Horizontal");
        float moveZ = Input.GetAxis("Vertical");
        // Направление воздействия силы
        Vector3 movement = new Vector3(moveX, 0, moveZ);
        // Применение силы в заданном направлении
        rb.AddForce(movement * speed);

        if (Input.GetKeyDown(KeyCode.Escape)) // Выход - Esc
        {
            //Application.Quit();
            UnityEditor.EditorApplication.isPlaying = false;
        }
    }
}

```

Скриншот листинга.

using UnityEngine;

public class Kol_Script : MonoBehaviour

```

{
    float speed = 5; // Определяет силу воздействия на коlobка
    float degrPerSecX = 100, // Скорость вращения (Градусов в секунду)
        degrPerSecX_; // Вспомогательная переменная
    // Переменная для получения ссылки на физическую модель коlobка
    private Rigidbody rb;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
        degrPerSecX_ = degrPerSecX;
    }

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.LeftArrow))
        {
            // Нажимаем [ -> ] Изменение направления вращения коlobка
            degrPerSecX_ = degrPerSecX;
        }
        if (Input.GetKeyDown(KeyCode.LeftArrow))
        {
            // Нажимаем [ <- ] Изменение направления вращения коlobка
            degrPerSecX_ = -degrPerSecX;
        }
        transform.Rotate(new Vector3(degrPerSecX_, 0, 0) * Time.deltaTime);

        // Считываем оси движения
        float moveX = Input.GetAxis("Horizontal");
        float moveZ = Input.GetAxis("Vertical");

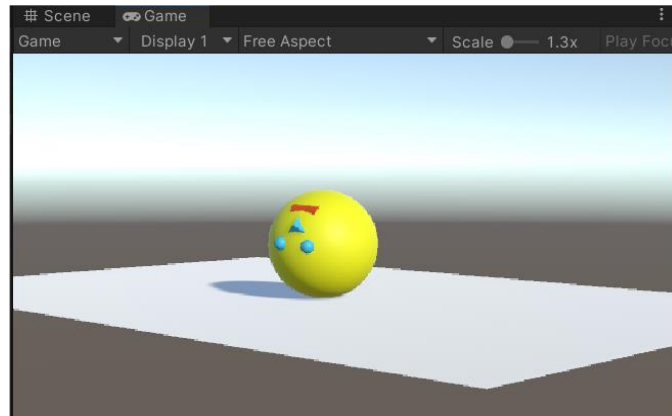
        // Направление воздействия силы
        Vector3 movement = new Vector3(moveX, 0, moveZ);

        // Применение силы в заданном направлении
        rb.AddForce(movement * speed);

        if (Input.GetKeyDown(KeyCode.Escape)) // Выход - Esc
        {
            //Application.Quit();
            UnityEditor.EditorApplication.isPlaying = false;
        }
    }
}

```

В этом случае выглядеть всё будет так.



Изменение направления вращения коlobка можно обеспечить и с использованием мыши. В приведённом ниже коде выделены те строки, которые следует добавить.

```
using UnityEngine;

public class Kol_Script : MonoBehaviour
{
    float speed = 5;          // Определяет силу воздействия на коlobка
    float degrSecX = 100,     // Скорость вращения (Градусов в секунду)
    degrSecX_;               // Вспомогательная переменная

    float h_Speed_Mouse = 10; // Горизонтальная скорость вращения
    float v_Speed_Mouse = 10; // Вертикальная скорость вращения

    // Переменная для получения ссылки на физическую модель коlobка
    private Rigidbody rb;

    void Start() {
        rb = GetComponent<Rigidbody>();
        degrPerSecX_ = degrPerSecX;
    }
    private void Update() {
        if (Input.GetKeyDown(KeyCode.LeftArrow)) {...}
        if (Input.GetKeyDown(KeyCode.LeftArrow)) {...}
        transform.Rotate(new Vector3(degrSecX_, 0, 0) * Time.deltaTime);

        // Изменение вращения коlobка с использованием мыши
        float h_Mouse = h_Speed_Mouse * Input.GetAxis("Mouse X");
        float v_Mouse = v_Speed_Mouse * Input.GetAxis("Mouse Y");
        transform.Rotate(v_Mouse, h_Mouse, 0);

        // Считываем оси движения
        float moveX = Input.GetAxis("Horizontal");
        float moveZ = Input.GetAxis("Vertical");

        // Направление воздействия силы
        Vector3 movement = new Vector3(moveX, 0, moveZ);
    }
}
```

```

        // Применение силы в заданном направлении
        rb.AddForce(movement * move_Speed);

        if (Input.GetKeyDown(KeyCode.Escape)) {    // Выход - Esc
            //Application.Quit();
            UnityEditor.EditorApplication.isPlaying = false;
        }
    }
}

```

Скриншот листинга.

```

using UnityEngine;

public class Kol_Script : MonoBehaviour
{
    float move_Speed = 5; // Определяет силу воздействия на коlobка при перемещении
    float degrPerSecX = 100, // Скорость вращения (Градусов в секунду)
    degrPerSecX_ = 0; // Вспомогательная переменная
    float h_Speed_Mouse = 10; // Горизонтальная скорость вращения
    float v_Speed_Mouse = 10; // Вертикальная скорость вращения

    // Переменная для получения ссылки на физическую модель коlobка
    private Rigidbody rb;

    void Start() {
        rb = GetComponent<Rigidbody>();
        degrPerSecX_ = degrPerSecX;
    }

    private void Update() {
        if (Input.GetKeyDown(KeyCode.LeftArrow)) ...
        if (Input.GetKeyDown(KeyCode.RightArrow)) ...
        transform.Rotate(new Vector3(degrPerSecX_, 0, 0) * Time.deltaTime);

        // Изменение вращения коlobка с использованием мыши
        float h_Mouse = h_Speed_Mouse * Input.GetAxis("Mouse X");
        float v_Mouse = v_Speed_Mouse * Input.GetAxis("Mouse Y");
        transform.Rotate(v_Mouse, h_Mouse, 0);

        // Считываем оси движения
        float moveX = Input.GetAxis("Horizontal");
        float moveZ = Input.GetAxis("Vertical");

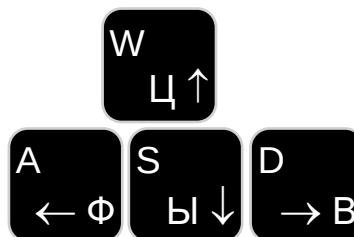
        // Направление воздействия силы
        Vector3 movement = new Vector3(moveX, 0, moveZ);

        // Применение силы в заданном направлении
        rb.AddForce(movement * move_Speed);

        if (Input.GetKeyDown(KeyCode.Escape)) ...
    }
}

```

Обращаем внимание на то, что наряду со стрелками можно использовать и буквенные клавиши для достижения того же эффекта.



9.2.1. Комментарии

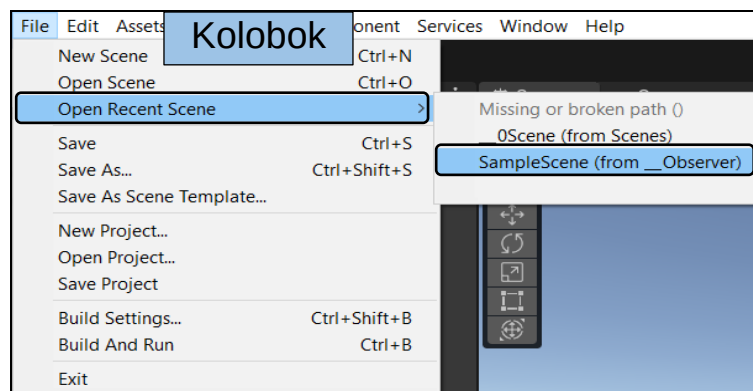
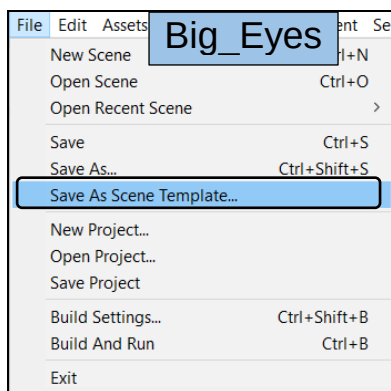
В скрипт-файле `Kol_Script.cs` переменные имеют следующий смысл:

- `speed` – определяет силу воздействия на колобка;
- `rb` – переменная для получения ссылки на физическую модель колобка;
- `degrPerSecX` – скорость вращения (градусов в секунду);
- `degrPerSecX_ = -degrPerSecX` – вспомогательная переменная для изменения направления вращения в зависимости от того, какая нажата стрелка – влево или вправо;
- `Rotate...(degrPerSecX_, 0, 0) ...` – обеспечивает вращение;
- `moveX = Input.GetAxis("Horizontal");`
`moveZ = Input.GetAxis("Vertical");` – определяются оси движения: вдоль OX или OZ соответственно;
- `rb.AddForce(movement * speed)` – применение силы в заданном направлении.
- `h_Speed_Mouse` – горизонтальная скорость вращения;
- `v_Speed_Mouse` – вертикальная скорость вращения;
- `h_Mouse=h_Speed_Mouse*Input.GetAxis("Mouse X")`
`v_Mouse=v_Speed_Mouse*Input.GetAxis("Mouse Y")` – изменение направления (горизонтального и вертикального) вращения колобка с использованием мыши;
- `transform.Rotate(v_Mouse, h_Mouse, 0);` – изменение вращения колобка.

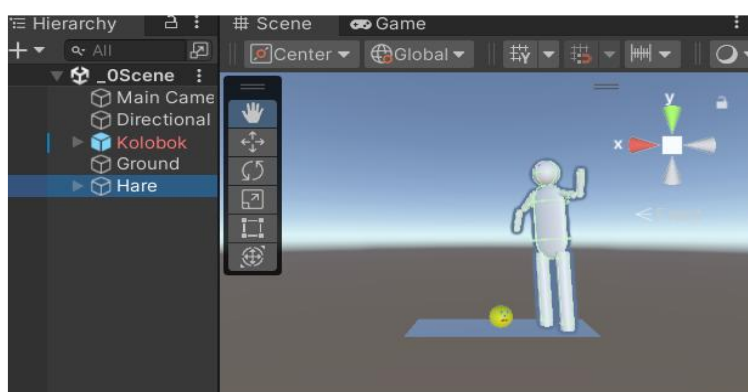
9.3. ПЕРСОНАЖ «ЗАЯЦ»

Наш колобок будет передвигаться по пересечённой местности и встречаться с различными жизненными трудностями (заяц, волк, медведь, лиса). В целях экономии производственных ресурсов ограничимся зайчишкой. Остальных персонажей и развитие событий вы без труда (ну не то, чтобы совсем без труда...) сможете доработать самостоятельно.

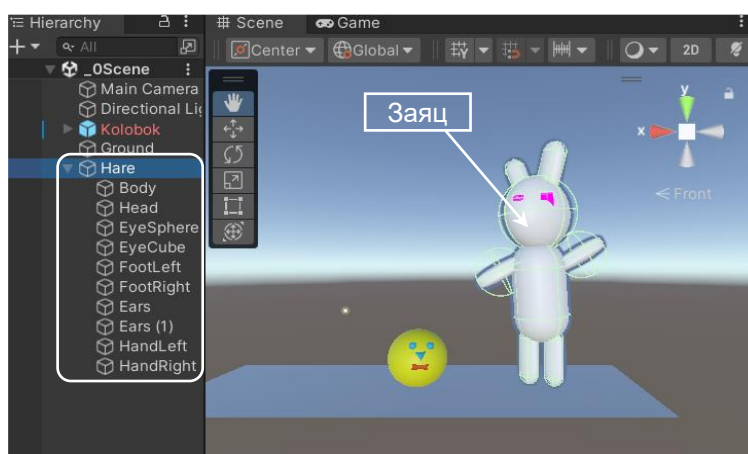
Можно воспользоваться объектом `Observer` (Наблюдатель) из предыдущего проекта. В проекте `Big_Eyes` выполняем `Save As Scene Template...` В нашем активном проекте `Kolobok` открываем сохраненную сцену.



Объект Observer переименуем в Hare (заяц). Пока получилось так.



Следующий шаг – из наблюдателя сделать зайчика. Мы, конечно, не художники или дизайнеры, главное обозначить узнаваемые уши. Для программиста важно выделить особые приметы, и в крайнем случае подписать – «Это заяц».

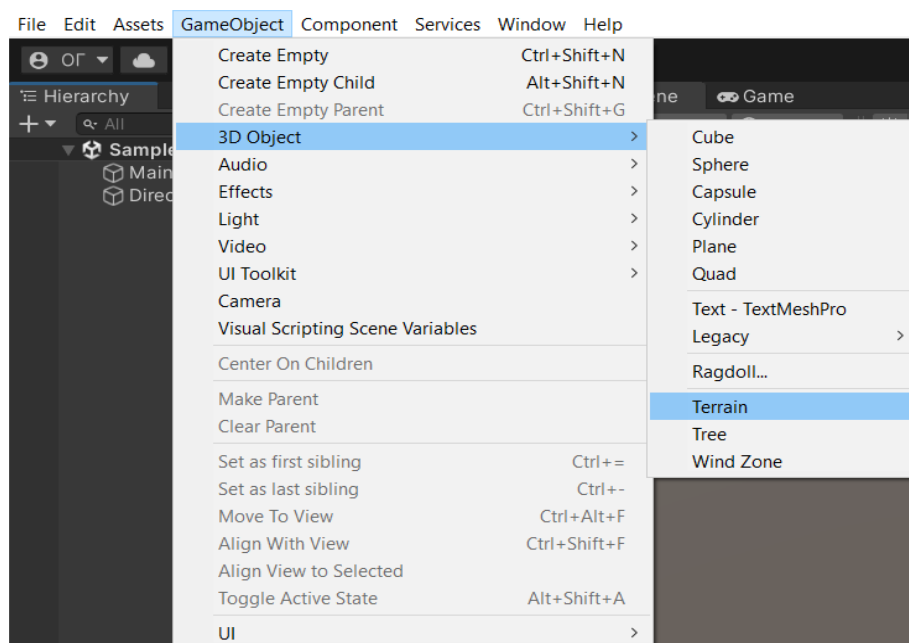


9.4. СОЗДАНИЕ ЛАНДШАФТА

Идея этой игры проста: колобок начинает движение в одной части некоторой области и передвигается в другую часть. На пути будут у него будут холмы, деревья и ... заяц.

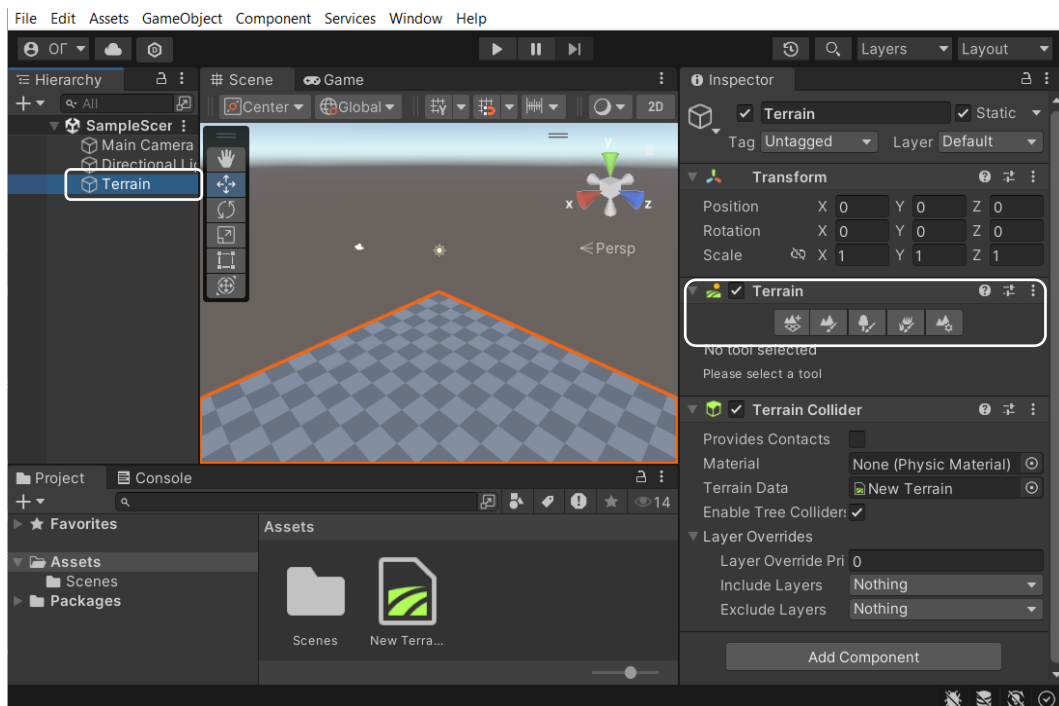
Для начала создадим окружающий ландшафт. В раскрывающемся списке верхнего меню выберите: GameObject → 3D Object → Terrain.

Это добавит в сцену новый объект Terrain.



Создание ландшафта также добавляет соответствующий ресурс Terrain в окно проекта.

Ваш ландшафт будет представлять собой большую плоскость. Когда в окне иерархии выбран игровой объект Terrain, в инспекторе появятся инструменты Terrain.



Панель инструментов с пятью значками в компоненте Terrain в инспекторе предоставляет опции для создания соседних ландшафтов, элементов ландшафта и нанесения текстурных карт на ландшафт.

9.4.1. Настройки рельефа местности

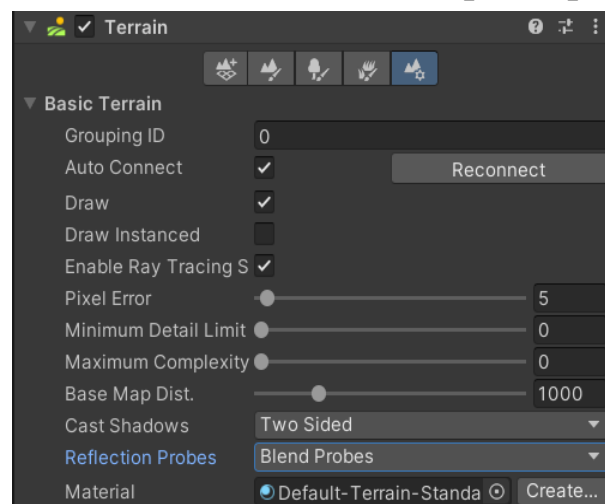


– **Terrain Settings** (Настройки рельефа местности)

Свойства делятся на разделы: Base Terrain, Tree & Detail Objects, Wind Settings for Grass, Resolution, Heightmap.

Внутри секции **Basic Terrain** можно определить следующие параметры.

Опция Draw включает или выключает рендеринг ландшафта.



Pixel Error представляет значение точности преобразования между картами. Более высокие значения представляют меньшую точность, а меньшие значения будут приводить к излишней обработке. Нужно сбалансировать это значение, чтобы создать хорошее, насыщенное окружение.

Base Map dist – максимальное расстояние, на котором текстуры ландшафта будут отображаться с полным разрешением.

Можно включить или отключить тени, выбрав Cast Shadows.

В пункте Material устанавливается материал, который будет использоваться для рендеринга ландшафта. Это повлияет на то, как интерпретируются цветовые каналы текстуры ландшафта. Здесь можно выбрать:

Built In Standard представляет материал, основанный на физическом рендеринге, который был введен в Unity 5. Если выбран этот параметр, для каждого слоя наложения можно использовать одну текстуру для альбедо и сглаживания, одну текстуру для нормали и одно скалярное значение для сдвига металла.

Built In Legacy Diffuse представляет собой устаревший встроенный материал ландшафта из предыдущих релизов.

Built In Legacy Specular использует модель освещения Blinn-Phong. Этот параметр позволяет задать общий цвет отражения и блеск для ландшафта.

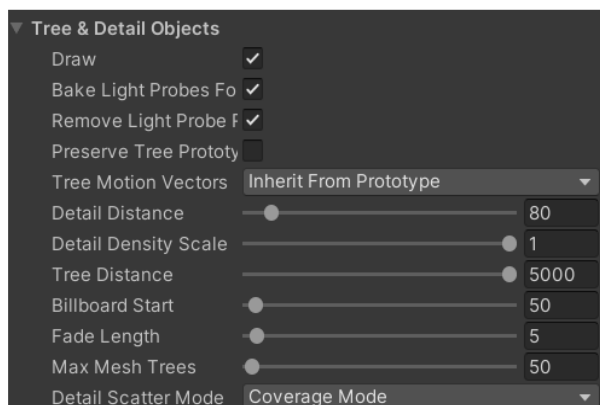
Опция Custom использует настраиваемый материал для ландшафта.

Можно также установить Reflection Probes (оптические датчики) для некоторых материалов, только при использовании встроенного стандартного материала или пользовательского материала, который поддерживает рендеринг с отражением.

Simple: Reflection probes включены, но между датчиками не происходит смешивания, если имеются два перекрывающихся объема.

Tree & Detail Objects (Tree & Detail Objects)

Опция Draw позволяет настроить прорисовку деталей.



Bake Light Probes создаёт световые датчики в позиции каждого дерева. Этот параметр доступен только для деревьев, на которых установлены световые датчики на их префабах.

Detail Distance – расстояние от камеры, после которого детали будут отсекаются.

Detail Density представляет количество объектов деталей/травы в данной условной единице площади.

Tree Distance представляет собой расстояние от камеры, за которым деревья будут отсекаются.

Billboard Start – расстояние от камеры, начиная с которого 3D-деревья начнут заменяться на спрайтовые изображения.

Fade Length представляет собой расстояние перехода деревьев из 3D-объектов в плоские спрайты.

Max Mesh Trees – максимальное количество видимых деревьев.

Wind Settings for Grass (Настройки ветра для травы)

Speed определяет скорость обдувающего траву ветра.

Size представляет собой размер ряби на травянистых участках.

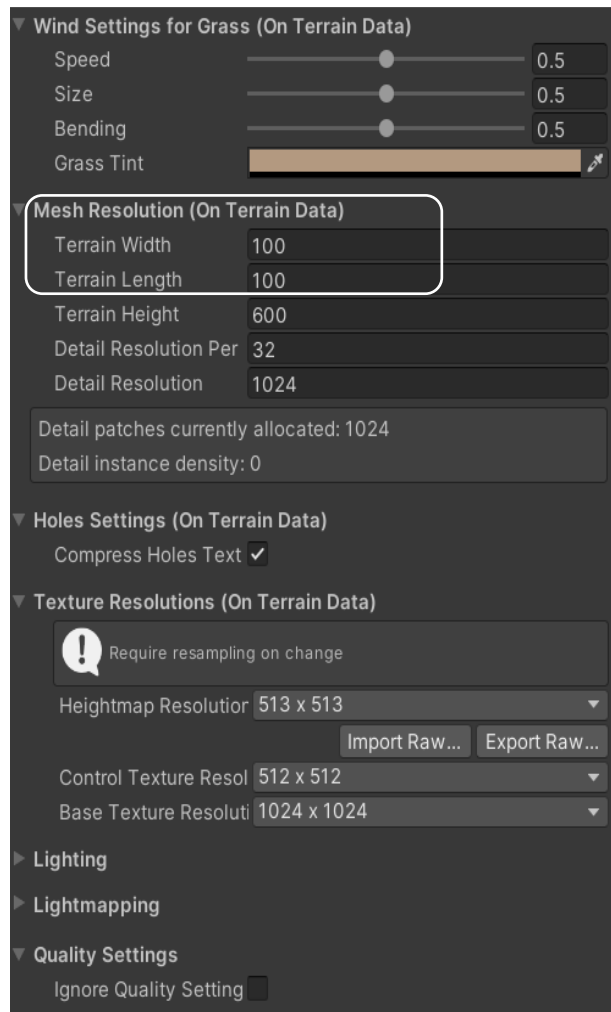
Bending позволяет определить степень, наклона объектов травы под действием ветра.

Grass Tint позволяет выбрать общий оттенок цвета, применяемый к объектам травы.

В разделе Mesh Resolution можно изменять свойства разрешения: Terrain Width, Length, и Height. Изменим значения полей Terrain Width и Terrain Length на 100.

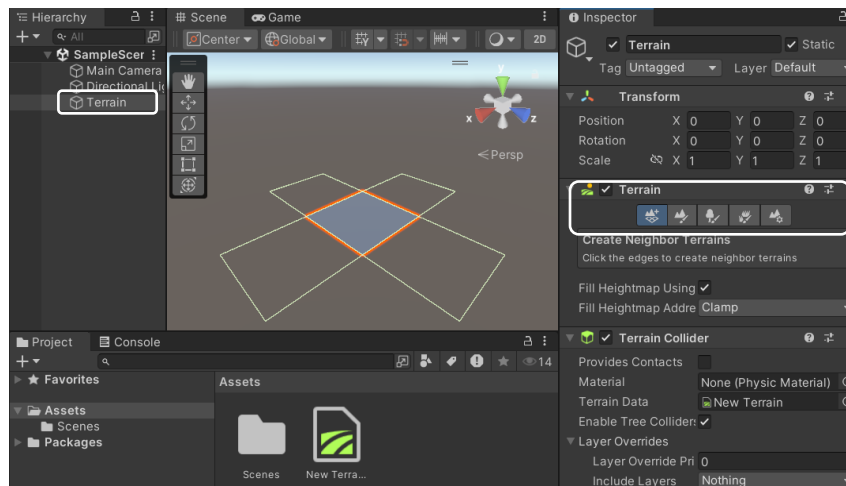
Detail Resolution позволяет определить разрешение карты. Чем выше разрешение, тем меньше и более детальны будут участки травы.

Detail Resolution per Patch устанавливает длину и ширину квадрата островков за один вызов рисования.

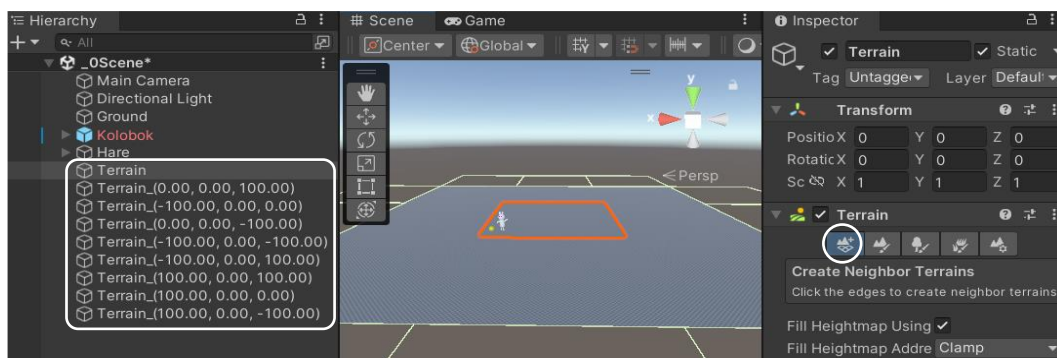


9.4.2. Создание соседних ландшафтов

 — Create Neighbor Terrains (создание соседних ландшафтов). Появляется новая сетка, показывающая 4 квадранта, очерченных к северу, югу, востоку и западу от местности.



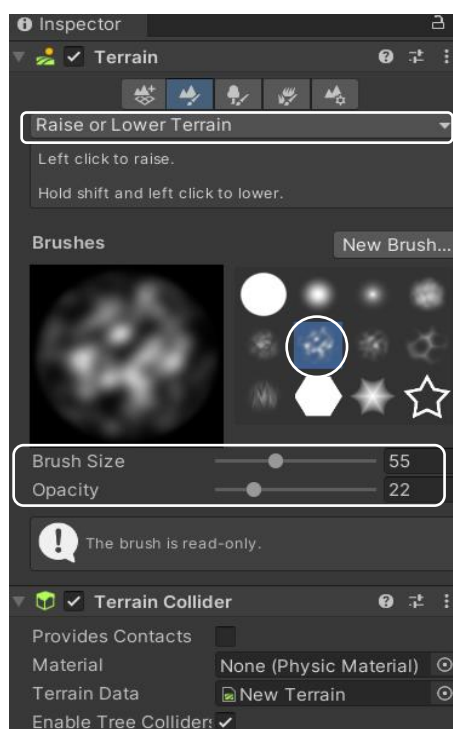
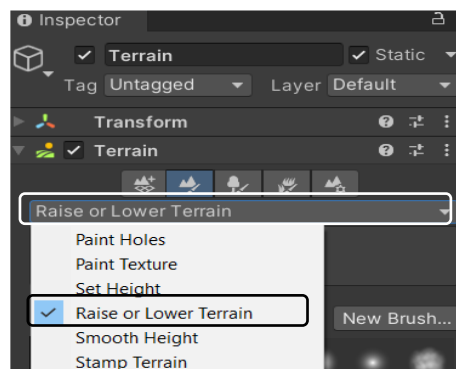
Добавляем области по «периметру» нашей начальной области соседние ландшафты. Обращаем внимание на то, что появились соседние ландшафты со своими координатами.



9.4.3. Создание рельефа

 – Paint Terrain. Для добавления рельефа перейдем в раздел Paint Terrain и в списке выберем пункт Raise or Lower Terrain (поднять или опустить рельеф местности). Ниже можно выбрать кисть, которой будем изменять ландшафт. Обратите внимание, что при изменении типа кисти на плоскости Terrain будут отображаться разные области. Этой кистью можно либо увеличить, либо уменьшить высоту ландшафта.

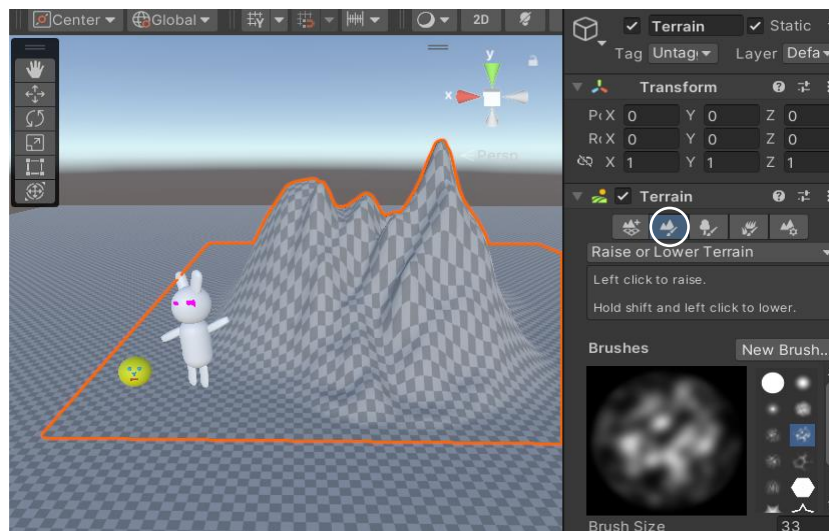
Размер кисти изменяется параметром Brush Size. Параметр Opacity изменяет высоту рельефа.



При помощи одиночного клика левой клавиши на объекте Terrain происходит единоразовое изменение рельефа. Если левую клавишу мыши держать зажатой, то изменение рельефа будет происходить пока клавиша зажата.

С удерживаемой клавишей Shift уменьшается высота рельефа.

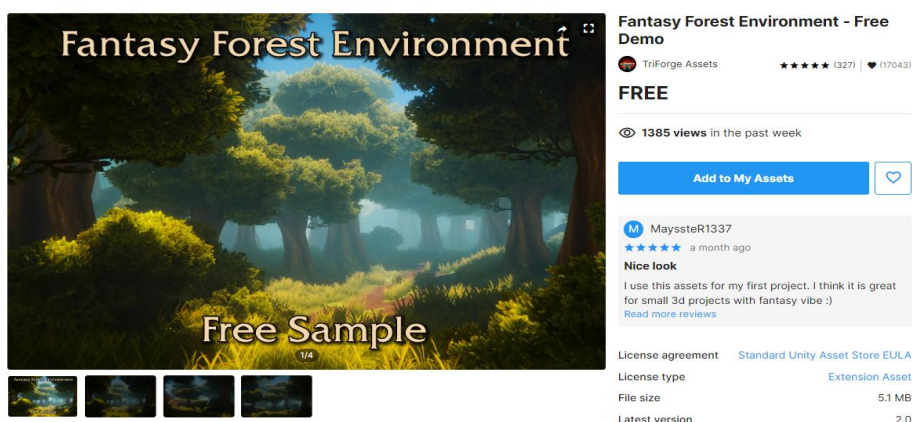
Используя разные кисти, изменяя их размеры и высоту, можно изменить рельеф ландшафта, например, вот так:



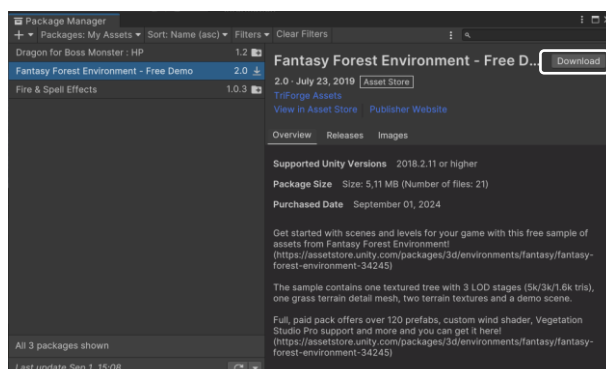
Чтобы наложить на рельеф картинку, нам необходимо импортировать текстуры земли, травы и т.д.

Импортируем вот такой набор:

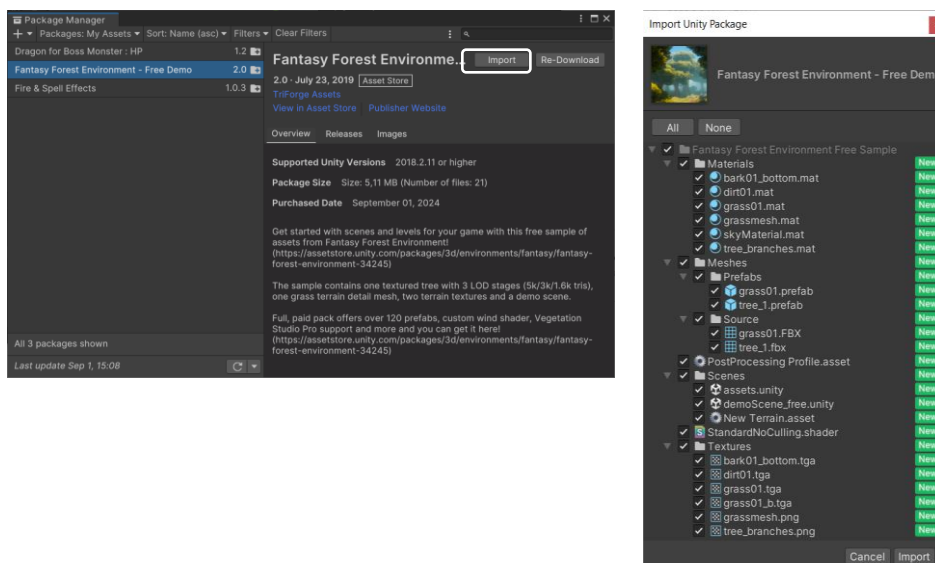
<https://assetstore.unity.com/packages/3d/environments/fantasy/fantasy-forest-environment-free-demo-35361>



Сначала скачиваем этот пакет.



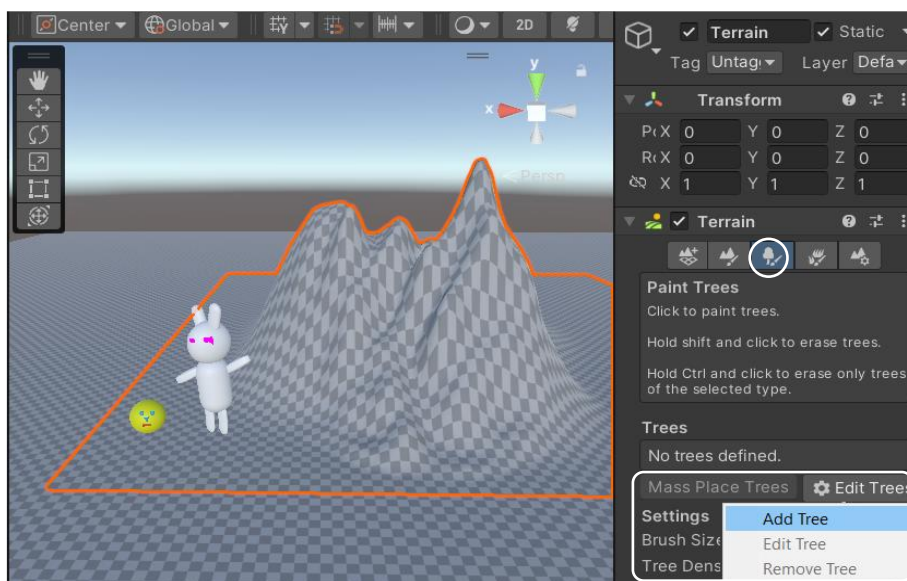
И импортируем его.



9.4.4. Создание объектов деревьев

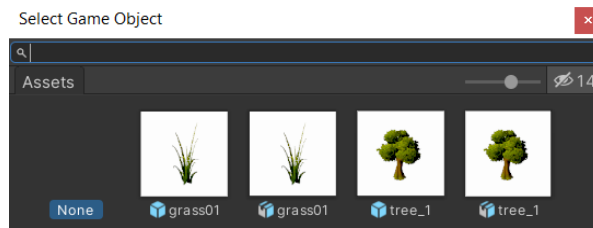


— переходим в раздел Paint Trees, нажимаем кнопку Edit Trees и выбираем в появившемся меню пункт Add Tree:

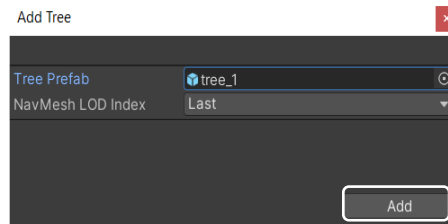


В появившемся окне необходимо кликнуть на точку справа в поле префаба дерева.

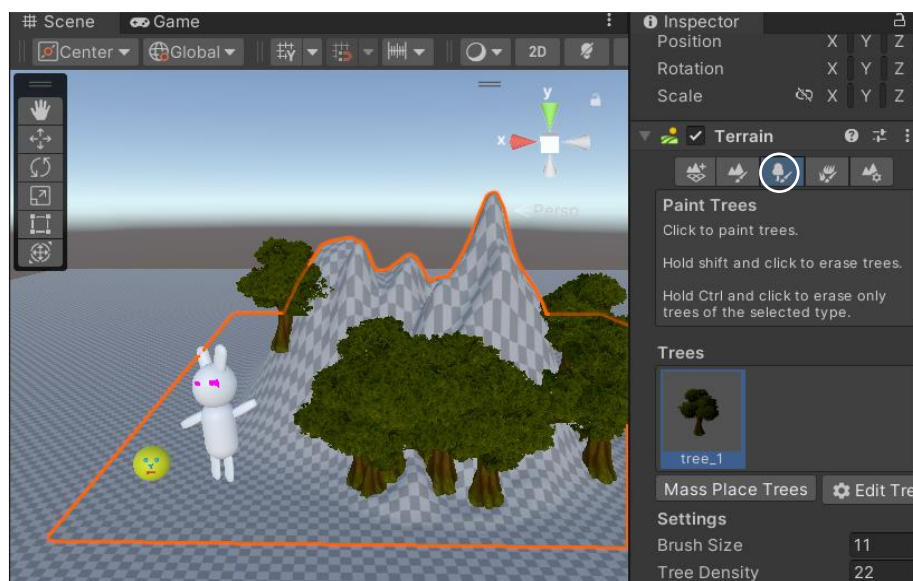
В появившемся окне двойным кликом выбираем понравившийся префаб дерева:



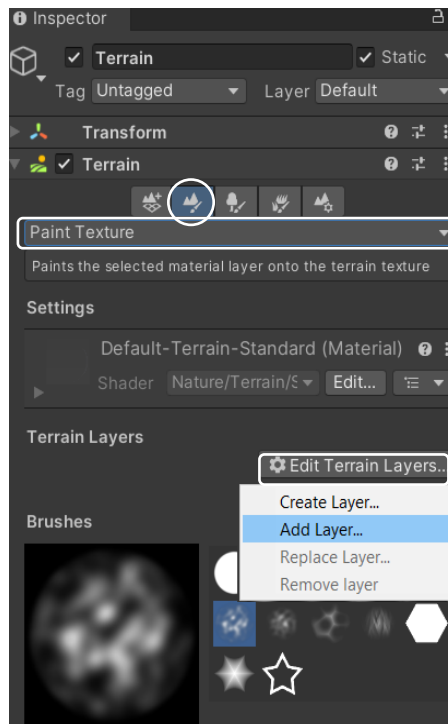
Нажимая кнопку Add, добавляем выбранный префаб в палитру:



Кликаая в нужном месте плоскости, помещаем дерево. Прodelывая эту операцию несколько раз, можно создать лес.



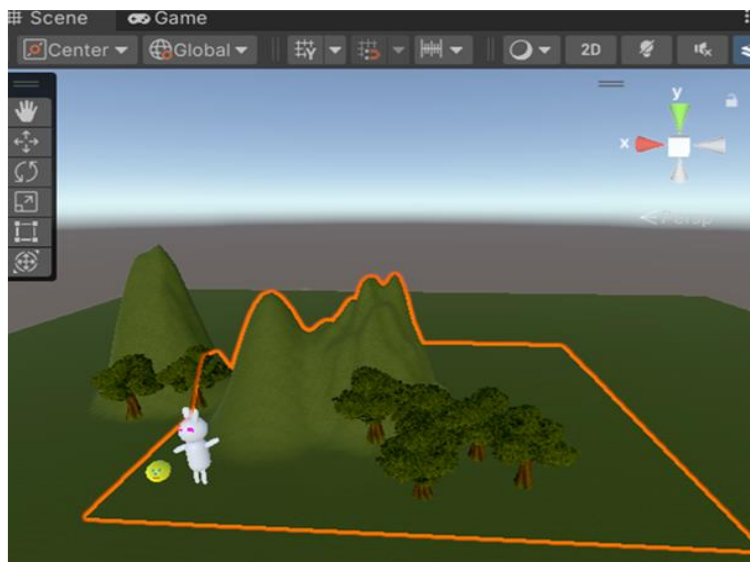
— выбираем режим рисования текстуры Paint Terrain, кликнем по кнопке Edit Terrain Layers и выберем пункт Add Layer...



Появится окно, в котором отображаются все текстуры, импортированные в наш проект. Выбираем нужную текстуру, которая будет применена ко всему слою, например, `layer_dirt01b3407d0e55802f81`

После этого вся плоскость Terrain окрасится выбранной текстурой.





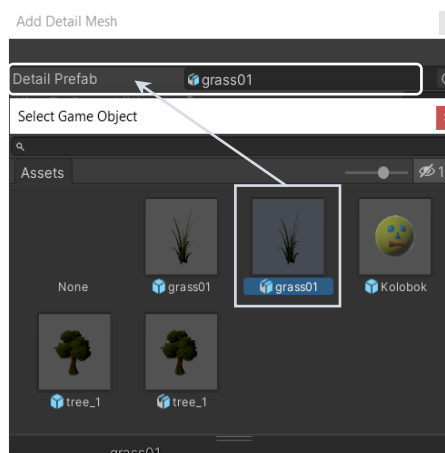
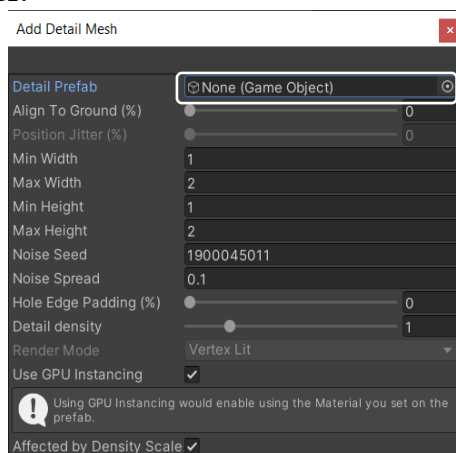
Таким образом первый слой всегда закрашивает весь Terrain, а слои выше накладываются сверху при помощи кисти.

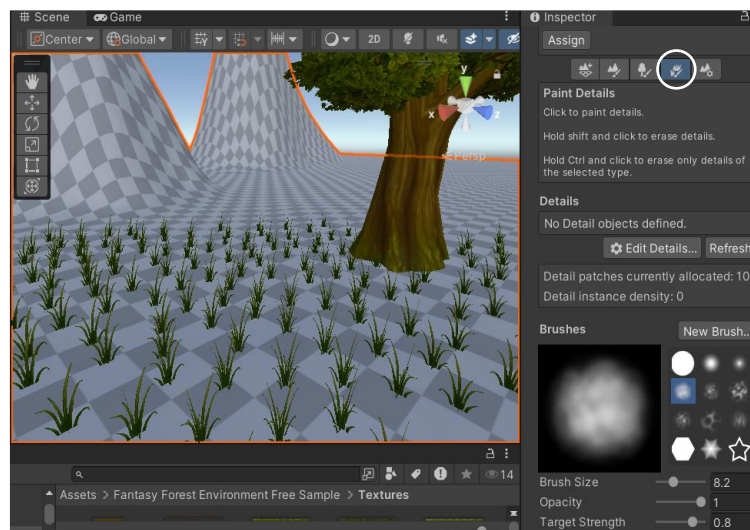
Краткое описание других режимов:

- Paint Holes предназначен для создания разрывов в объекте Terrain;
- Smooth Height для сглаживания высот, когда вам нужно, например, сгладить слишком острую вершину горы;
- Stamp Terrain позволяет использовать выбранную кисть как штамп.

9.4.5. Прорисовка деталей

 – Paint Details. Для отрисовки травы переходим в этот режим и кликаем по кнопке Edit Details. В появившемся меню выбираем Add Detail Mesh, чтобы выбрать форму травы. Теперь в поле Detail Prefab кликаем справа по кружочку для выбора формы травы.





После этого выбираем размер и прозрачность кисти и можно нарисовать траву по одному элементу или сразу выбранной областью.

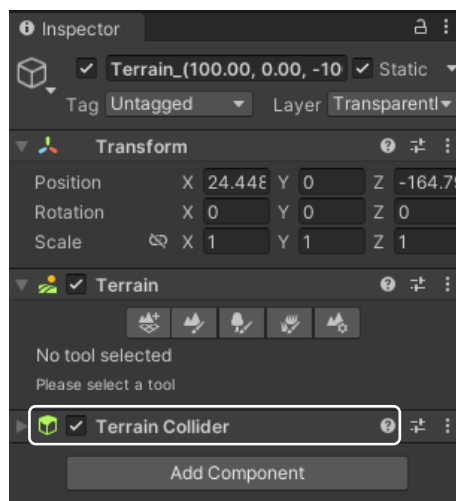
Таким образом в трехмерной игре можно создать свой индивидуальный ландшафт.



9.4.6. Движение персонажа по рельефу

В нашем случае все созданные ландшафты объединены в объект Terrain_All.

Для того чтобы колобок не проваливался сквозь Terrain, следует добавить компонент Terrain Collider. Таким образом, колобок будет не проваливаться, а катиться по рельефу Terrain_All.



9.4.7. Создание тумана

В редакторе Unity можно добавить на сцену туман, чтобы имитировать множество различных природных явлений, таких как дымка, реалистичный туман или пропадание объектов из видимости на больших расстояниях. Можете использовать туман, чтобы придать созданному миру таинственный вид.

Добавить туман можно следующим образом:

1. Выбрать пункт меню Window → Lighting → Settings. Откроется диалоговое окно Lighting.

2. Включить туман, установив флажок Fog в группе параметров Other Settings.

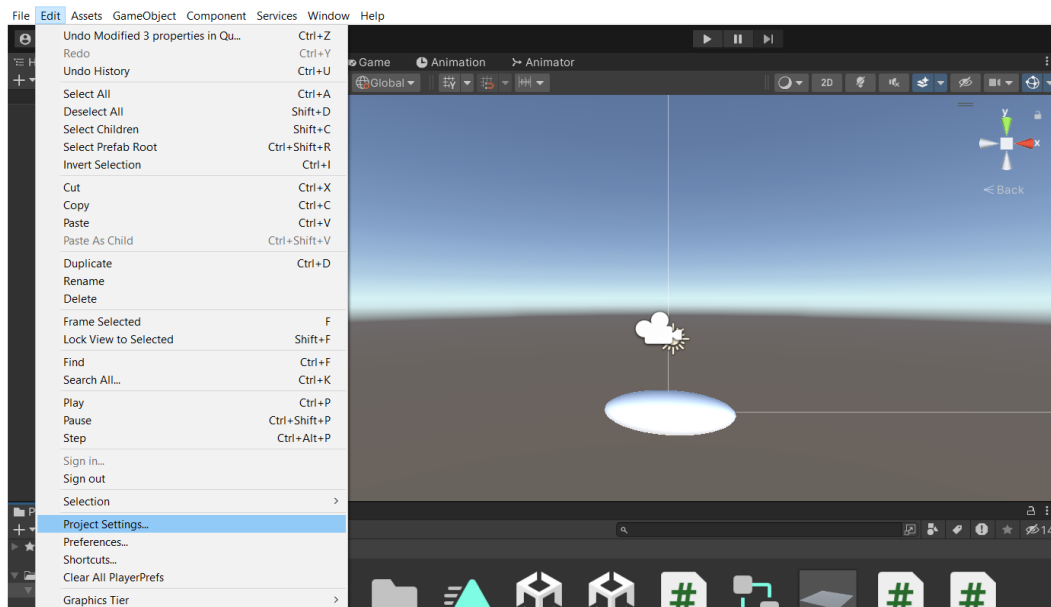
3. Изменить цвет на белый и присвоить параметру Density (плотность) значение 0,005.

Ниже приведены различные свойства тумана.

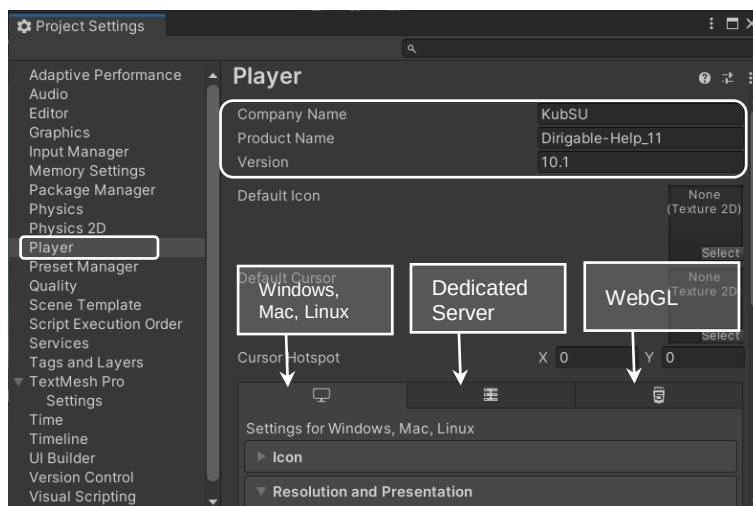
Параметр	Описание
Fog Color	Задаёт цвет тумана
Fog Mode	Этот режим определяет, как рассчитывается туман. Есть три режима: Linear (линейный), Exponential (экспоненциальный) и Exponential Squarer (экспоненциальный в квадрате). Для мобильных устройств лучше работает режим Linear.
Density	Определяет, насколько силен эффект тумана. Это свойство используется только в случае, если задан режим тумана Exponential или Exponential Squared.
Start и End	Определяет, на каком расстоянии от камеры начинается и заканчивается отображение тумана. Эти свойства работают только в режиме Linear.

10. НАСТРОЙКА И СБОРКА ПРОЕКТА

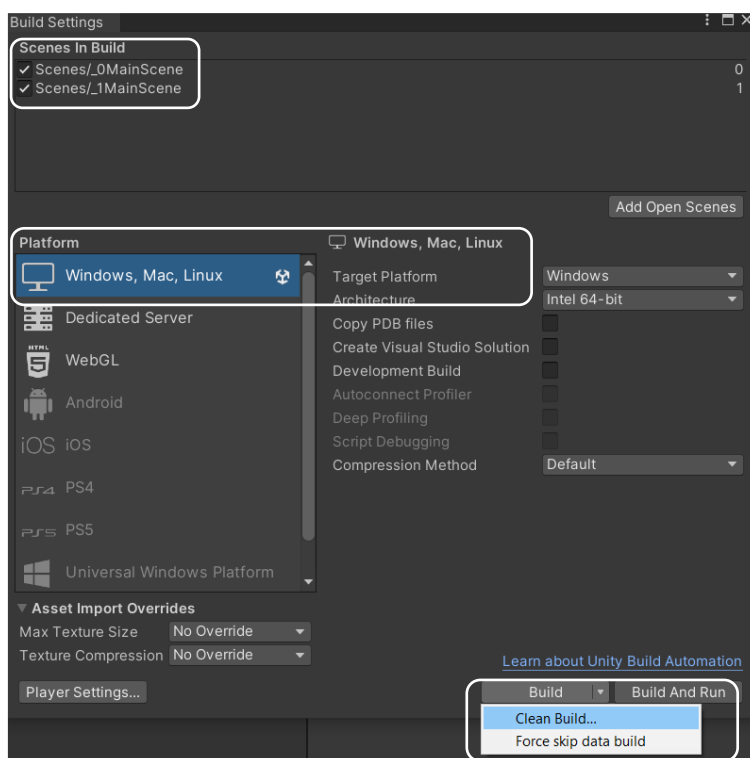
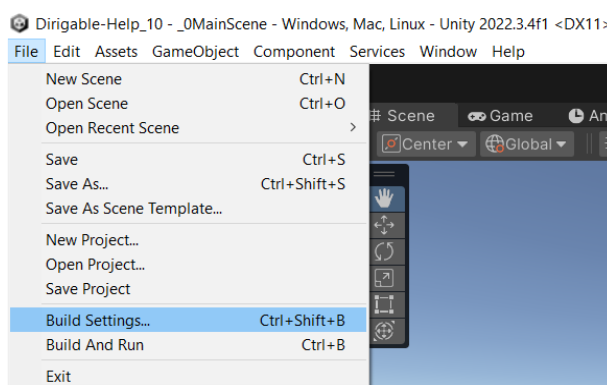
Настройки проекта можно изменить из главного меню Edit → Project Settings...



Из множества настроек выделим пункт Player, где можно указать имя приложения, организацию, операционную систему и т.п. (прил. 2).



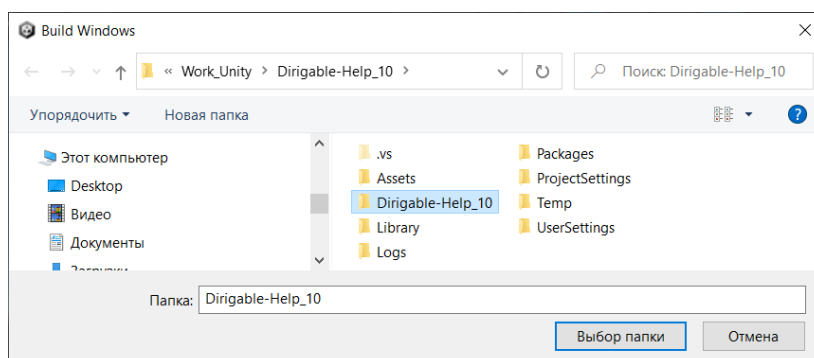
Для создания приложения в главном меню следует изменить настройки проекта в меню File → Build Settings... (Ctrl+Shift+B).



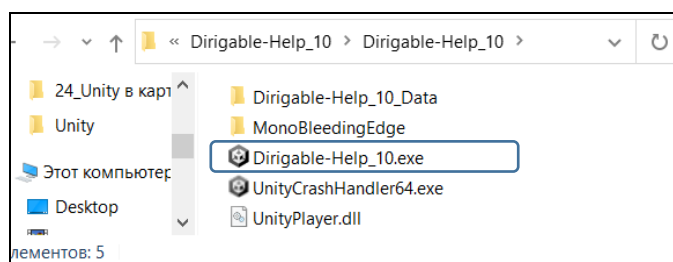
Запустить построение приложения (Ctrl+B).

Построить приложение можно и из меню File → Build And Run... (Ctrl+B).

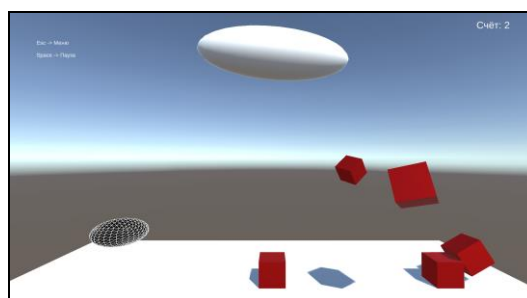
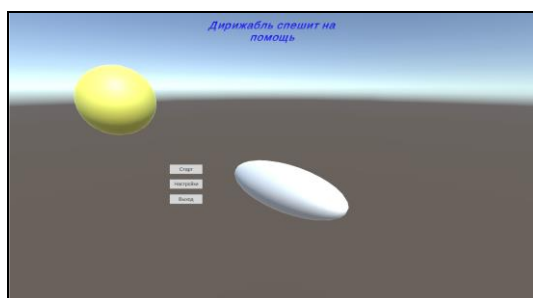
Предварительно нужно создать папку для всех создаваемых файлов, например, Dirigable-Help_10.



В этой папке будут все файлы, необходимые для работы приложения Dirigable-Help_10.exe.

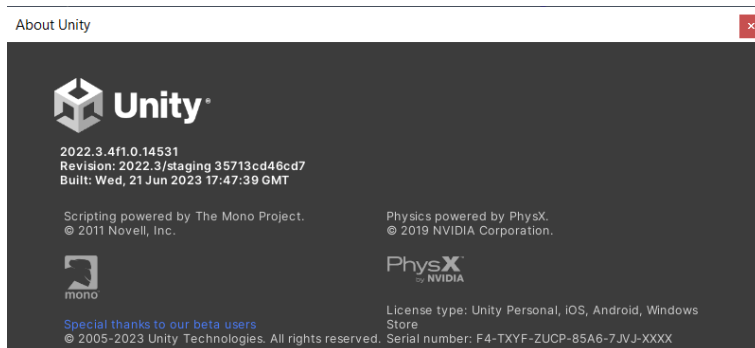


Произойдет построение и запуск на выполнение нашего проекта.



11. ИСПОЛЬЗОВАННОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

При разработке проекта были использованы приложения Unity и Visual Studio.

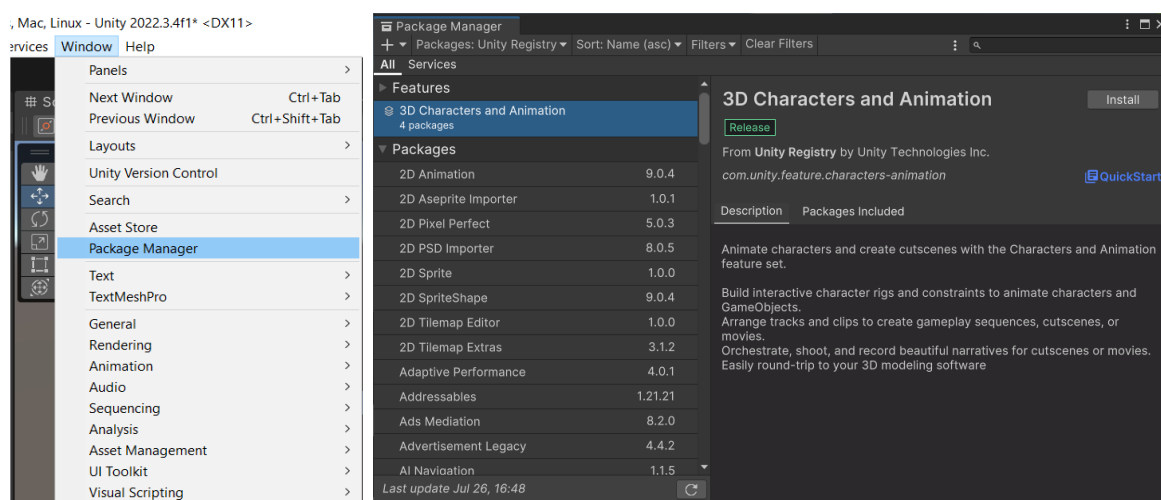


Visual Studio		Состояние лицензии Условия лицензии
Microsoft Visual Studio Community 2022 (64-разрядная версия) - Current Версия 17.11 © Корпорация Майкрософт (Microsoft Corporation), 2022 г. Все права защищены.	Microsoft .NET Framework Версия 4.8.04084 © Корпорация Майкрософт (Microsoft Corporation), 2022 г. Все права защищены.	

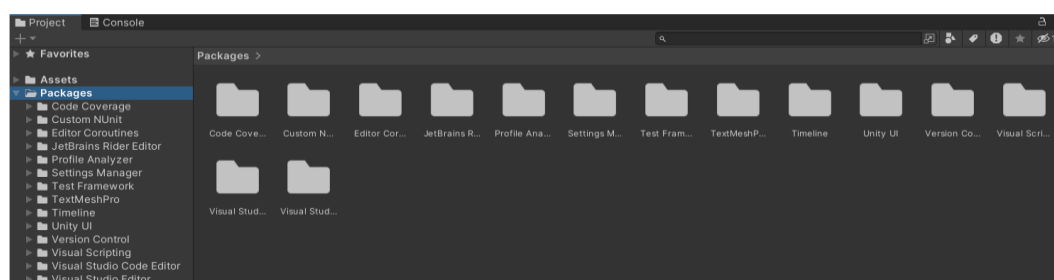
12. МЕНЕДЖЕР ПАКЕТОВ UNITY

Менеджер пакетов Unity – это официальная система управления пакетами для Unity. Этот менеджер позволяет Unity быстро и легко распространять новые функции и обновлять существующие, предоставляет пользователям платформу для поиска повторно используемых компонентов и обмена ими.

Если пакеты Packages не установлены, то с использованием Package Manager его можно установить так:



Пакет содержит функции, соответствующие различным потребностям вашего проекта. Сюда могут входить любые основные функции Unity, установленные вместе с редактором, или другие пакеты, которые вы можете установить по мере необходимости.



Информацию об использовании пакетов Unity можно найти по этой ссылке <https://unityhub.ru/manual/PackagesList>

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. Руководство пользователя Unity 2020.2. URL: <https://docs.unity3d.com/ru/2020.2/Manual/UnityManual.html>
2. Гибсон Б. Дж. Unity и C#. Геймдев от идеи до реализации: 2-е изд. СПб., 2019.
3. Онлайн школа Skysmart. URL: <https://skysmart.ru/articles/programming/animaciya-unity-3d>
4. Ларкович С.Н., Куприянова А.В. Привет, UNITY! Моя первая книга по созданию игр. СПб., 2021.
5. Гейг М. Разработка игр на Unity 2018 за 24 часа / пер. с англ. М.А. Райтмана. М., 2020.
6. Ламмерс К. Шейдеры и эффекты в Unity. Книга рецептов / пер. с англ. Е. А. Шапочкина; под ред. В.В. Симонова. М., 2014.
7. Торн А. Основы анимации в Unity / пер. с англ. Р. Рагимова. М., 2016.

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ 1

Здесь приведены свойства, методы и функции некоторых объектов.

Renderer – средства визуализации

<https://unityhub.ru/scripting/Renderer>

Визуализатор – это то, что заставляет объект появляться на экране. Используйте этот класс для доступа к рендереру любого объекта, сетки или системы частиц. Рендереры можно отключить, чтобы сделать объекты невидимыми (см. `enabled`), и через них можно получить доступ к материалам и изменить их (см. `material`).

Свойства	Функции
Mass	Масса объекта (по умолчанию в килограммах).
Drag	Насколько сопротивление воздуха влияет на объект при движении от сил. 0 означает отсутствие сопротивления воздуха, а бесконечность заставляет объект немедленно останавливаться.
allowOcclusionWhenDynamic	Определяет, следует ли выполнять динамическое отсечение окклюзии для этого средства визуализации.
bounds	Ограничивающий объем средства визуализации (только для чтения).
enabled	Делает отображаемый 3D-объект видимым, если включено.
forceRenderingOff	Позволяет отключить визуализацию для определенного компонента.
isPartOfStaticBatch	Указывает, является ли средство визуализации частью статического пакета с другими средствами визуализации.
isVisible	Виден ли этот модуль визуализации на любой камере? (Только чтение)
lightmapIndex	Индекс запеченной карты освещения, примененной к этому рендереру.

Свойства	Функции
lightmapScaleOffset	Масштаб и смещение UV, используемые для карты освещения.
lightProbeProxyVolumeOverride	Если установлено, средство визуализации будет использовать компонент Light Probe Proxy Volume, прикрепленный к исходному игровому объекту.
lightProbeUsage	Тип интерполяции светового зонда.
localToWorldMatrix	Матрица, преобразующая точку из локального пространства в мировое пространство (только для чтения).
material	Возвращает первый созданный материал, назначенный средству визуализации.
materials	Возвращает все экземпляры материалов этого объекта.
motionVectorGenerationMode	Указывает режим визуализации вектора движения.
probeAnchor	Если установлено, Renderer будет использовать положение этого Transform для поиска источника света или отражения.
rayTracingMode	Описывает, как этот модуль визуализации обновляется для трассировки лучей.
realtimeLightmapIndex	Индекс карты освещения в реальном времени, примененной к этому средству визуализации.
realtimeLightmapScaleOffset	Масштаб и смещение UV, используемые для карты освещения в реальном времени.
receiveShadows	Получает ли этот объект тени?
reflectionProbeUsage	Должны ли для этого средства визуализации использоваться датчики отражения?
RendererPriority	Это значение сортирует средства визуализации по приоритету. Меньшие значения отображаются первыми, а более высокие значения отображаются последними.

Свойства	Функции
renderingLayerMask	Определяет, на каком слое рендеринга находится этот модуль рендеринга.
shadowCastingMode	Отбрасывает ли этот объект тени?
sharedMaterial	Общий материал этого объекта.
sharedMaterials	Все общие материалы этого объекта.
sortingLayerID	Уникальный идентификатор слоя сортировки визуализатора.
sortingLayerName	Имя слоя сортировки визуализатора.
sortingOrder	Порядок визуализации в слое сортировки.
worldToLocalMatrix	Матрица, преобразующая точку из мирового пространства в локальное пространство (только для чтения).

*Унаследованные объекты **Renderer***

Свойства	Функции
gameObject	Игровой объект, к которому прикреплен компонент. Компонент всегда привязан к игровому объекту.
tag	Тег игрового объекта.
transform	Transform, прикрепленный игровому объекту.
hideFlags	Должен ли объект быть скрыт, сохранен вместе со сценой или изменен пользователем?
name	Имя объекта

*Публичные методы **Renderer***

Свойства	Функции
BroadcastMessage	Вызывает метод с именем methodName для каждого MonoBehaviour в игровом объекте или любом из его дочерних элементов.
CompareTag	Помечен ли этот игровой объект тегом?
GetComponent	Возвращает компонент, если он прикреплен к игровому объекту, и null, если нет. Также возвращает отключенные компоненты.
GetComponentInChildren	Возвращает компонент дочернего элемента игрового объекта, используя поиск в глубину.

Свойства	Функции
GetComponentInParent	Возвращает компонент игрового объекта в любом из его родителей.
GetComponents	Возвращает все компоненты игрового объекта
GetComponentsInChildren	Возвращает все компоненты дочерних элементов игрового объекта. Работает рекурсивно.
GetComponentsInParent	Возвращает все компоненты игрового объекта любого из родителя.
SendMessage	Вызывает метод с именем methodName для каждого MonoBehaviour в этом игровом объекте.
SendMessageUpwards	Вызывает метод с именем methodName для каждого MonoBehaviour в этом игровом объекте и для каждого behaviour.
TryGetComponent	Получает компонент указанного типа, если он существует.
GetInstanceID	Возвращает идентификатор экземпляра объекта.
ToString	Возвращает имя объекта.

Статические методы Renderer

Свойства	Функции
Destroy	Удаляет игровой объект, компонент или актив.
DestroyImmediate	Уничтожает объект obj. Настоятельно рекомендуется использовать Destroy.
DontDestroyOnLoad	Не уничтожает целевой объект при загрузке новой сцены.
FindObjectOfType	Возвращает первый активный загруженный объект.
FindObjectsOfType	Получает список всех загруженных объектов.
Instantiate	Клонирует исходный объект и возвращает клон.

Material

<https://unityhub.ru/scripting/Material>

Этот класс предоставляет все свойства материала, позволяя анимировать их. Можно использовать его для установки пользовательских свойств шейдера, к которым нельзя получить доступ через инспектор (например, матрицы).

Чтобы получить материал, используемый объектом, примените свойство `Renderer.material`.

Свойства	Функции
<code>color</code>	Основной цвет материала.
<code>doubleSidedGI</code>	Получает и задает, включена ли для этого материала настройка двустороннего глобального освещения.
<code>enableInstancing</code>	Получает и задает, включено ли создание экземпляров графического процессора для этого материала.
<code>globalIlluminationFlags</code>	Определяет, как материал должен взаимодействовать с картами освещения и датчиками освещения.
<code>mainTexture</code>	Основная текстура.
<code>mainTextureOffset</code>	Смещение основной текстуры.
<code>mainTextureScale</code>	Масштаб основной текстуры.
<code>passCount</code>	Сколько проходов в этом материале (только для чтения).
<code>renderQueue</code>	Очередь рендеринга этого материала.
<code>shader</code>	Шейдер, используемый материалом.
<code>shaderKeywords</code>	Массив, содержащий имена ключевых слов локального шейдера, которые в настоящее время включены для этого материала.

Конструктор

```
public Material(Shader shader);
public Material(Material source);
```

Параметр	Функции
<code>shader</code>	Создаёт материал с заданным Shader.
<code>source</code>	Создаёт материал, скопировав все свойства другого материала.

Публичные свойства Material

Свойства	Функции
ComputeCRC	Вычисляет хэш-значение CRC из содержимого материала.
CopyPropertiesFromMaterial	Копирует свойства из другого материала в этот материал.
DisableKeyword	Отключает ключевое слово локального шейдера для этого материала.
EnableKeyword	Включает ключевое слово локального шейдера для этого материала.
FindPass	Возвращает индекс прохода passName.
GetColor	Получает именованное значение цвета.
GetColorArray	Получает именованный массив цветов.
GetFloat	Получает именованное значение с плавающей запятой.
GetFloatArray	Получает именованный массив с плавающей запятой.
GetInt	Получает именованное целочисленное значение.
GetMatrix	Получает именованное значение матрицы из шейдера.
GetMatrixArray	Получает именованный матричный массив.
GetPassName	Возвращает имя прохода шейдера на проходе индекса.
GetShaderPassEnabled	Проверяет, включен ли данный проход шейдера для этого Материала.
GetTag	Получает значение тега шейдера материала.
GetTexture	Получить именованную текстуру.

Свойства	Функции
GetTextureOffset	Получает смещение размещения текстуры propertyName.
GetTexturePropertyNameID	Возвращает идентификаторы имен всех свойств текстуры, представленных в этом материале.
GetTexturePropertyNames	Возвращает имена всех свойств текстуры, представленных в этом материале.
GetTextureScale	Получает масштаб размещения текстуры propertyName.
GetVector	Получить именованное значение вектора.
GetVectorArray	Получить массив именованных векторов.
HasProperty	Проверяет, имеет ли шейдер материала свойство с заданным именем.
IsKeywordEnabled	Проверяет, включено ли ключевое слово локального шейдера для этого материала.
Lerp	Интерполировать свойства двух материалов.
SetBuffer	Устанавливает значение именованного буфера.
SetColor	Устанавливает именованное значение цвета.
SetColorArray	Задаст свойство массива цветов.
SetConstantBuffer	Устанавливает ComputeBuffer или GraphicsBuffer в качестве именованного буфера констант для материала.
SetFloat	Устанавливает именованное значение с плавающей запятой.
SetFloatArray	Задаст свойство массива с плавающей запятой.
SetInt	Устанавливает именованное целочисленное значение.

Свойства	Функции
SetMatrix	Задаёт именованную матрицу для шейдера.
SetMatrixArray	Задаёт свойство матричного массива.
SetOverrideTag	Устанавливает переопределяющий тег/значение для материала.
SetPass	Активировать данный проход для рендеринга.
SetShaderPassEnabled	Включает или отключает передачу шейдера на уровне каждого материала.
SetTexture	Устанавливает именованную текстуру.
SetTextureOffset	Устанавливает смещение размещения текстуры propertyName.
SetTextureScale	Устанавливает масштаб размещения текстуры propertyName.
SetVector	Устанавливает значение именованного вектора.
SetVectorArray	Задаёт свойство векторного массива.

Публичные методы Material

Метод	Функции
GetInstanceID	Возвращает id экземпляра объекта.
ToString	Возвращает имя объекта.

Статические методы Material

Метод	Функции
Destroy	Удаляет игровой объект, компонент или актив.
DestroyImmediate	Уничтожает объект obj. Вместо этого рекомендуется использовать метод Destroy.
DontDestroyOnLoad	Не уничтожает целевой объект при загрузке новой сцены. .

Метод	Функции
FindObjectOfType	Возвращает первый активный загруженный объект.
FindObjectsOfType	Получает список всех загруженных объектов.
Instantiate	Клонирует исходный объект и возвращает клон.

Rigidbody

<https://unityhub.ru/manual/class-Rigidbody>

Rigidbody является основным компонентом, обеспечивающим физическое поведение для GameObject.

Rigidbody может воспринимать силы и крутящий момент, чтобы объекты двигались реалистично. Любой GameObject должен содержать Rigidbody, чтобы на него воздействовала гравитация, чтобы он находился под действием дополнительных сил с помощью сценариев или взаимодействовал с другими объектами через физический движок NVIDIA PhysX.

Свойства	Функции
Mass	Масса объекта (по умолчанию в килограммах).
Drag	Насколько сопротивление воздуха влияет на объект при движении от сил. 0 означает отсутствие сопротивления воздуха, а бесконечность заставляет объект немедленно останавливаться.
Angular Drag	Насколько сопротивление воздуха влияет на объект при вращении от крутящего момента. 0 означает отсутствие сопротивления воздуха. Обратите внимание, что вы не можете остановить вращение объекта, просто установив его угловое перетаскивание на бесконечность.
Use Gravity	Если включено, на объект действует гравитация.
Is Kinematic	Если этот параметр включен, физический движок не будет управлять объектом, и им можно будет манипулировать только с помощью преобразования. Это полезно для движущихся платформ или если вы хотите анимировать Rigidbody, к которому прикреплено HingeJoint.

Свойства	Функции
Interpolate	Попробуйте один из вариантов, только если вы видите рывки в движении вашего Rigidbody.
– None	Интерполяция не применяется.
– Interpolate	Преобразование сглаживается на основе преобразования предыдущего кадра.
– Extrapolate	Преобразование сглаживается на основе предполагаемого преобразования следующего кадра.
Collision Detection	Используется для предотвращения прохождения быстро движущихся объектов через другие объекты без обнаружения столкновений.
– Discrete	Используйте дискретное обнаружение столкновений со всеми другими коллайдерами в Сцене. Другие коллайдеры будут использовать дискретное обнаружение столкновений при проверке на столкновение с ним. Используется для обычных столкновений (это значение по умолчанию).
– Continuous	Используйте дискретное обнаружение столкновений с динамическими коллайдерами (с жестким телом) и основанное на развертке непрерывное обнаружение столкновений против статических коллайдеров (без Rigidbody). Твердые тела, для которых установлено значение Continuous Dynamic, будут использовать непрерывное обнаружение столкновений при тестировании на столкновение с этим твердым телом. Другие твердые тела будут использовать обнаружение дискретных столкновений. Используется для объектов, с которыми необходимо столкнуться при обнаружении Continuous Dynamic. (Это сильно влияет на производительность физики, оставьте значение Discrete, если у вас нет проблем со столкновениями быстрых объектов)

Свойства	Функции
– Continuous Dynamic	Используйте непрерывное обнаружение столкновений на основе развертки для объектов GameObject, для которых задано столкновение Continuous и Continuous Dynamic. Он также будет использовать непрерывное обнаружение столкновений со статическими коллайдерами (без Rigidbody). Для всех других коллайдеров используется дискретное обнаружение столкновений. Используется для быстро движущихся объектов.
– Continuous Speculative	Используйте непрерывное обнаружение столкновений о твердых телах и коллайдерах. Это также единственный режим ПЗС, в котором можно задавать кинематические тела. Этот метод, как правило, дешевле, чем непрерывное обнаружение столкновений на основе развертки.
Constraints	Ограничения на движение твердого тела:
– Freeze Position	Выборочно останавливает твердое тело, перемещающееся по мировым координатам X, Y и Z.
– Freeze Rotation	Выборочно останавливает вращение Rigidbody вокруг локальных осей X, Y и Z.

ПРИЛОЖЕНИЕ 2

Описание параметров сборки

<https://docs.unity3d.com/6000.0/Documentation/Manual/build-profiles-reference.html>

Чтобы создать свое приложение, нужно выбрать один из следующих вариантов:

Настройка	Описание	
Build and Run (Сборка и запуск)	Скомпилируйте плеер и откройте его на вашей целевой платформе. В этом варианте всегда используется инкрементная сборка.	
Build (Сборка)	Скомпилировать плеер. Сборка по умолчанию инкрементная, за исключением первой сборки.	
	Clean build	Создайте чистую, <u>не инкрементную</u> сборку.
	Force skip data build (Принудительно пропускать сборку данных)	Используйте этот параметр для создания изменений кода, но пропустите процесс создания сцен и других ресурсов, не связанных с кодом.

Настройки **сборки** и **сборки и запуска** видны только для активного профиля.

Build data (Данные сборки)

Раздел «Данные сборки» используется для настройки следующих параметров:

Список сцен

При сборке приложения Unity создает все выбранные сцены на панели Scene List (Список сцен). Unity создает сцены в том порядке, в котором они отображаются в списке.

Вы можете добавлять, исключать, удалять и изменять порядок сцен в списке.

- Add (Добавить): Чтобы добавить все открытые в данный момент сцены в список, выберите Add Open Scenes (Добавить открытые сцены). Можно перетаскивать сцены из окна вашего проекта в список.
- Exclude (Исключить): Чтобы исключить сцену из списка из сборки, снимите флажок рядом с этой сценой. При этом сцена удаляется из сборки, но не из списка.

- Remove (Удалить): Чтобы удалить сцену из списка, выберите сцену и нажмите Delete на клавиатуре.
- Adjust the order (Измените порядок): Чтобы изменить порядок сцен, перетащите сцены в другое положение в списке.

Scripting Defines (Определения сценариев)

Этот раздел позволяет добавлять конкретные определения сценариев для каждого профиля сборки.

Platform settings (Настройки платформы)

Список допустимых платформ для сборки:

- Windows, Mac, Linux.
- Dedicated Server.
- WebGL.
- Android.
- iOS.
- tvOS.
- Universal Windows Platform

Shared build settings (Общие настройки сборки)

Следующие настройки сборки доступны для всех типов профилей. Значения этих настроек являются общими для профилей платформы, но не для профилей сборки.

Примечание: При обновлении общих настроек активного профиля платформы с помощью EditorUserBuildSettings применяются изменения ко всем профилям платформы. Однако обновление общих настроек активного профиля сборки с помощью EditorUserBuildSettings обновляет только этот конкретный профиль сборки.

Свойство	Описание
Development Build (Сборка для разработки)	Включите символы отладки сценариев и Profiler в вашей сборке. Используйте этот параметр, когда хотите протестировать свое приложение. При выборе этого параметра Unity устанавливает символ DEVELOPMENT_BUILD определения сценария. Затем ваша сборка включает директивы препроцессора, которые задаются DEVELOPMENT_BUILD в качестве условия.

Свойство	Описание	
Autoconnect Profiler (Профилировщик автоматического подключения)	Автоматическое подключение профилировщика Unity к вашей сборке. Эта опция доступна, только если вы выберете Development Build.	
Deep Profiling (Глубокое профилирование)	Разрешение профилировщику обрабатывать весь код скрипта и записывать каждый вызов функции, возвращая подробные данные профилирования.	
Script Debugging (Отладка скрипта)	Удаленно подключает отладчики скриптов к плейеру.	
Wait for Managed Debugge (Дождитесь запуска управляемого отладчика)	Ожидание плейера подключения отладчика, прежде чем он выполнит какой-либо код скрипта.	
Compression Method (Метод сжатия)	Определяет метод, который Unity использует для сжатия данных в вашем проекте при сборке проигрывателя. Сюда входят Assets, Scenes, Player settings и GI data.	
	Default	В Windows, Mac, Linux и iOS сжатие по умолчанию отсутствует. На Android сжатие по умолчанию – ZIP, что дает несколько лучшие результаты сжатия, чем LZ4HC. Однако распаковка ZIP-данных происходит медленнее.
	LZ4	Формат быстрого сжатия, полезный для сборок разработки.
	LZ4HC	Вариант LZ4 с высоким уровнем сжатия, который медленнее в сборке, но дает лучшие результаты в релизных сборках.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. ЧТО ТАКОЕ ИГРА.....	3
2. СРЕДА РАЗРАБОТКИ UNITY	5
2.1. ИНСТРУМЕНТЫ РАЗРАБОТКИ	5
2.2. НАСТРОЙКА ПРОЕКТА	6
3. СОЗДАНИЕ ПЕРВОГО ПРОЕКТА.....	8
3.1. ПАДАЮЩИЙ КУБ	8
3.2. КУБ, ПАДАЮЩИЙ НА ПЛОСКОСТЬ	11
3.2.1. Принятие пользовательского ввода	16
3.2.2. Комментарии	17
3.3. КОМПОНЕНТЫ GAMEOBJECTS.....	18
3.4. СИСТЕМА СОБЫТИЙ (EVENTSYSTEM)	22
3.4.1. Модули ввода	22
3.4.2. Трассировщики лучей (Raycasters)	23
4. ГРАФИЧЕСКИЙ ИНТЕРФЕЙС ПОЛЬЗОВАТЕЛЯ	23
4.1. КООРДИНАТЫ В GUI UNITY3D.....	24
4.1.1. Мировые координаты.....	24
4.1.2. Система координат экрана.....	24
4.1.3. Система координат графического интерфейса.....	25
4.1.4. Комментарии	27
5. ИГРОВОЙ ПРОТОТИП «ДИРИЖАБЛЬ»	27
5.1. СОЗДАНИЕ ИГРОВОГО ОБЪЕКТА – «ДИРИЖАБЛЯ»	28
5.2. НАСТРОЙКА 3D АНИМАЦИИ	30
5.3. СОЗДАНИЕ ЯЩИКА С ПОДАРКОМ.....	40
5.3.1. Комментарии	44
5.4. НАСТРОЙКА ПОВЕРХНОСТИ.....	44
5.5. НАСТРОЙКА КАМЕРЫ И ИГРОВОЙ ОБЛАСТИ	47
5.5.1. Комментарии	49
6. ПРОГРАММИРОВАНИЕ ИГРОВЫХ ОБЪЕКТОВ.....	50
6.1. СКРИПТ-ФАЙЛ ВОХ ДЛЯ РАЗРУШЕНИЯ «ЯЩИКА»	50
6.2. СОЗДАНИЕ ДВИЖУЩЕЙСЯ «КОРЗИНЫ» BASKET	56

6.2.1. Комментарии	59
6.3. СКРИПТ-ФАЙЛ СЦЕНАРИЯ ИГРЫ	59
6.4. СОЗДАНИЕ СЧЕТЧИКА ОЧКОВ	62
6.5. ДОБАВЛЕНИЕ ОЧКОВ ЗА ПОЙМАННЫЕ ОБЪЕКТЫ	65
6.6. УВЕДОМЛЕНИЕ О ТОМ, ЧТО «ЯЩИК» ВОХ НЕ БЫЛ ПОЙМАН	67
6.7. УНИЧТОЖЕНИЕ BASKET ПОСЛЕ ПОТЕРИ «ЯЩИКА» ВОХ	70
6.7.1. Комментарии	74
7. ДОРАБОТКА СЦЕН И СОЗДАНИЕ ИГРОВОГО МЕНЮ	75
7.1. ВОЗМОЖНАЯ ДОРАБОТКА ЛОКАЦИИ	75
7.2. СОЗДАНИЕ ИГРОВОГО МЕНЮ	78
7.3. ПАУЗА И ВОЗВРАЩЕНИЕ В ГЛАВНОЕ МЕНЮ.....	91
7.4. СОЗДАНИЕ ДВИЖЕНИЯ БЕЗ АНИМАЦИИ	95
7.4.1. Комментарии	99
7.5. ИЗМЕНЕНИЕ МАТЕРИАЛА В СКРИПТЕ	99
7.5.1. Комментарии	101
7.6. ИЗМЕНЕНИЕ SKYVOX В UNITY.....	102
8. ИГРОВОЙ ПРОТОТИП «БОЛЬШОЙ ГЛАЗ».....	108
8.1. СОЗДАНИЕ «ОКРУЖЕНИЯ».....	108
8.2. СОЗДАНИЕ «РОБОТА»	112
8.3. КАМЕРЫ.....	114
8.4. СКРИПТ-ФАЙЛ ROBOTSCRIPT.CS	115
8.4.1. Комментарии	117
8.5. СКРИПТ-ФАЙЛ MOVEMENT.CS.....	117
8.5.1. Комментарии	121
8.6. РЕАЛИСТИЧНЫЕ КООРДИНАТЫ ОБЪЕКТОВ	122
9. ИГРОВОЙ ПРОТОТИП «КОЛОБОК».....	123
9.1. СОЗДАНИЕ ИГРОВОГО ОБЪЕКТА	123
9.2. ВРАЩЕНИЕ И ДВИЖЕНИЕ ИГРОВОГО ОБЪЕКТА	125
9.2.1. Комментарии	129
9.3. ПЕРСОНАЖ «ЗАЯЦ».....	129
9.4. СОЗДАНИЕ ЛАНДШАФТА.....	131

9.4.1. Настройки рельефа местности	132
9.4.2. Создание соседних ландшафтов	136
9.4.3. Создание рельефа.....	137
9.4.4. Создание объектов деревьев.....	139
9.4.5. Прорисовка деталей.....	142
9.4.6. Движение персонажа по рельефу.....	143
9.4.7. Создание тумана	144
10. НАСТРОЙКА И СБОРКА ПРОЕКТА	145
11. ИСПОЛЬЗОВАННОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ.	148
12. МЕНЕДЖЕР ПАКЕТОВ UNITY	149
РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА.....	150
ПРИЛОЖЕНИЕ 1	151
Renderer – средства визуализации	151
Унаследованные объекты Renderer.....	153
Публичные методы Renderer	153
Статические методы Renderer	154
Material.....	154
Конструктор.....	155
Публичные свойства Material	156
Публичные методы Material	158
Статические методы Material.....	158
Rigidbody	159
ПРИЛОЖЕНИЕ 2	162
Описание параметров сборки.....	162
Build data (Данные сборки)	162
Список сцен	162
Scripting Defines (Определения сценариев)	163
Platform settings (Настройки платформы)	163
Shared build settings (Общие настройки сборки)	163

Учебное издание

Гаркуша Олег Васильевич

UNITY В КАРТИНКАХ

Учебное пособие

Подписано в печать 20.11.2024. Выход в свет 6.12.2024.

Печать цифровая. Формат 60×84 ¹/₁₆. Уч.-изд. л. 10,4.

Тираж 500 экз. Заказ № 5894

Кубанский государственный университет
350040, г. Краснодар, ул. Ставропольская, 149.

Издательско-полиграфический центр
Кубанского государственного университета
350040, г. Краснодар, ул. Ставропольская, 149.